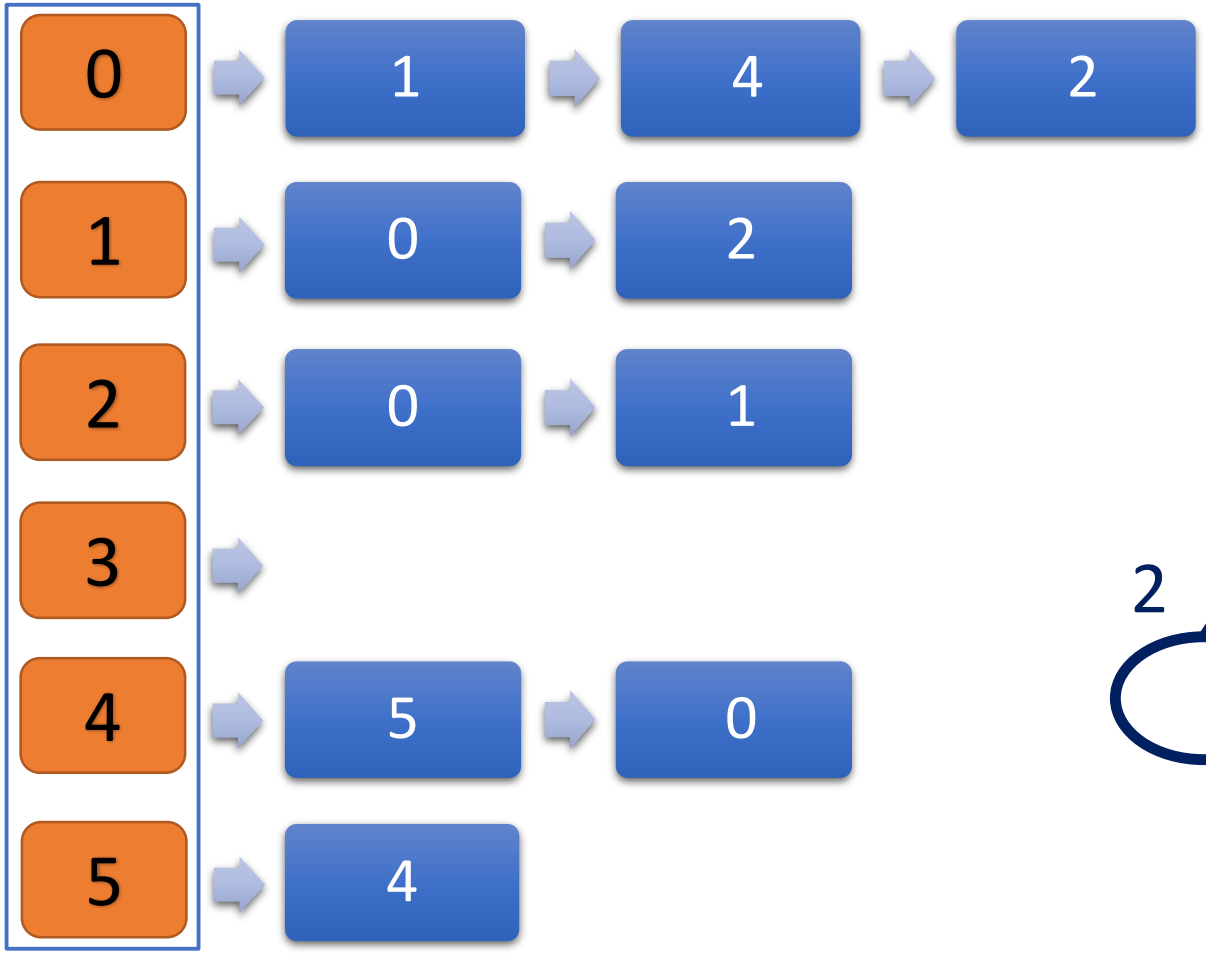
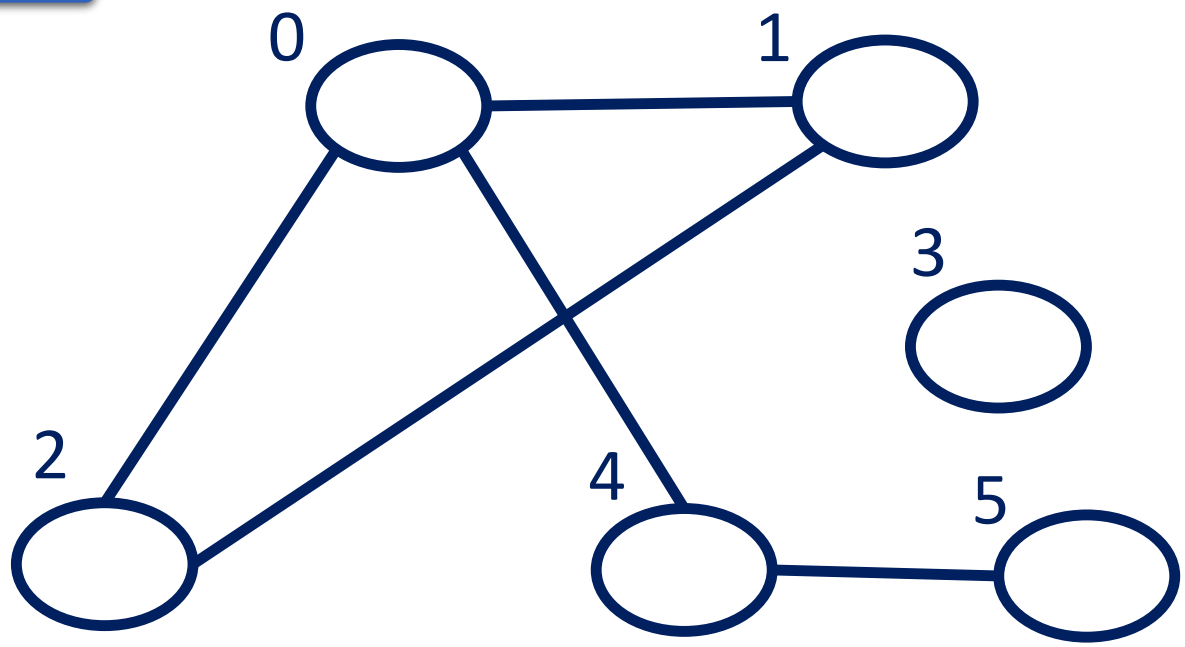
Graphs (cont.)

Adjacency list

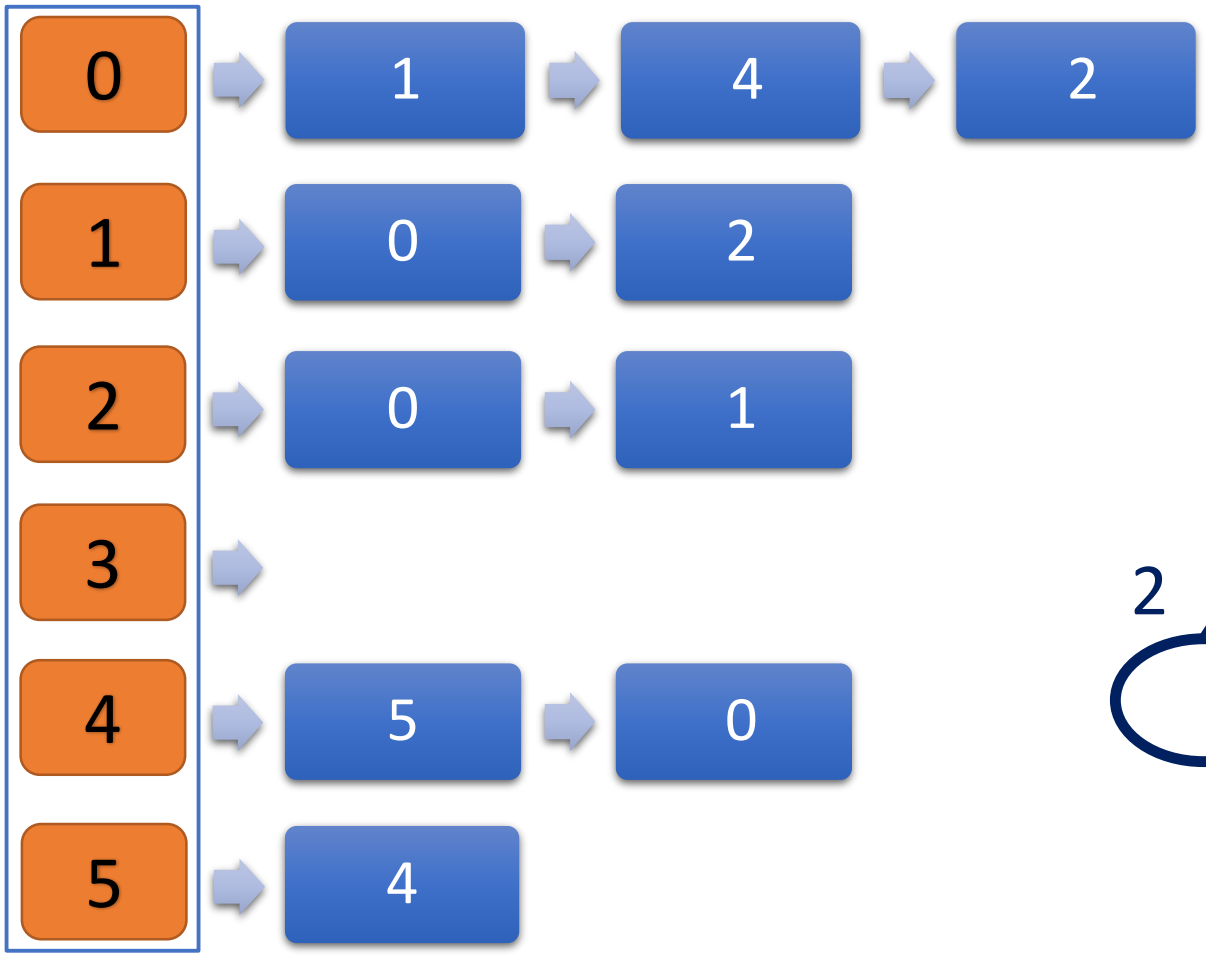
- A collection (an array / a hash map) of (linked) lists / arrays
- Each list
  - ✓ corresponds to a node
  - ✓ stores a list of nodes that are adjacent to the node

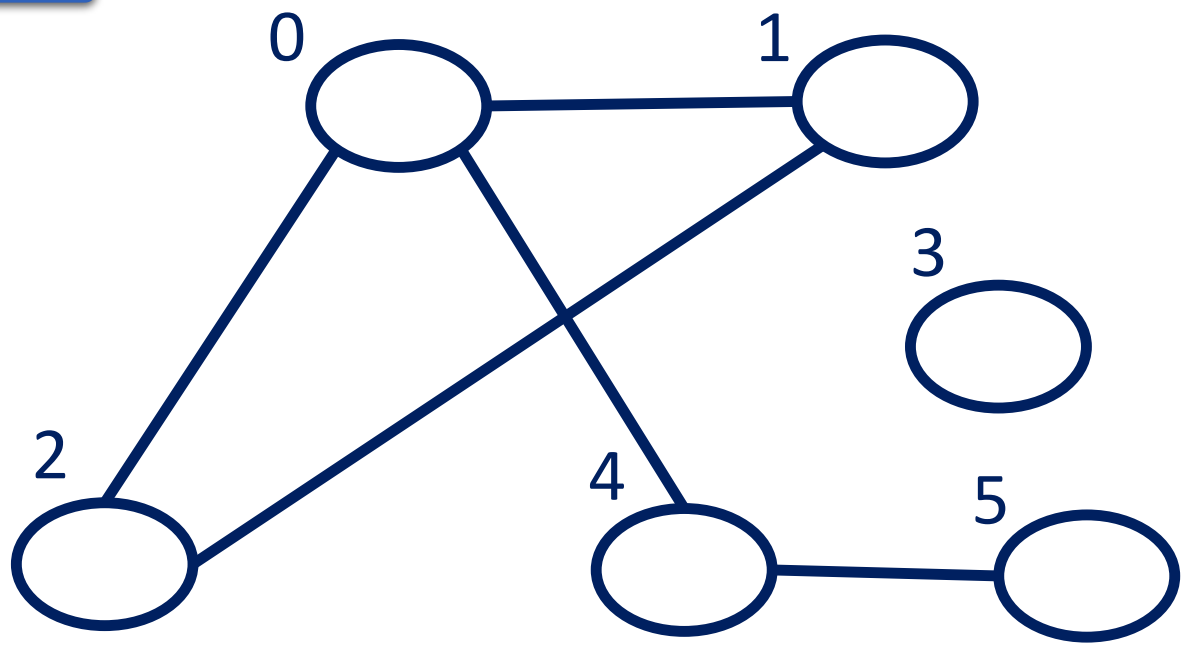
Adjacency list: example



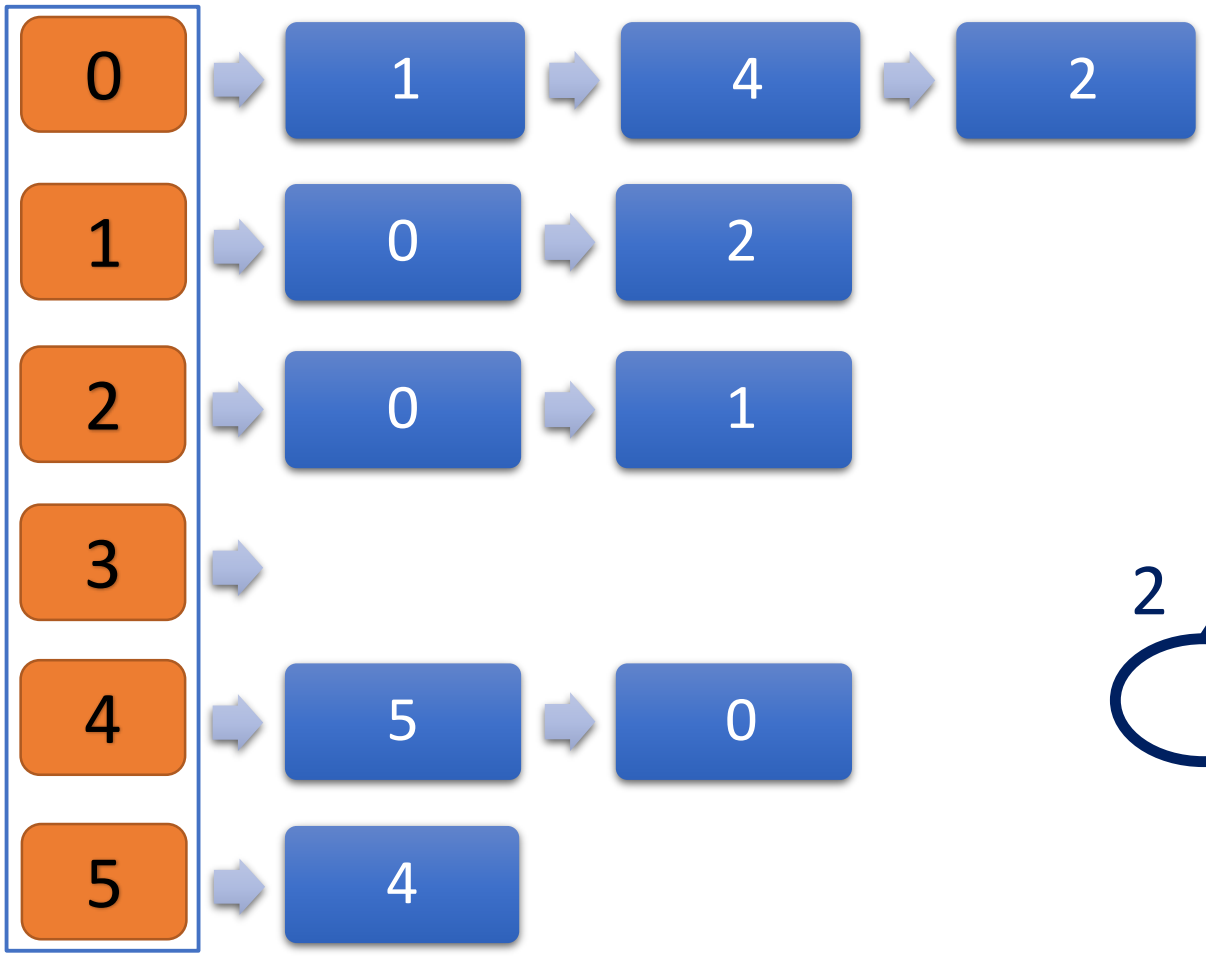


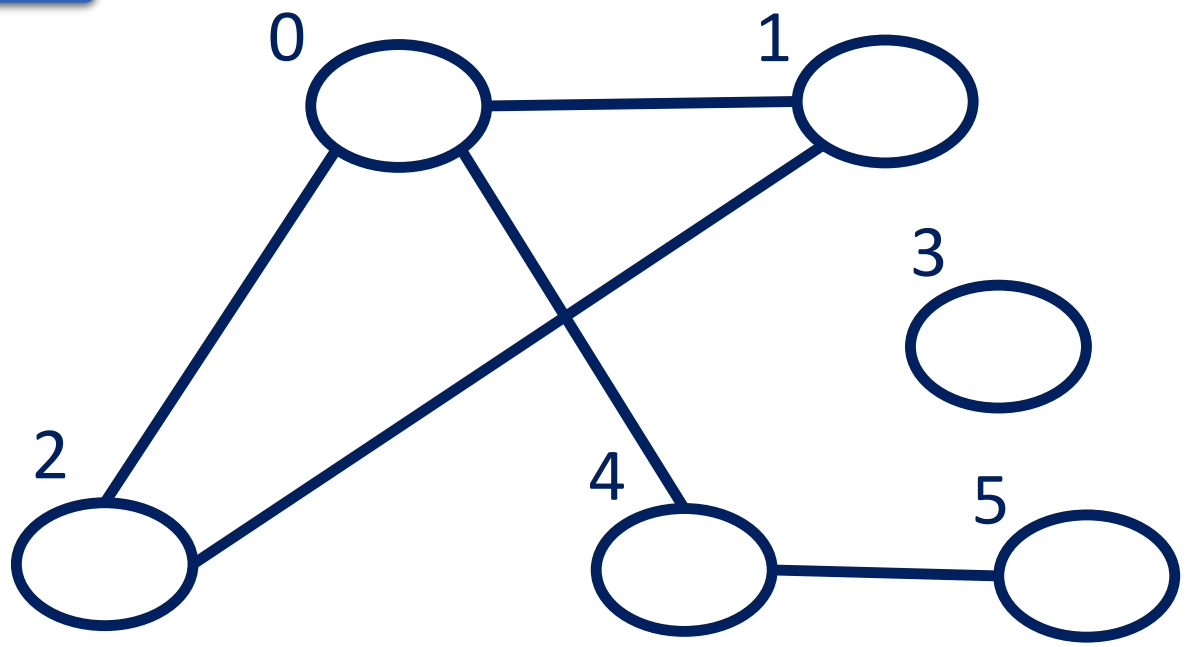
Add a node – O(1)



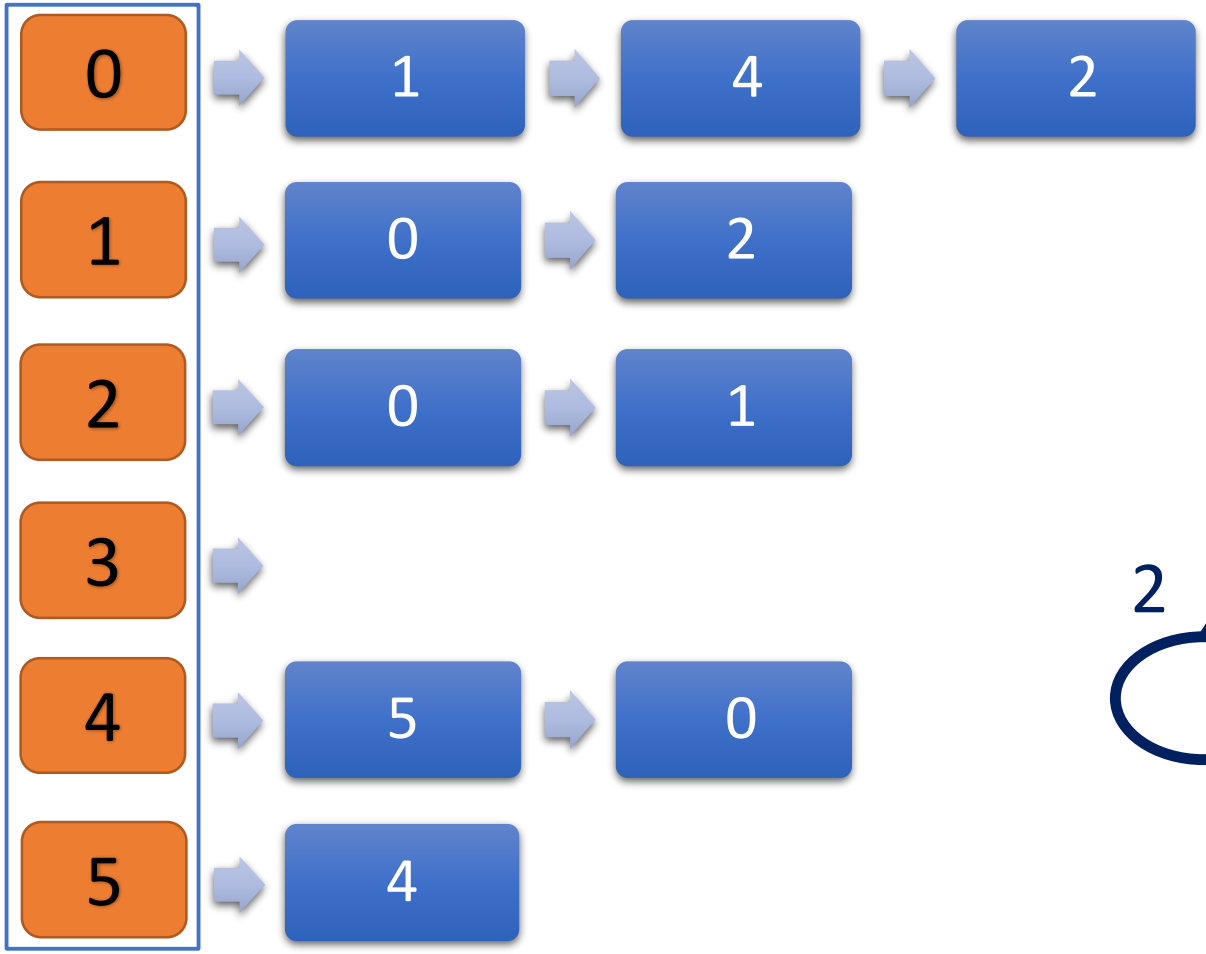


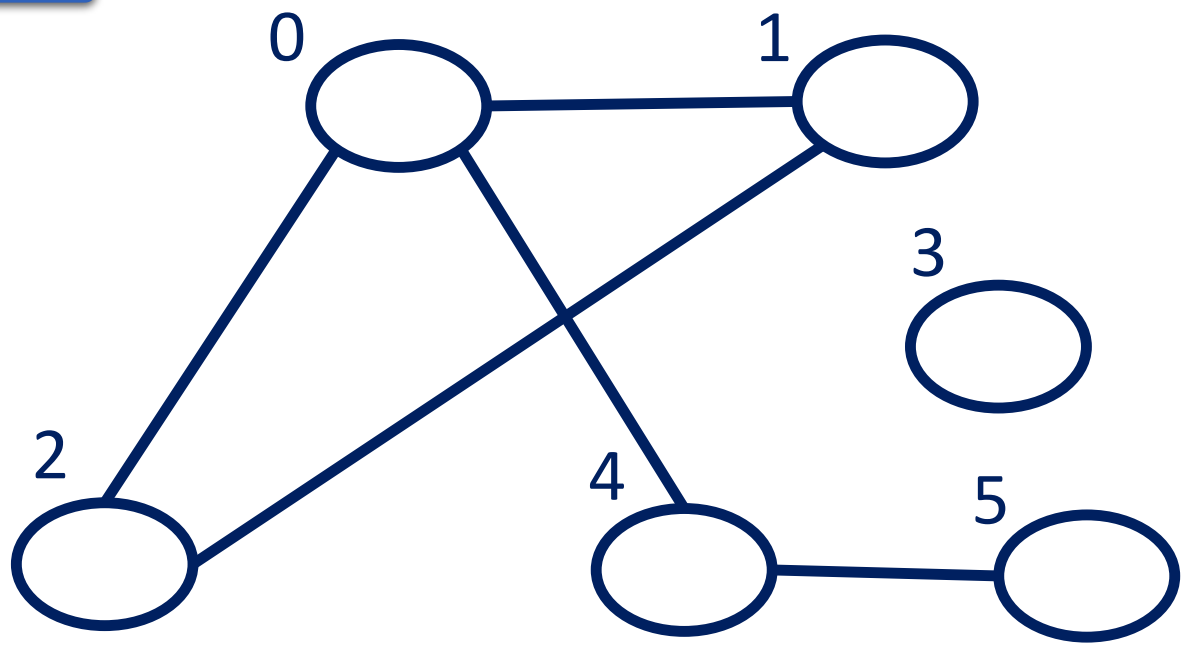
Remove a node – O(m)



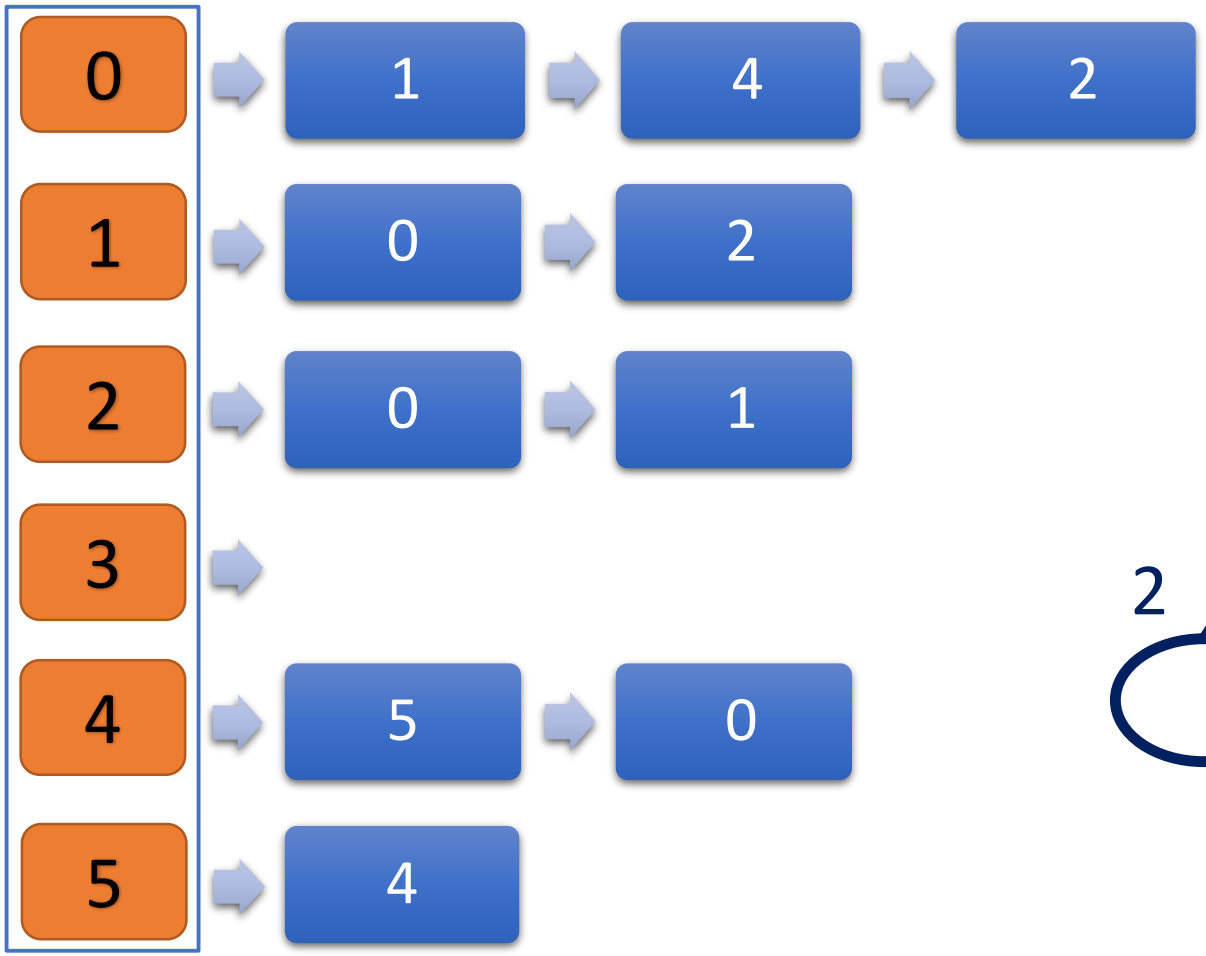


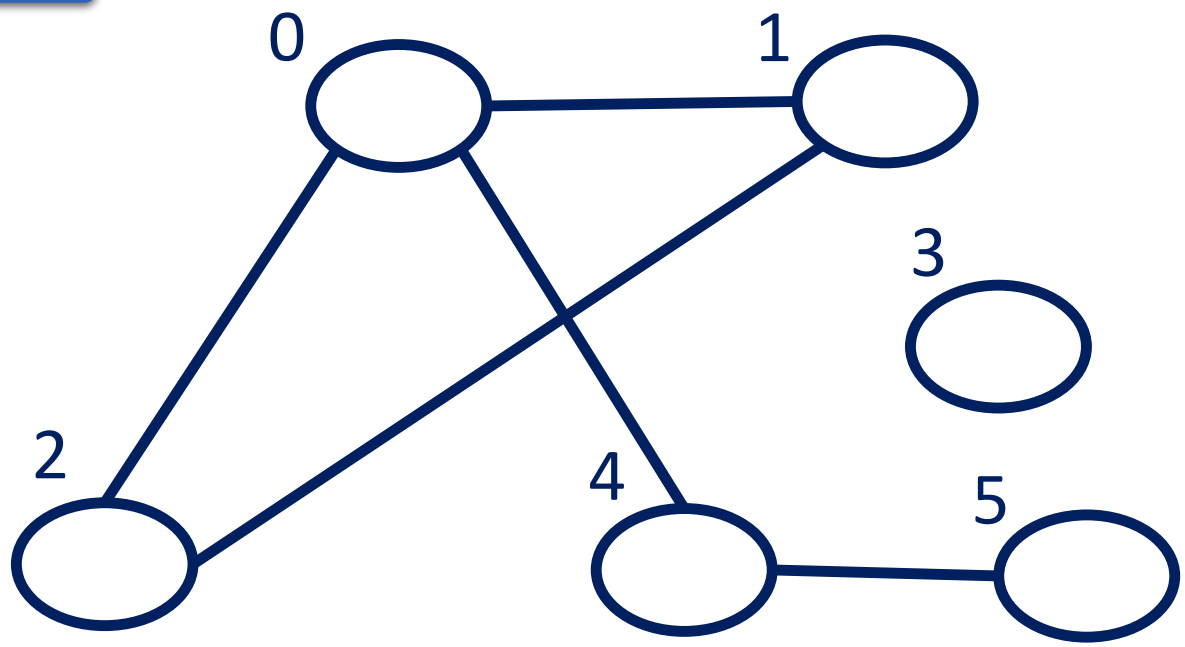
Add an edge – O(1)



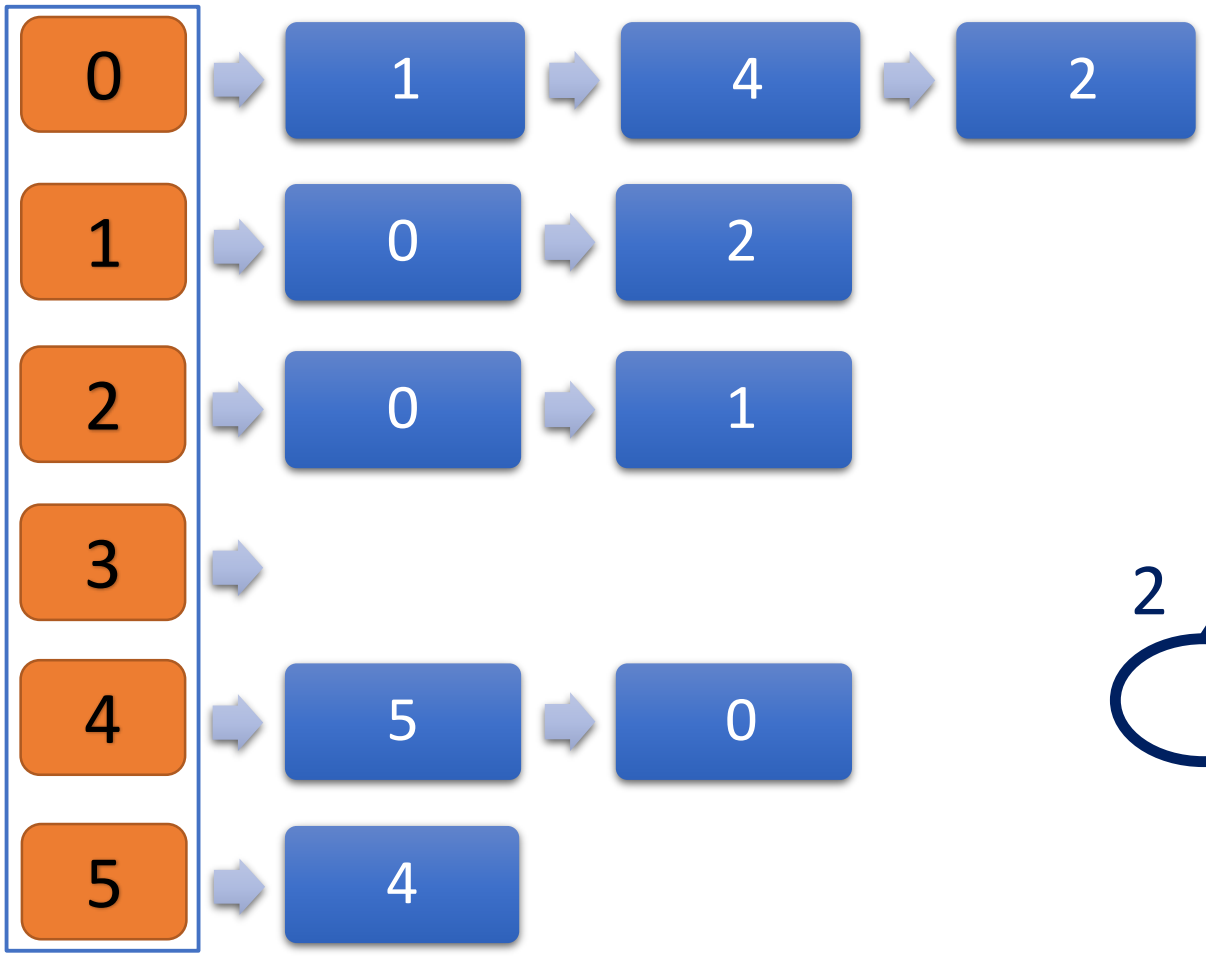


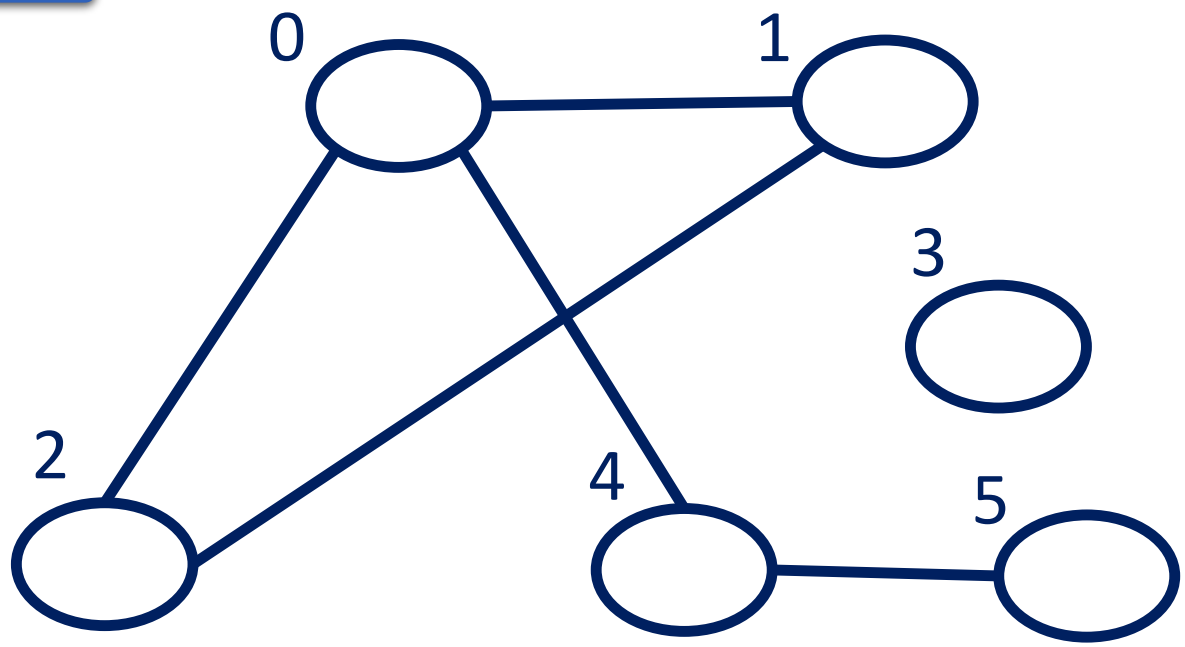
Remove an edge – O(n)



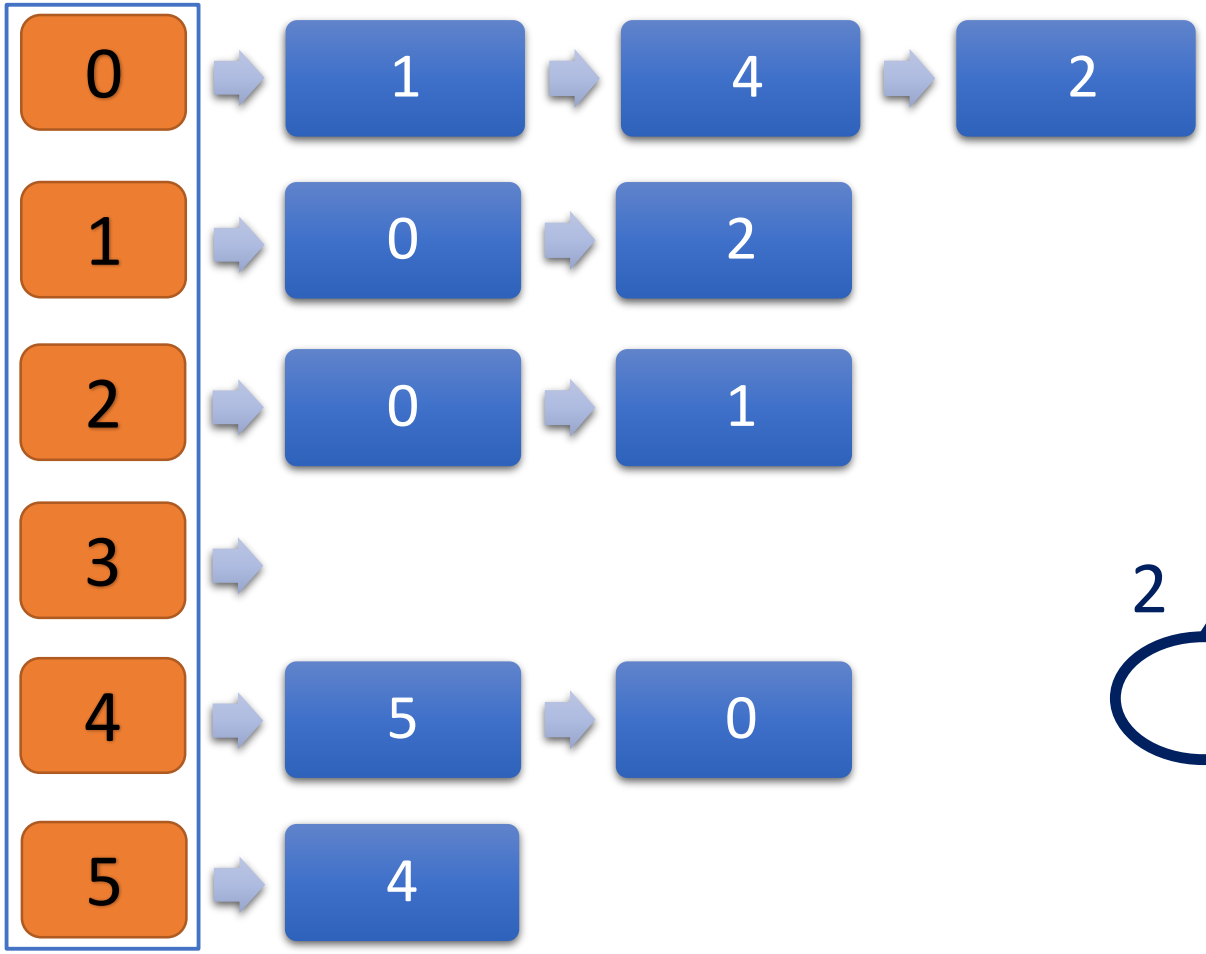


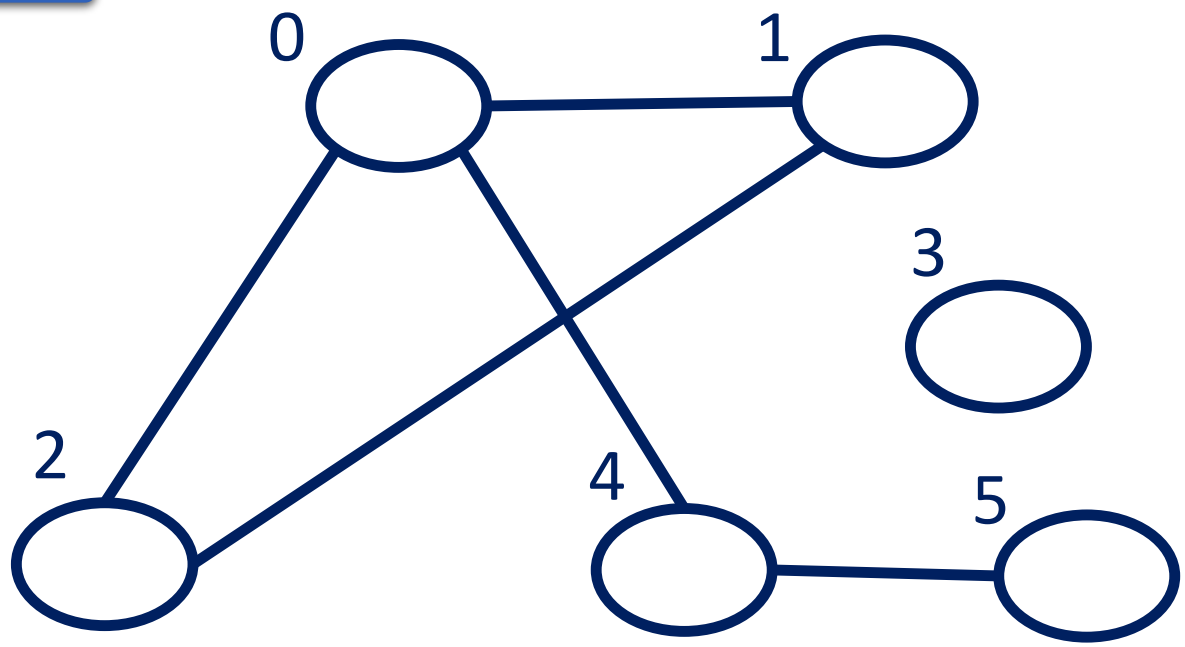
Are two nodes adjacent – O(n)





Find neighbors of a node – O(n)





Adj List - Complexity Summary

- ❑ Time

$O(1)$ ✓ Add a node

$O(m)$ ✓ Remove a node

$O(1)$ ✓ Add an edge

$O(n)$ ✓ Remove an edge

$O(n)$ ✓ Are two nodes adjacent?

$O(n)$ ✓ Find neighbors of a node

- ❑ Space

➢ $O(n + m)$

Time complexity: summary

	Adjacency matrix	Adjacency list
Store a whole graph	$O(n^2)$	$O(n+m)$
Add a node	$O(n^2)$	$O(1)$
Remove a node	$O(n^2)$	$O(m)$
Add an edge	$O(1)$	$O(1)$
Remove an edge	$O(1)$	$O(n)$
Are two nodes adjacent?	$O(1)$	$O(n)$
Find neighbors of a node	$O(n)$	$O(n)$
Comments	Slow to **add/remove nodes** as the matrix must be resized/copied	Slow to **remove nodes and edges** because it must find all nodes/edges

Graph as adjacency matrix

The class

```
class GraphOnAdjacencyMatrix:
    def __init__(self):
        # Dictionary to store node labels and their indices
        self.nodes = {}

        self.n = 0  # Number of nodes

        # Adjacency matrix (a list of lists)
        self.matrix = []
```

Add a node

```
def add_node(self, label):
    if label not in self.nodes:
        self.nodes[label] = self.n
        self.n += 1

        # Add a 0 to each existing row in the adjacency matrix
        for row in self.matrix:
            row.append(0)

        # Add a new row of 0s to represent the new node
        self.matrix.append([0] * (self.n))
```

Remove a node

```
def remove_node(self, label):
    idx_to_remove = self.nodes.get(label)
    if idx_to_remove is None:
        return

    # Adjust indices in the nodes dictionary
    for node, idx in list(self.nodes.items()):
        if idx > idx_to_remove:
            self.nodes[node] = idx - 1

    # Remove the column and row for the node in the adjacency matrix
    for row in self.matrix:
        row.pop(idx_to_remove)
    self.matrix.pop(idx_to_remove)

    # Remove the node and decrement node count
    self.nodes.pop(label)
    self.n -= 1
```

Add an edge

```
def add_edge(self, u, v): # Note: u and v are labels
    i = self.nodes.get(u)
    j = self.nodes.get(v)

    if i is None or j is None:
        raise ValueError("No such node exists!")

    # Set the edges in the adjacency matrix
    self.matrix[i][j] = 1
    self.matrix[j][i] = 1
```

Remove an edge

```
def remove_edge(self, u, v):
    i = self.nodes.get(u)
    j = self.nodes.get(v)

    if idx_one is None or idx_two is None:
        return

    # Remove the edges in the adjacency matrix
    self.matrix[i][j] = 0
    self.matrix[j][i] = 0
```

Are two nodes adjacent?

```
def are_adjacent(self, u, v):
    i = self.nodes.get(u)
    j = self.nodes.get(v)

    if i is None or j is None:
        return False

    return self.matrix[i][j] != 0
```

Print adj matrix

```
def print_adj_matrix(self):
    for row in self.matrix:
        print(row)
    print()
```

Adjacency list as a dictionary of arrays

Graph – Adjacency List Based

```
class Graph:
    def __init__(self):
        # A dictionary to store adjacency list
        #(node -> list of nodes/ neighbors)
        self.adj = {}
```

Add and remove a node

```
def add_node(self, label):
    if label not in self.adj:
        self.adj[label] = []

def remove_node(self, label):
    if label not in self.adj:
        return
    for node in self.adj:
        if label in self.adj[node]:
            self.adj[node].remove(label)

    self.adj.pop(label, None)
```

Add an edge

```
def add_edge(self, u, v):
    if u not in self.adj or v not in self.adj:
        raise ValueError("One or both nodes do not exist!")

    if v not in self.adj[u]:
        self.adj[u].append(v)
    if u not in self.adj[v]:
        self.adj[v].append(u)
```

Remove an edge

```
def remove_edge(self, u, v):
    if u not in self.adj or v not in self.adj:
        raise ValueError("One or both nodes do not exist!")

    if v in self.adj[u]:
        self.adj[u].remove(v)
    if u in self.adj[v]:
        self.adj[v].remove(u)
```

Are two nodes adjacent?

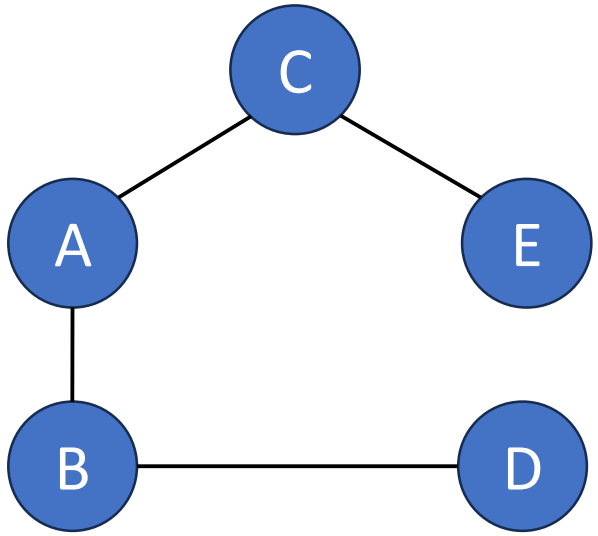
```
def are_adjacent(self, u, v):
    if u not in self.adj or v not in self.adj:
        raise ValueError("One or both nodes do not exist!")

    return v in self.adj[u]
```

Print Adjacency List

```
def print_adj_list(self):
    for node, neighbors in self.adj.items():
        print(f"Neighbors of {node}: {self.adj[node]}")
```

Driver code / Example Usage

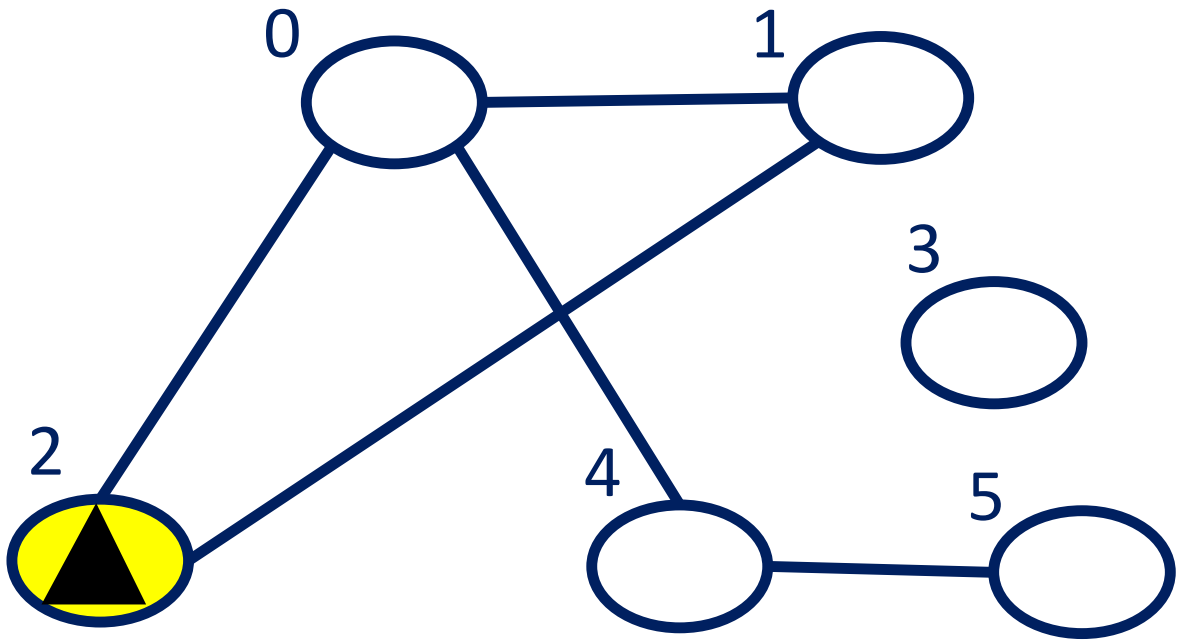
```
graph = Graph()
for label in ['A','B','C','D','E']:
    graph.add_node(label)

graph.add_edge('A','B')
graph.add_edge('A','C')
graph.add_edge('B','D')
graph.add_edge('C','E')
graph.print_adj_list()
```



Depth first search (DFS)

- Explores as far as possible along each branch before backtracking
- A main procedure for many graph algorithms
- Several versions exist

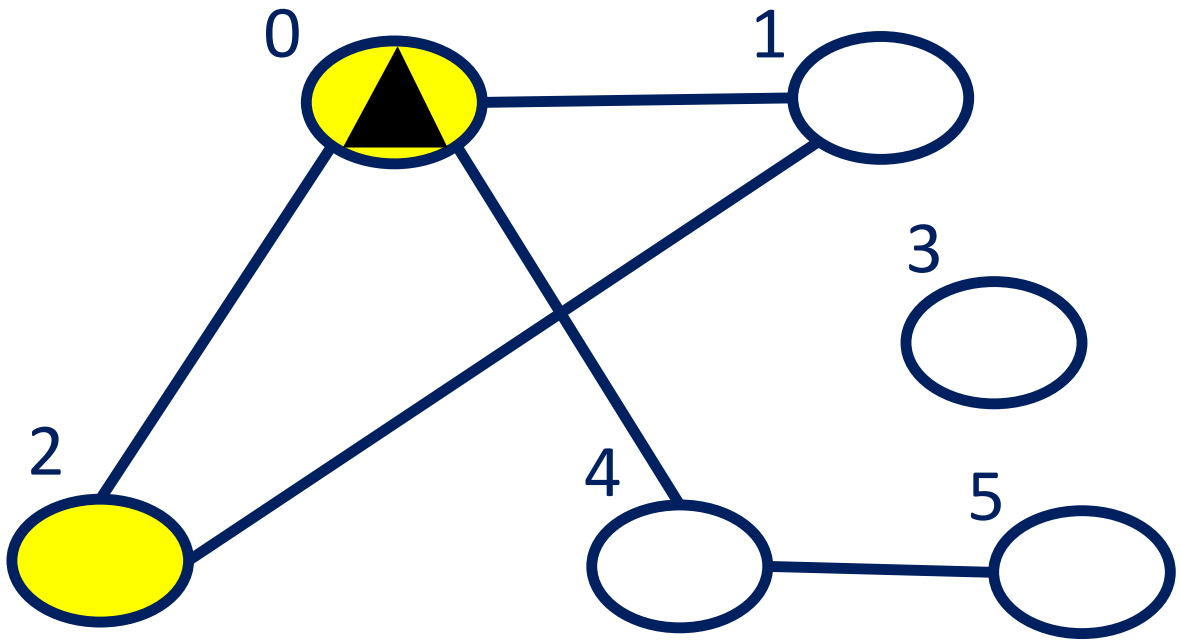
- Full path: 2
- Short path: 2



Depth first search

- Explores as far as possible along each branch before backtracking
- A main procedure for many graph algorithms
- Several versions exist

- Full path: 2,0
- Short path: 2,0

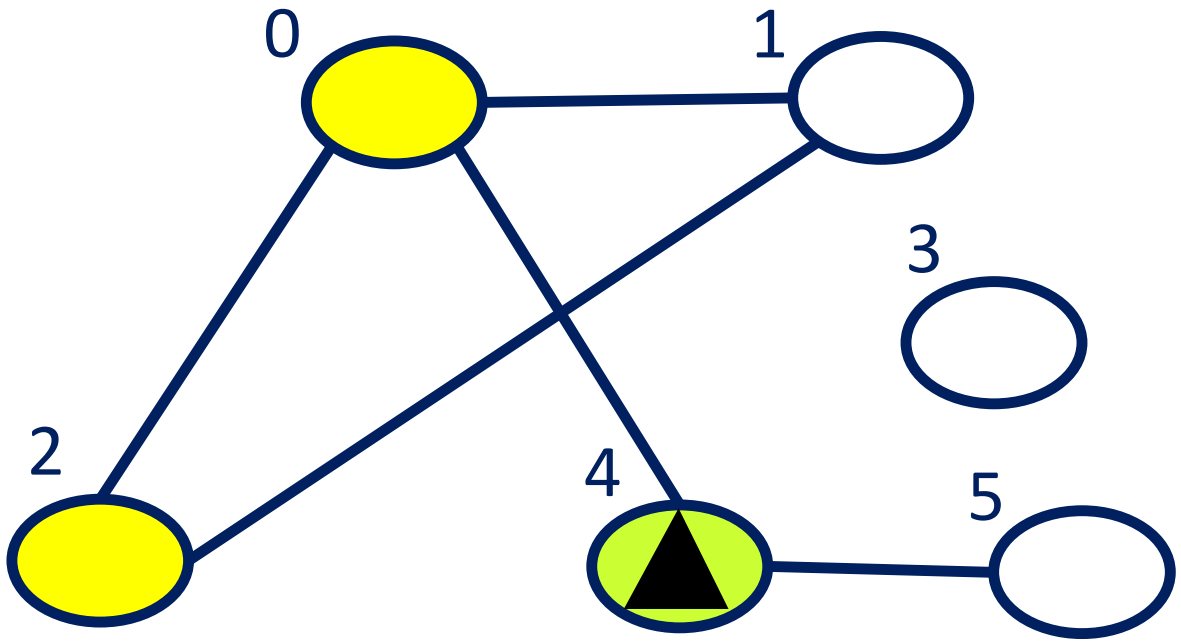


Depth first search

- Explores as far as possible along each branch before backtracking
- A main procedure for many graph algorithms
- Several versions exist

- Full path: 2,0,4

- Short path: 2,0,4

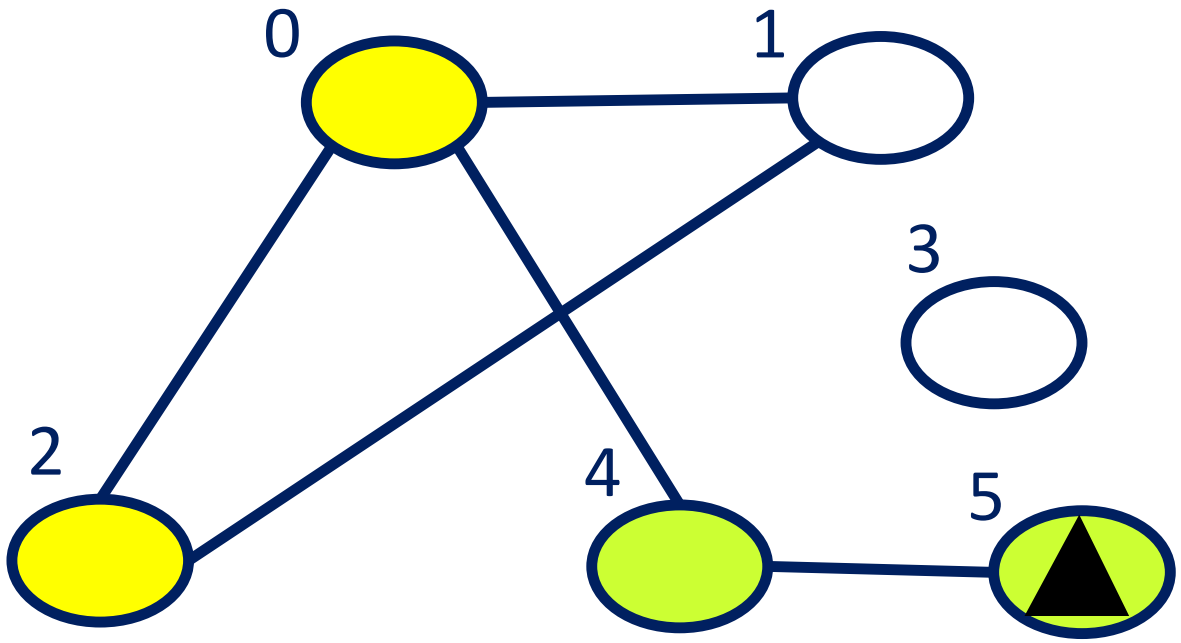


Depth first search

- Explores as far as possible along each branch before backtracking
- A main procedure for many graph algorithms
- Several versions exist

- Full path: 2,0,4,5

- Short path: 2,0,4,5

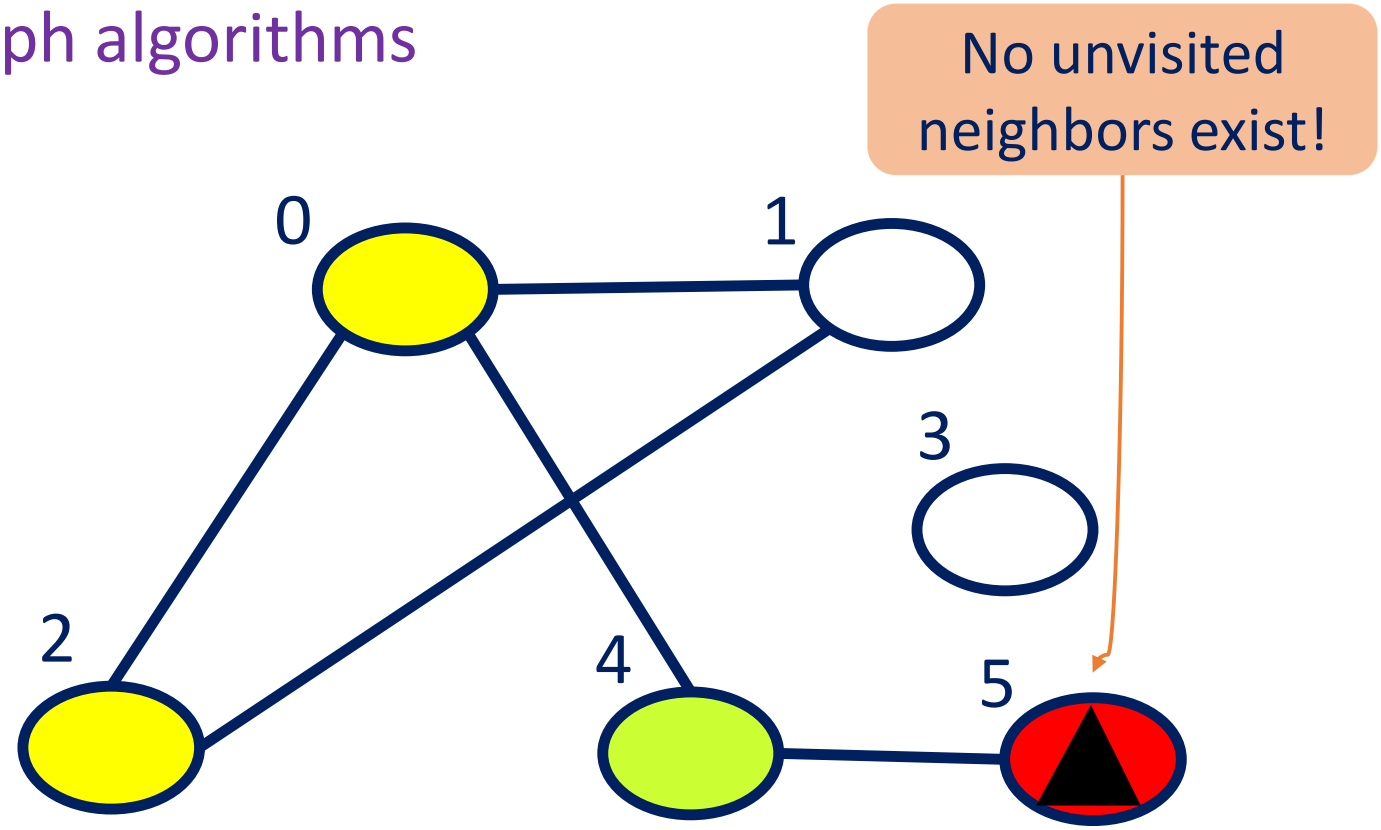


Depth first search

- Explores as far as possible along each branch before backtracking
- A main procedure for many graph algorithms
- Several versions exist

- Full path: 2,0,4,5

- Short path: 2,0,4,5

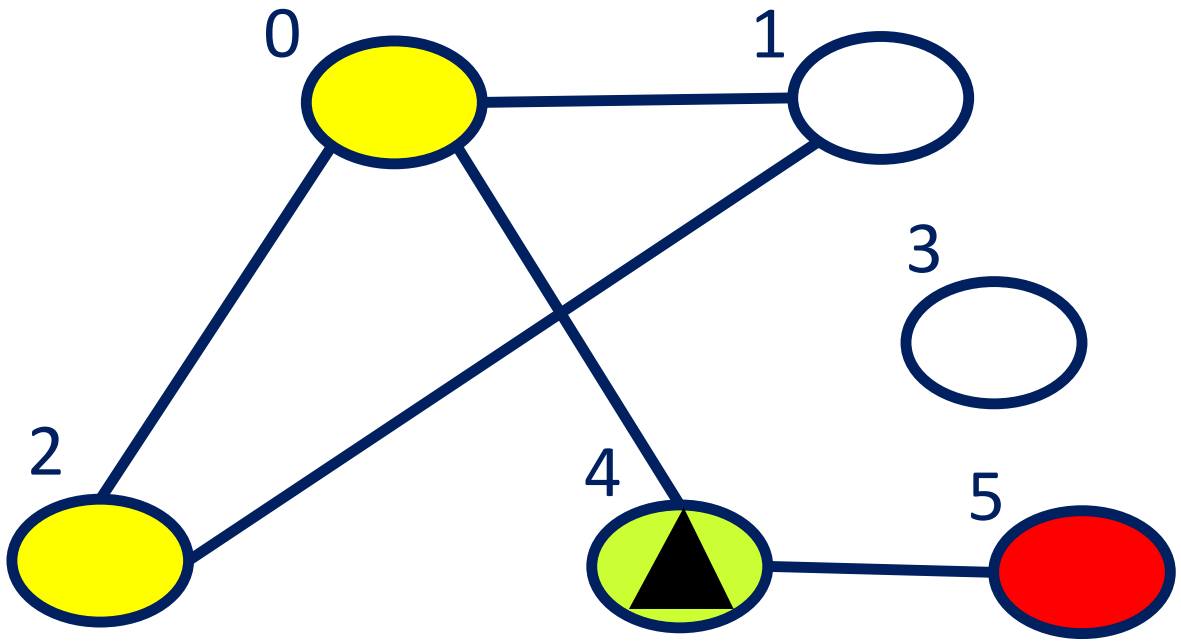


Depth first search

- Explores as far as possible along each branch before backtracking
- A main procedure for many graph algorithms
- Several versions exist

- Full path: 2,0,4,5,4

- Short path: 2,0,4,5

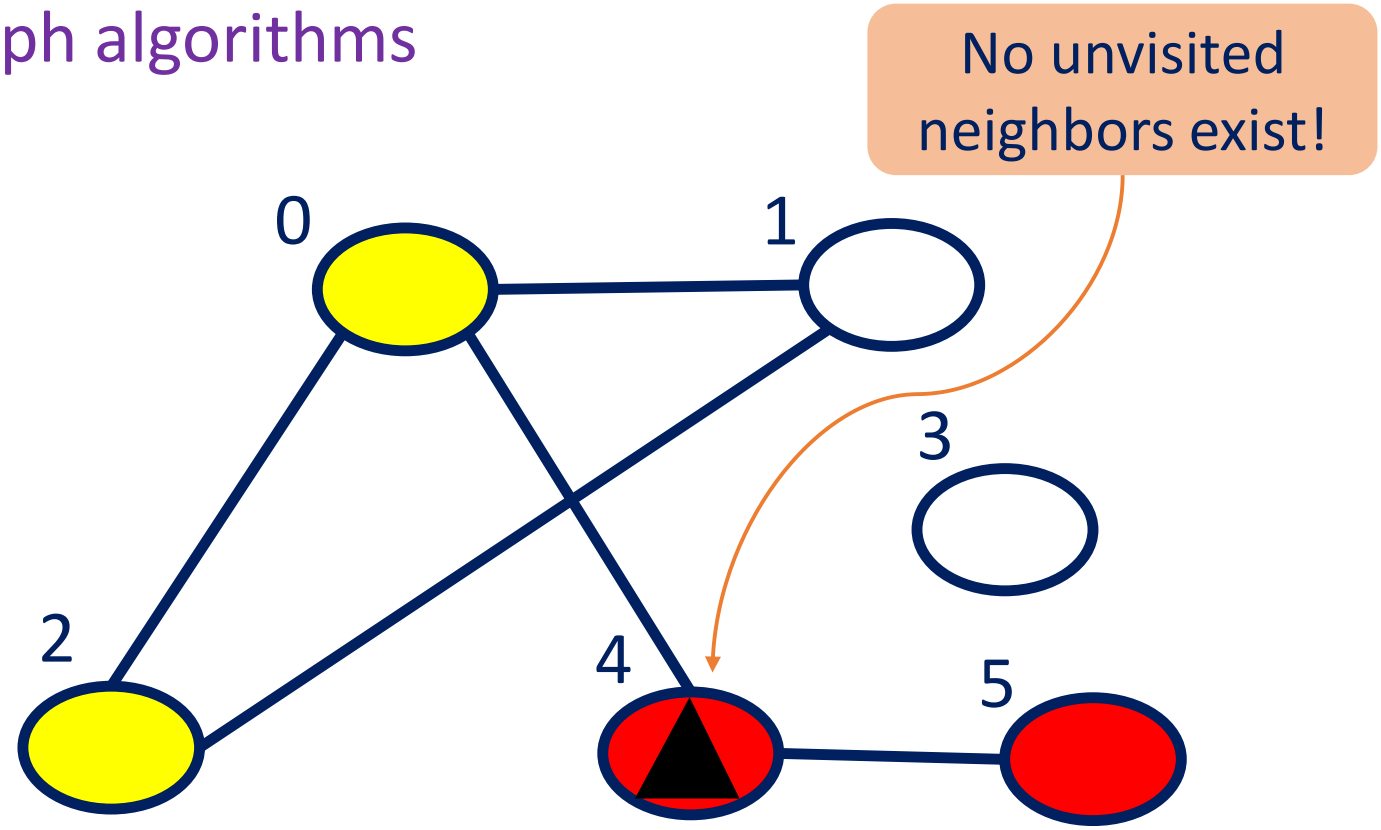


Depth first search

- Explores as far as possible along each branch before backtracking
- A main procedure for many graph algorithms
- Several versions exist

- Full path: 2,0,4,5,4

- Short path: 2,0,4,5

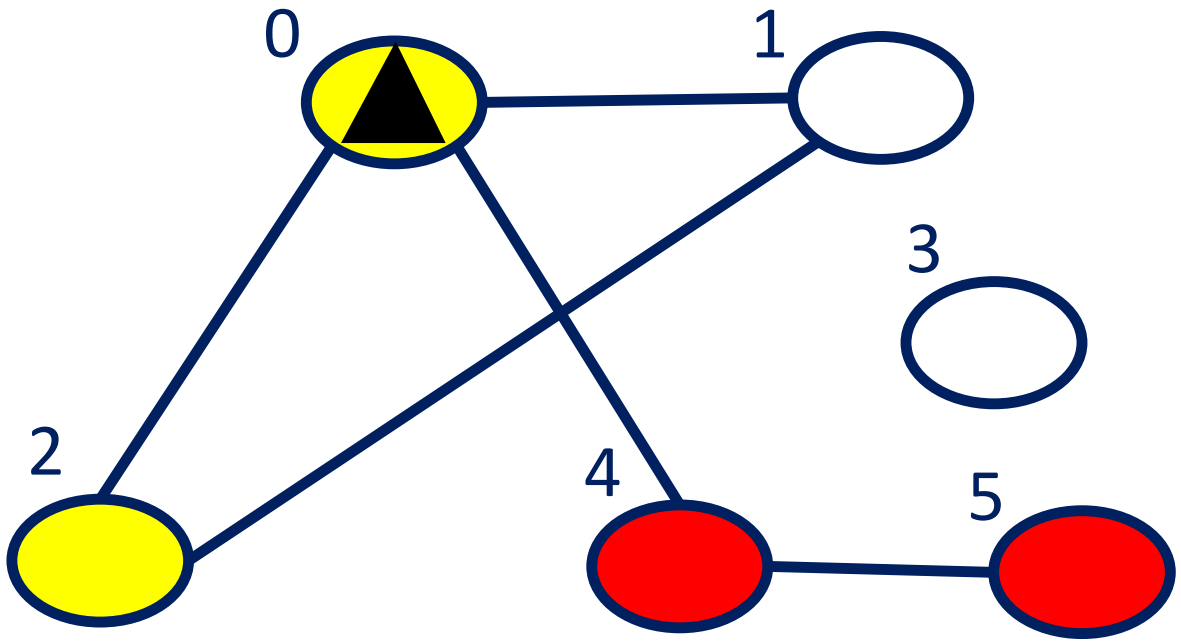


Depth first search

- Explores as far as possible along each branch before backtracking
- A main procedure for many graph algorithms
- Several versions exist

- Full path: 2,0,4,5,4,0

- Short path: 2,0,4,5

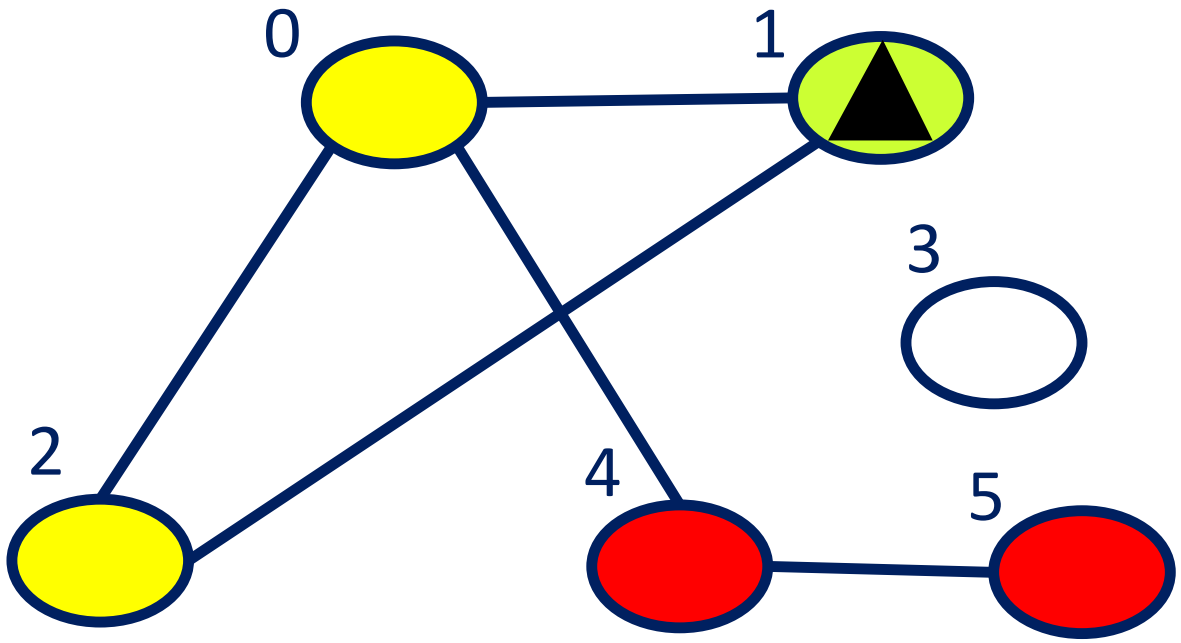


Depth first search

- Explores as far as possible along each branch before backtracking
- A main procedure for many graph algorithms
- Several versions exist

- Full path: 2,0,4,5,4,0,1

- Short path: 2,0,4,5,1

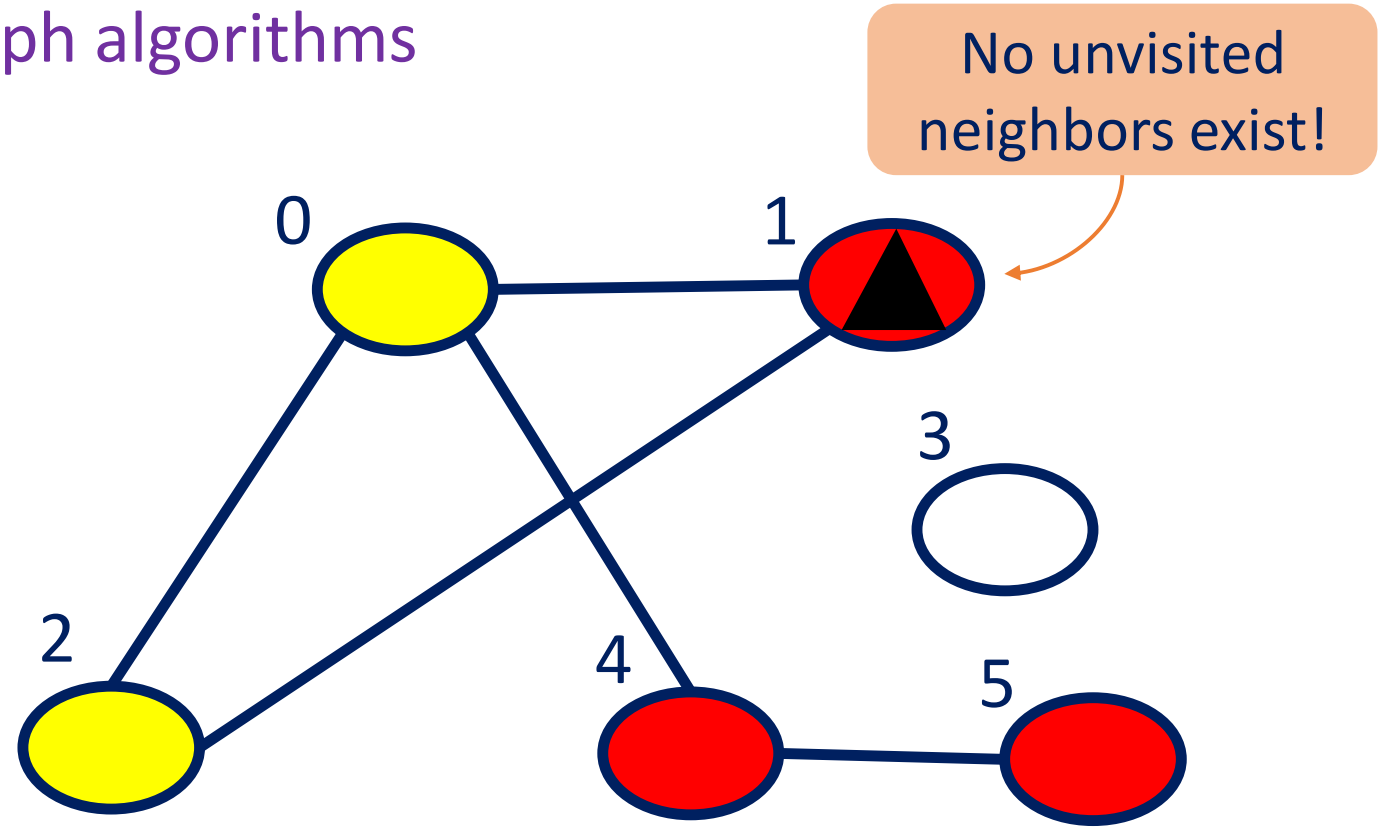


Depth first search

- Explores as far as possible along each branch before backtracking
- A main procedure for many graph algorithms
- Several versions exist

- Full path: 2,0,4,5,4,0,1

- Short path: 2,0,4,5,1

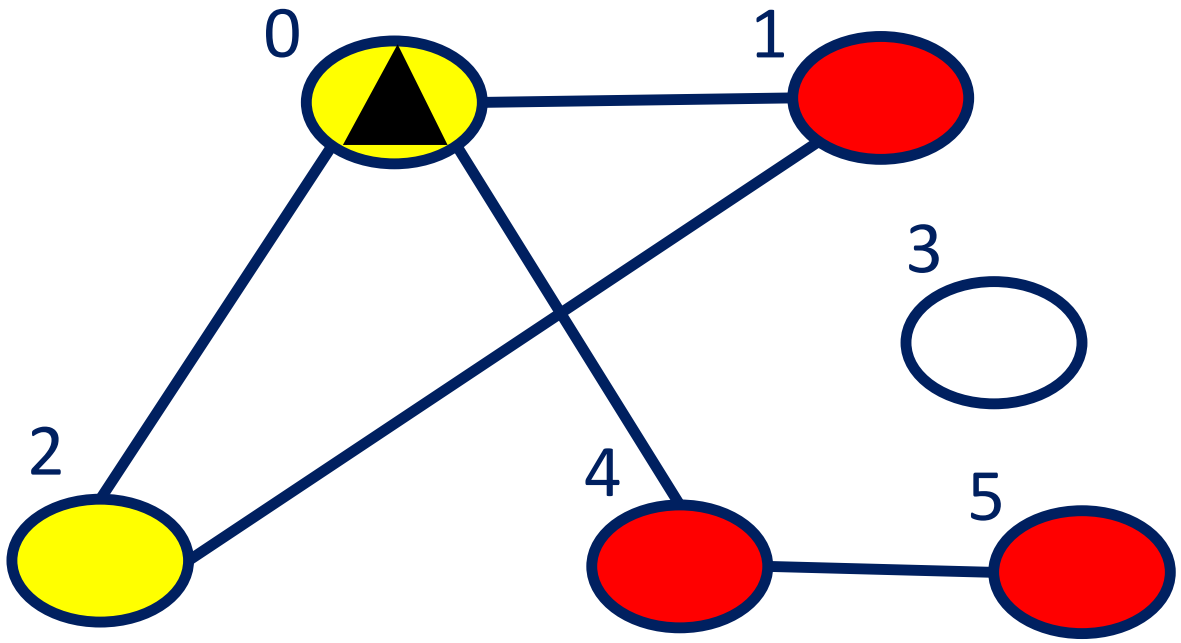


Depth first search

- Explores as far as possible along each branch before backtracking
- A main procedure for many graph algorithms
- Several versions exist

- Full path: 2,0,4,5,4,0,1,0

- Short path: 2,0,4,5,1

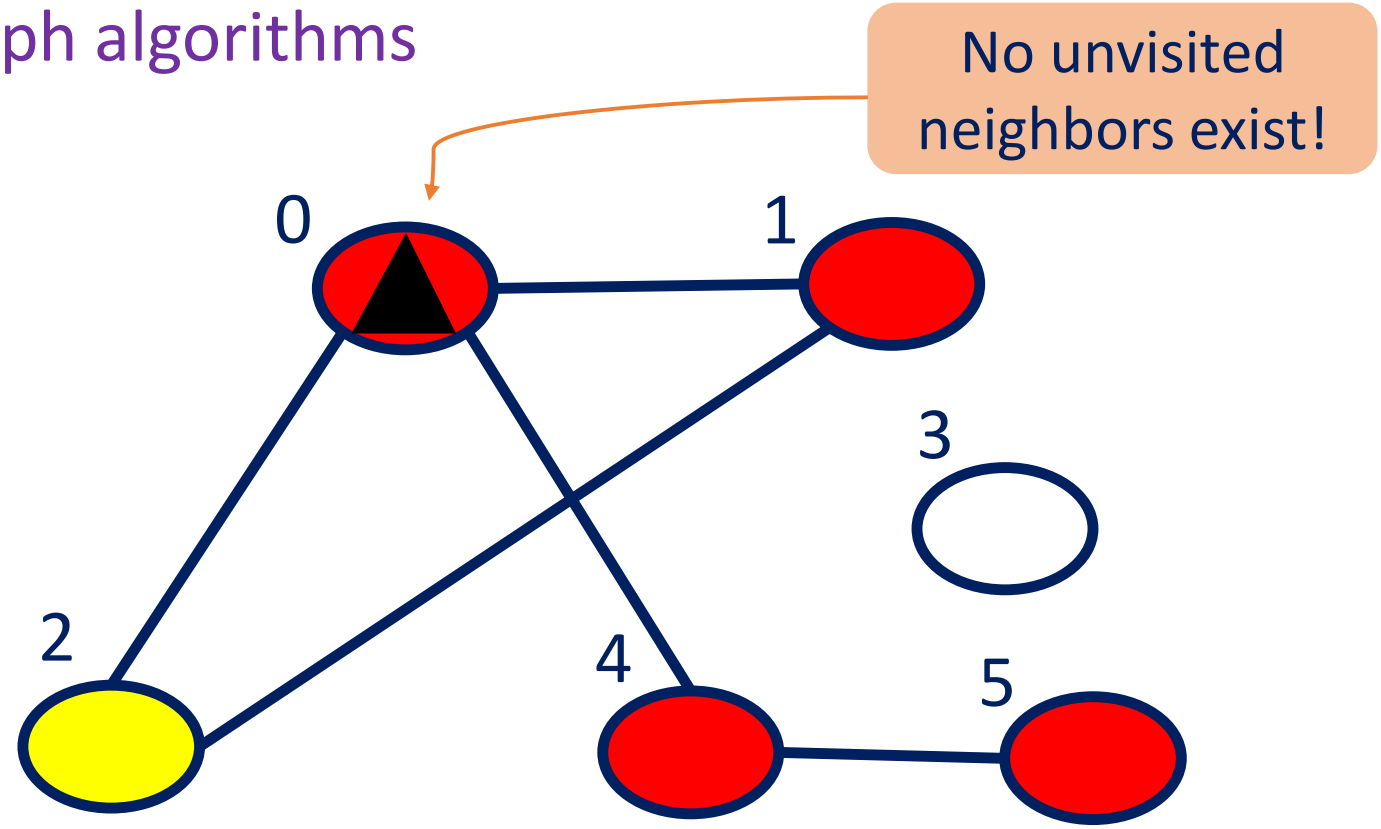


Depth first search

- Explores as far as possible along each branch before backtracking
- A main procedure for many graph algorithms
- Several versions exist

- Full path: 2,0,4,5,4,0,1,0

- Short path: 2,0,4,5,1

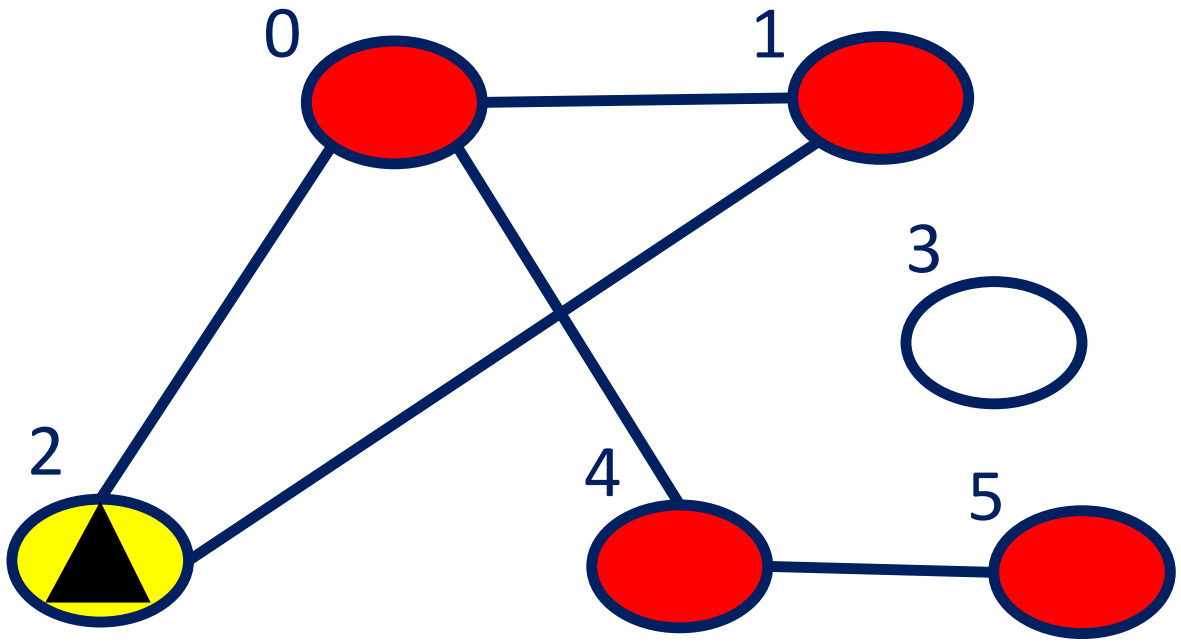


Depth first search

- Explores as far as possible along each branch before backtracking
- A main procedure for many graph algorithms
- Several versions exist

- Full path: 2,0,4,5,4,0,1,0,2

- Short path: 2,0,4,5,1

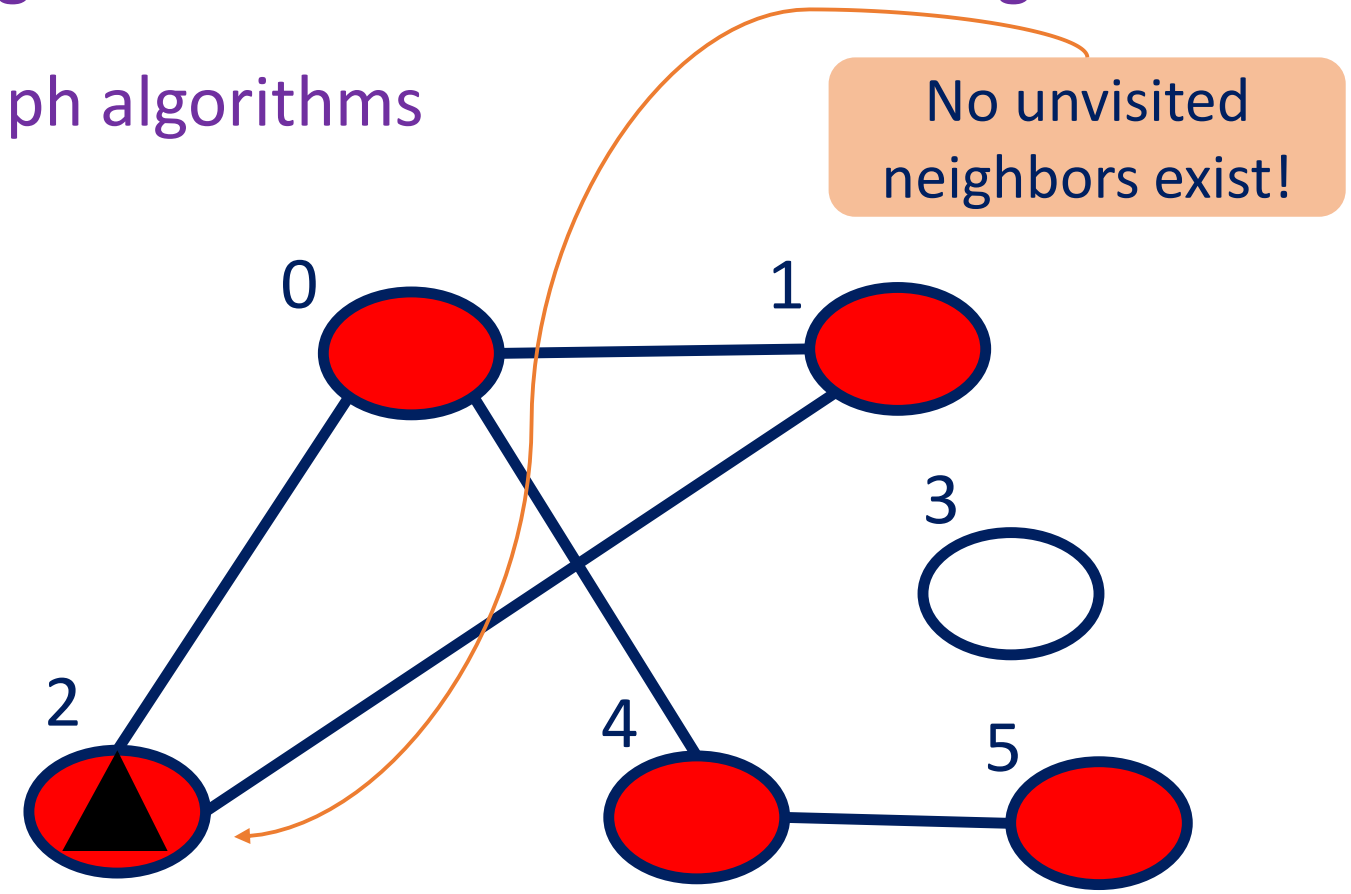


Depth first search

- Explores as far as possible along each branch before backtracking
- A main procedure for many graph algorithms
- Several versions exist

- Full path: 2,0,4,5,4,0,1,0,2

- Short path: 2,0,4,5,1

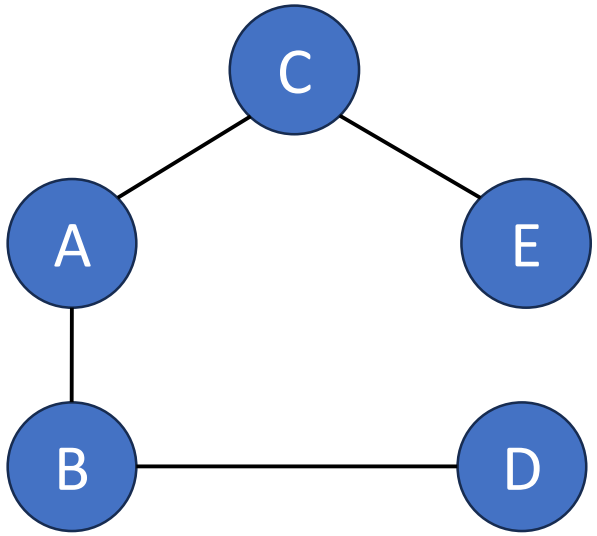


DFS: recursive implementation

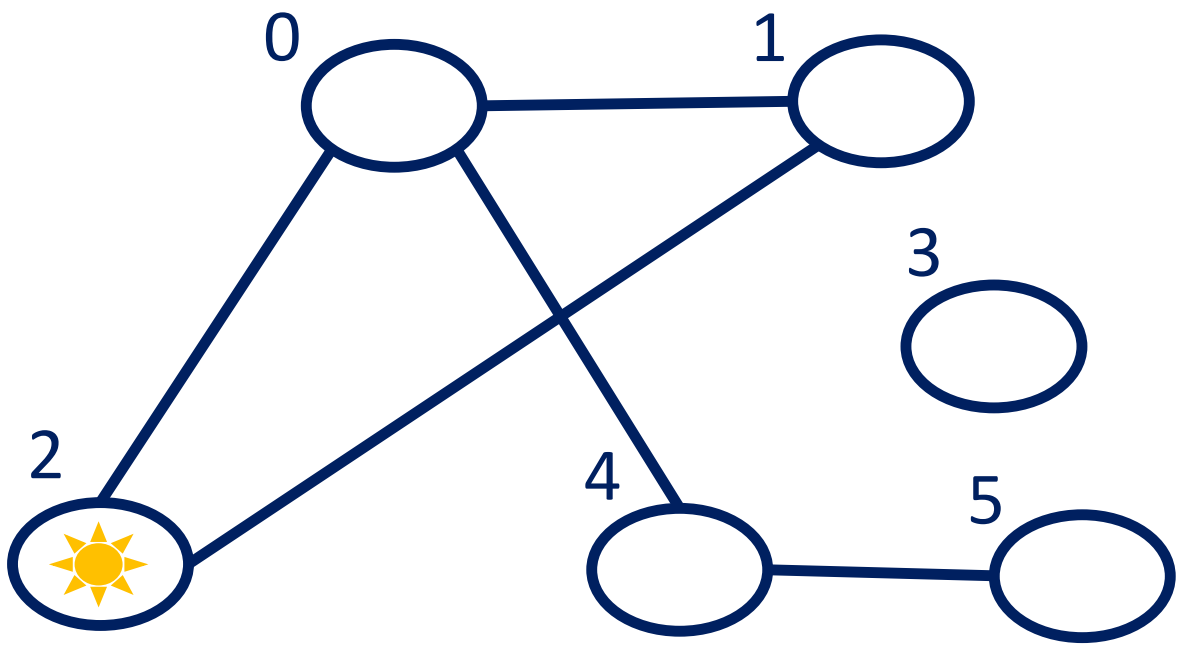
```
def dfs_recursive(self, start, visited):
    visited.add(start)
    print(start, end=' ')

    #Recur for all adjacent nodes
    for neighbor in self.adj[start]:
        if neighbor not in visited:
            self.dfs_recursive(neighbor, visited)
```

DFS: recursive implementation

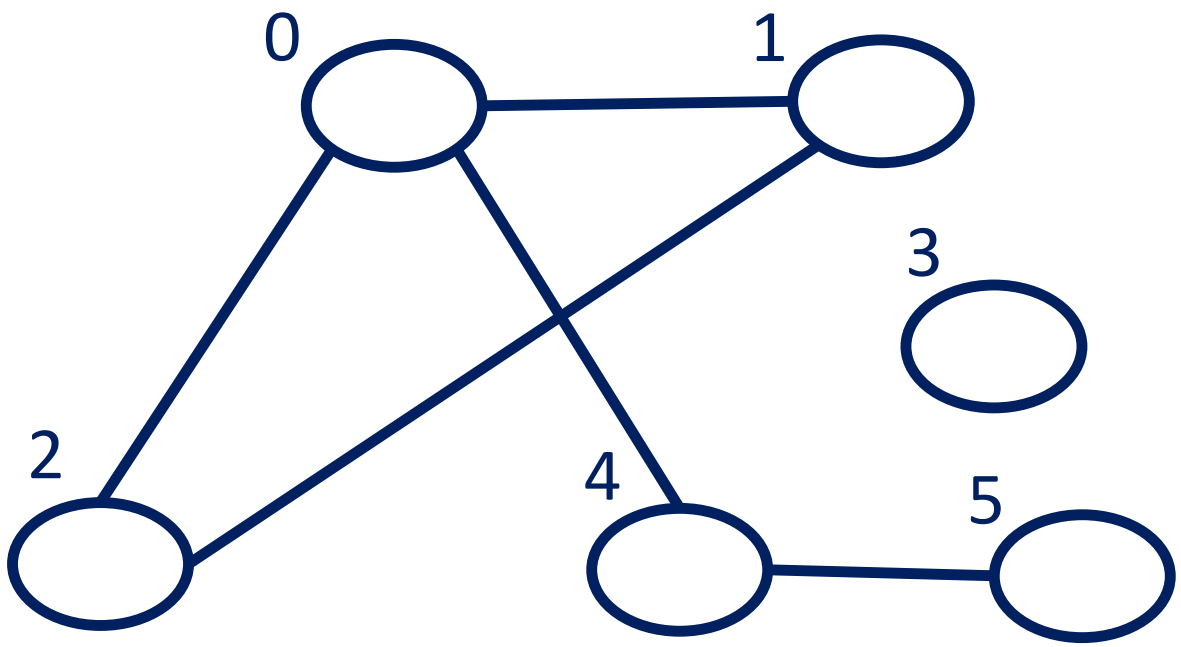
```
# Driver code
graph = Graph()
for label in ['A','B','C','D','E']:
    graph.add_node(label)

graph.add_edge('A','B')
graph.add_edge('A','C')
graph.add_edge('B','D')
graph.add_edge('C','E')
print('DFS traversal result (start = B):')
graph.dfs_recursive('B', set())
```



Depth First Search: Iterative



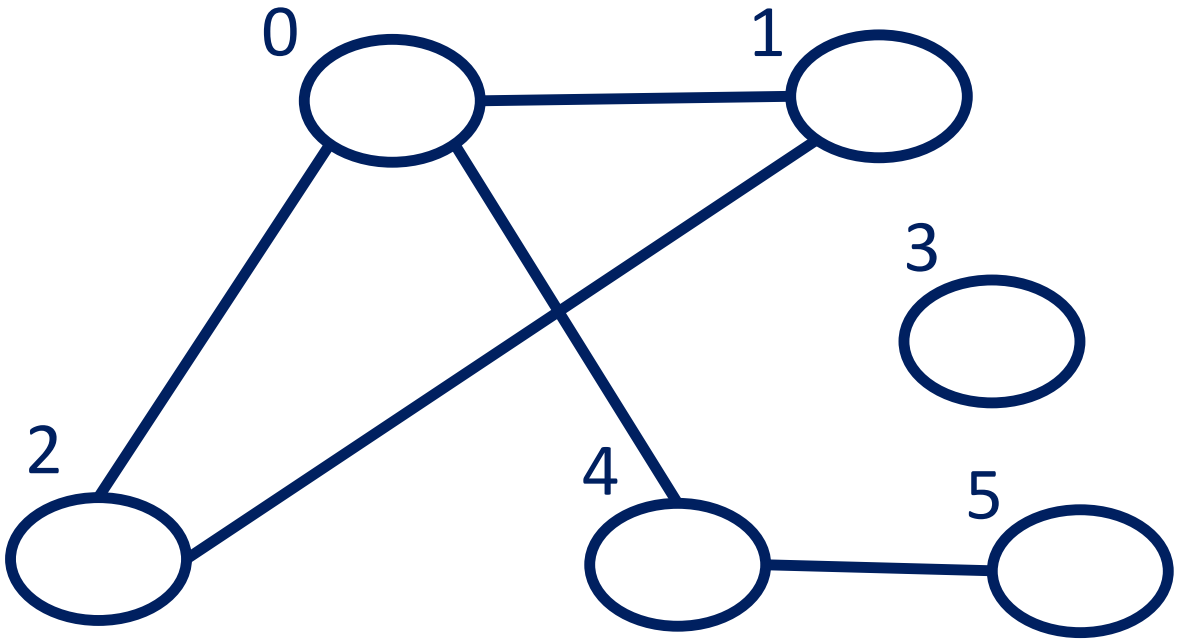
Depth first search: updated version



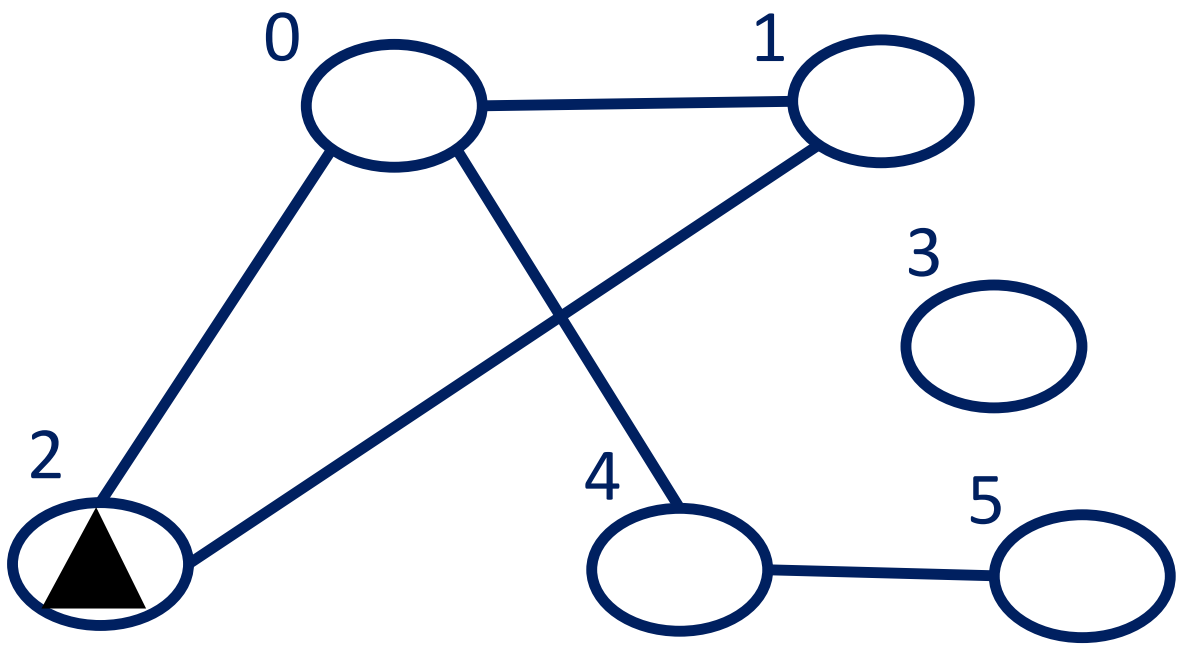
Is the stack empty?

Depth first search: updated version

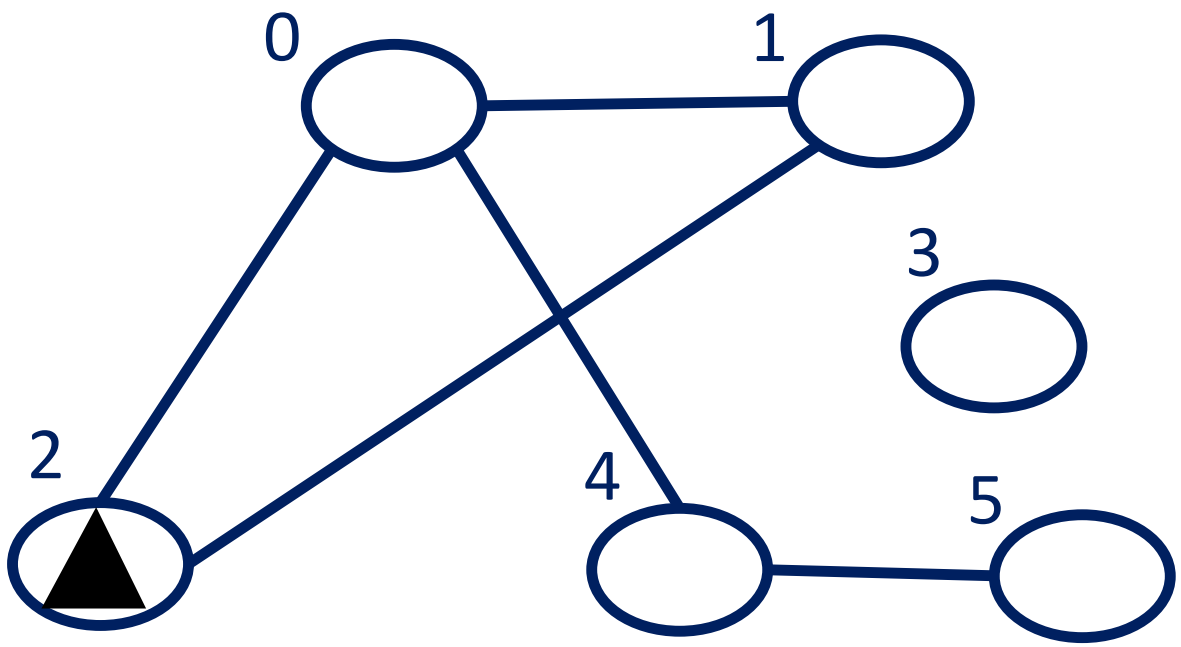
No! => Pop the node and make it current



Depth first search: updated version

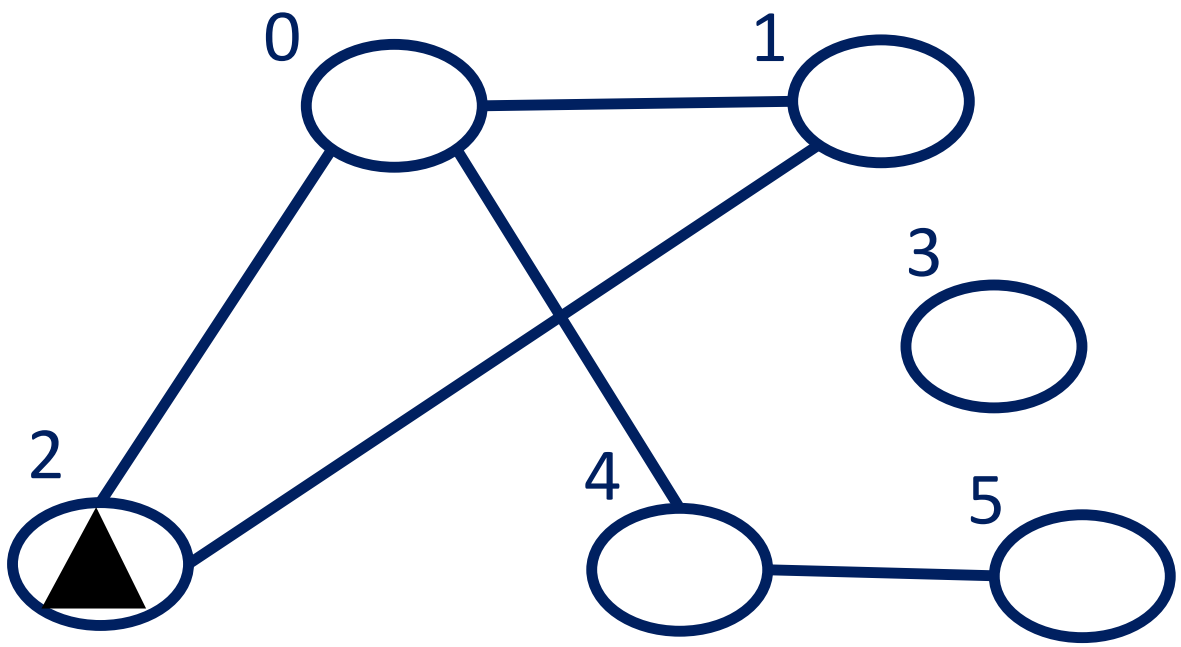


Depth first search: updated version



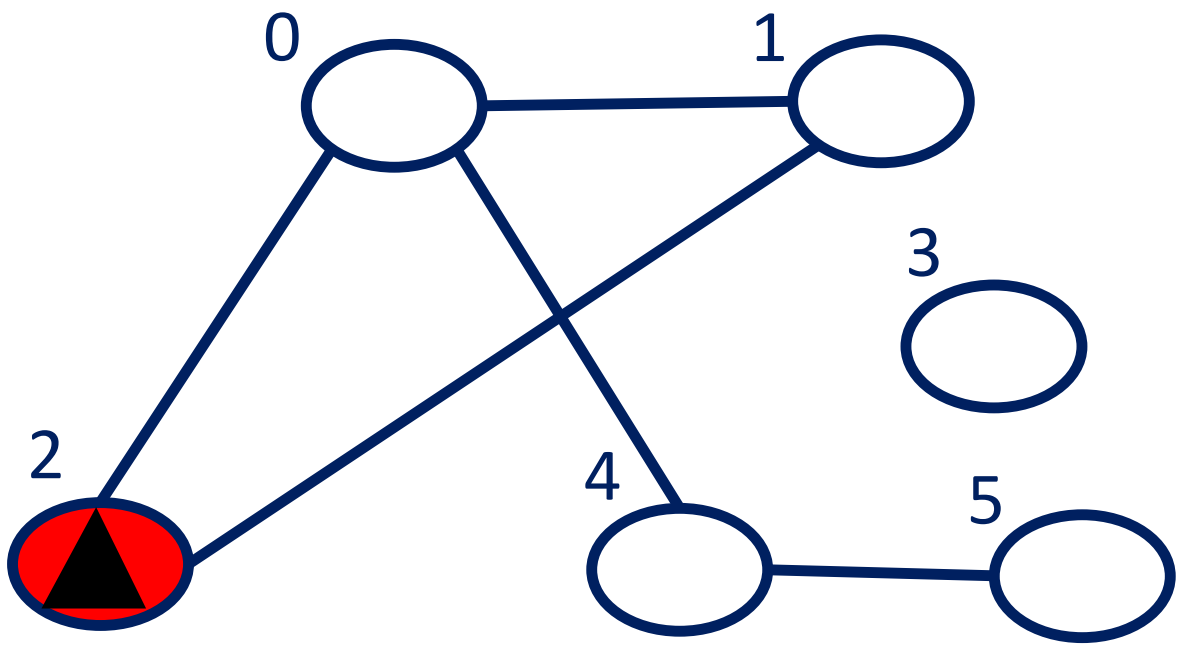
Is the node visited?

Depth first search: updated version



No! => Make it visited and add to the traversal

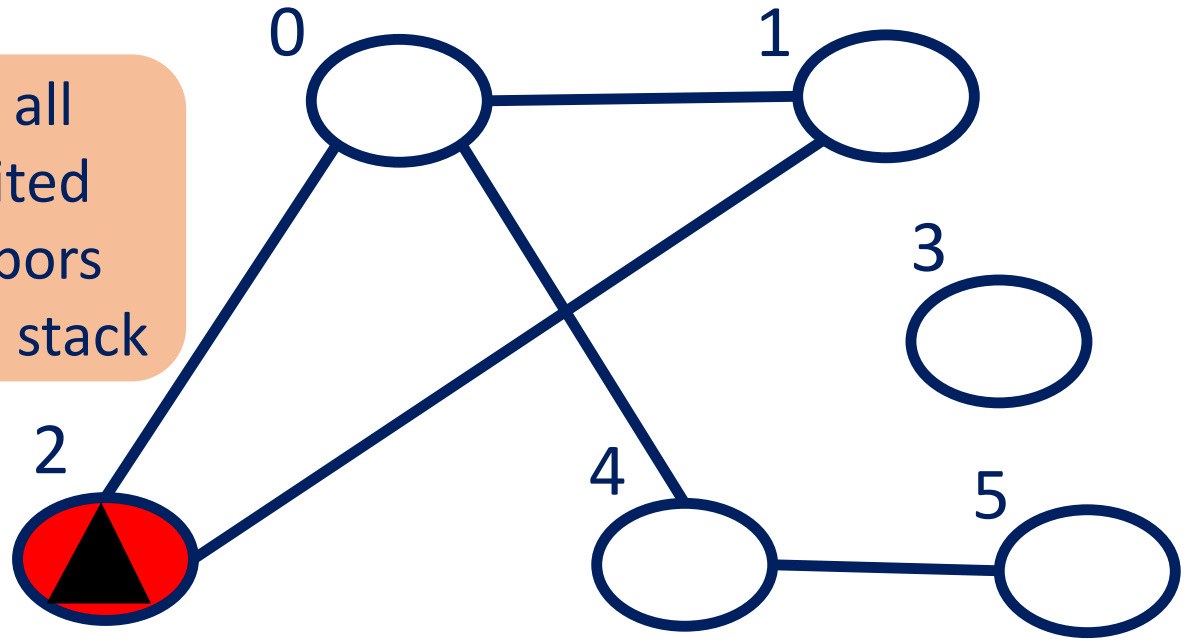
Depth first search: updated version



Traversal: 2

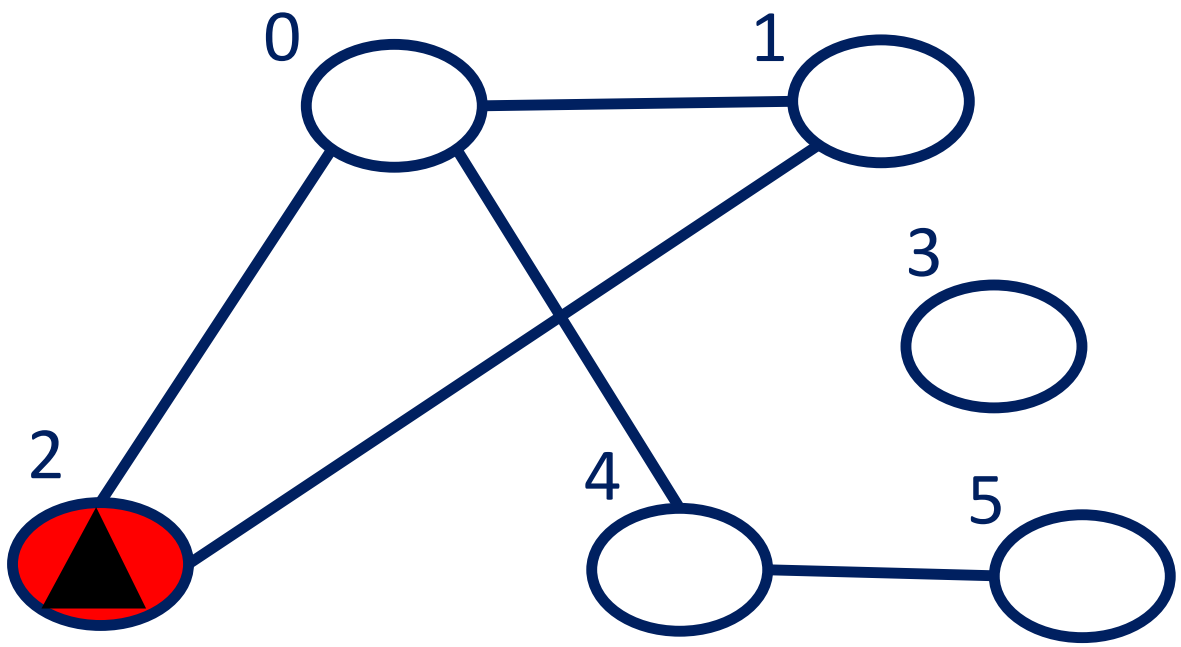
Depth first search: updated version

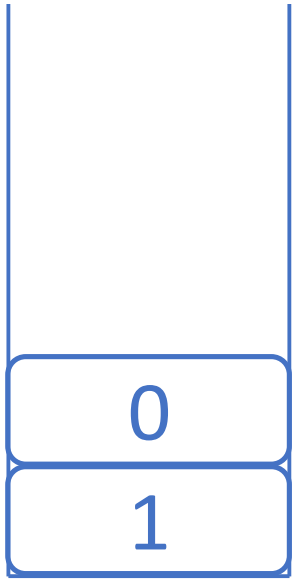
Push all unvisited neighbors into the stack



Traversal: 2

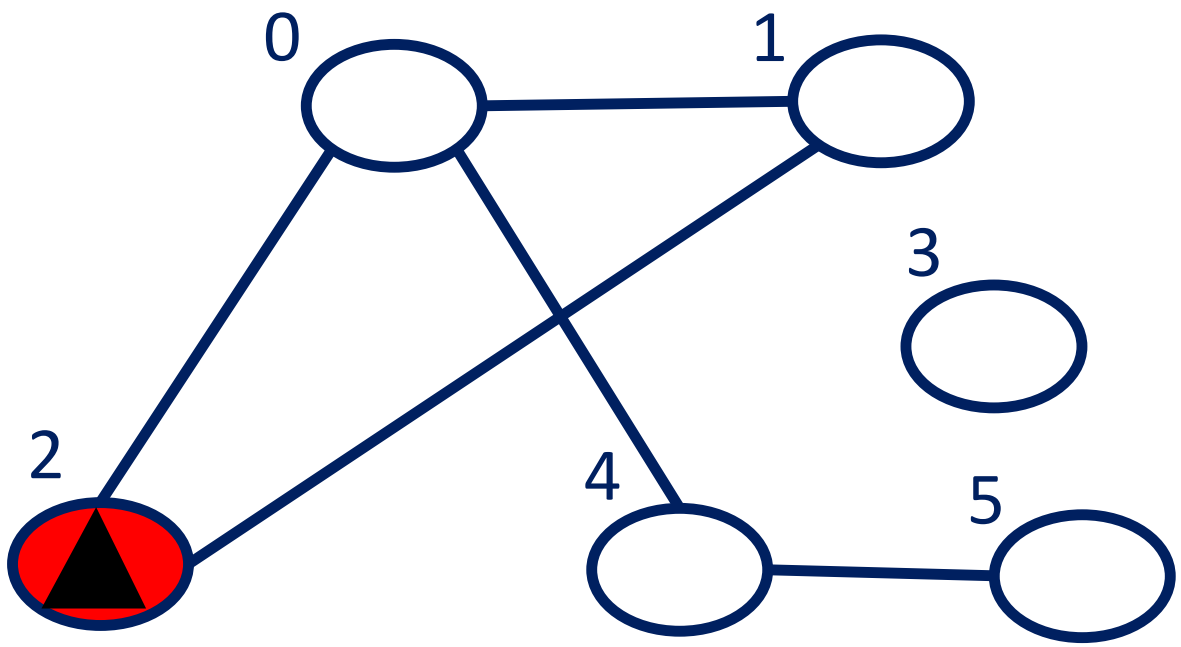
Depth first search: updated version



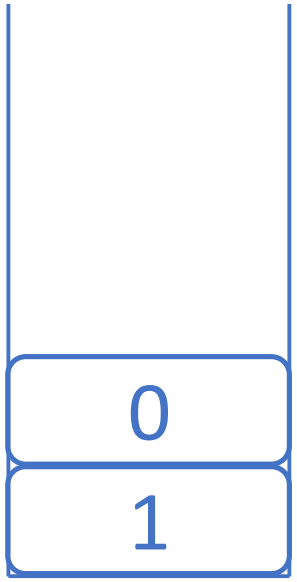


Traversal: 2

Depth first search: updated version



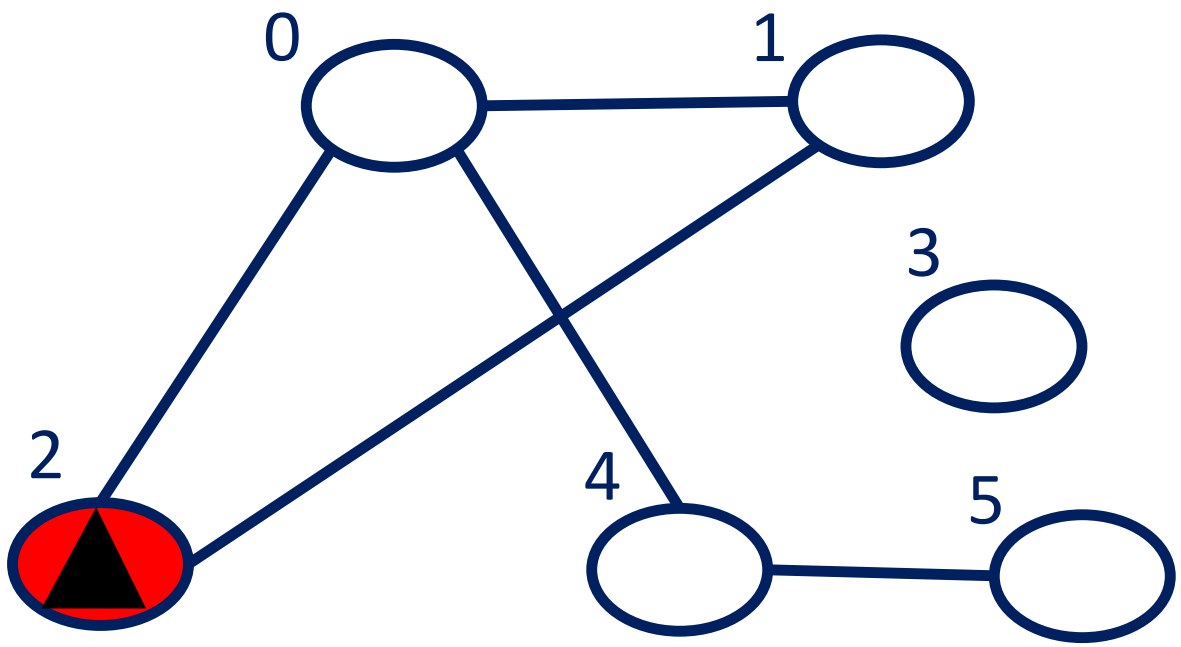
Is the stack empty?

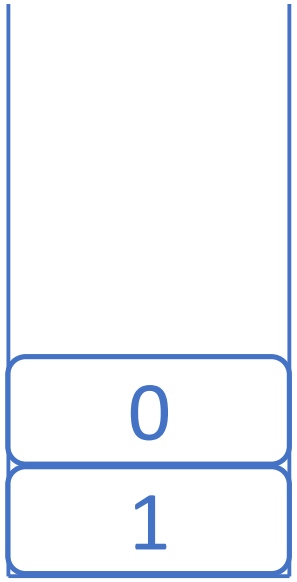


Traversal: 2

Depth first search: updated version

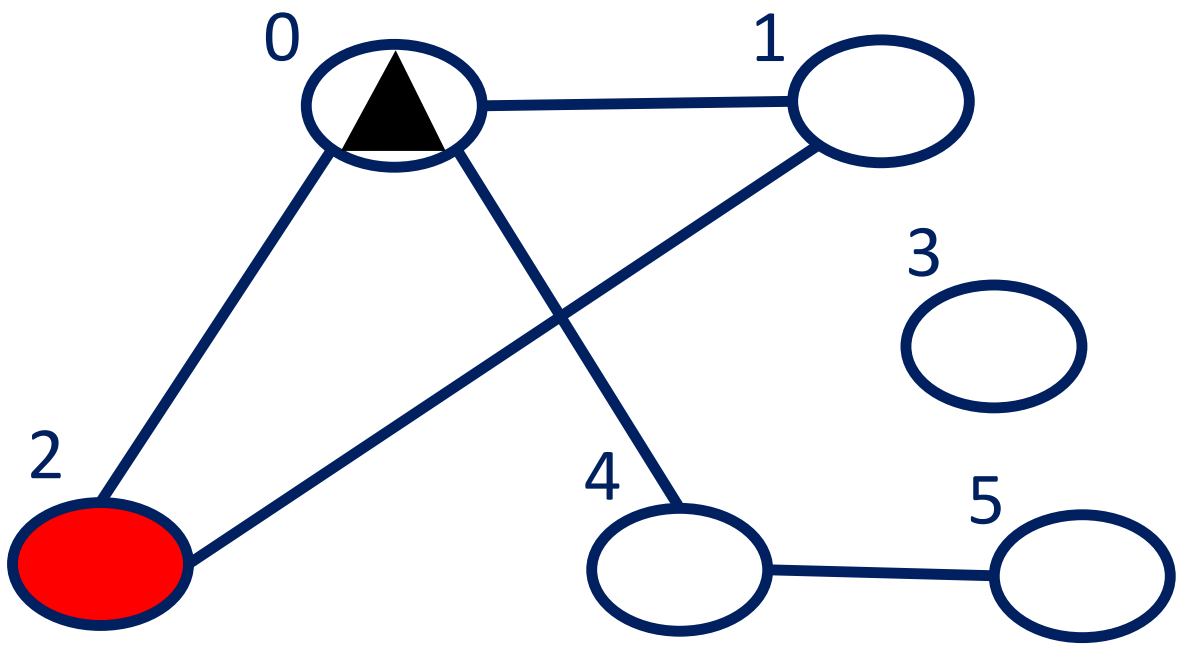
No! => Pop the node and make it current





Traversal: 2

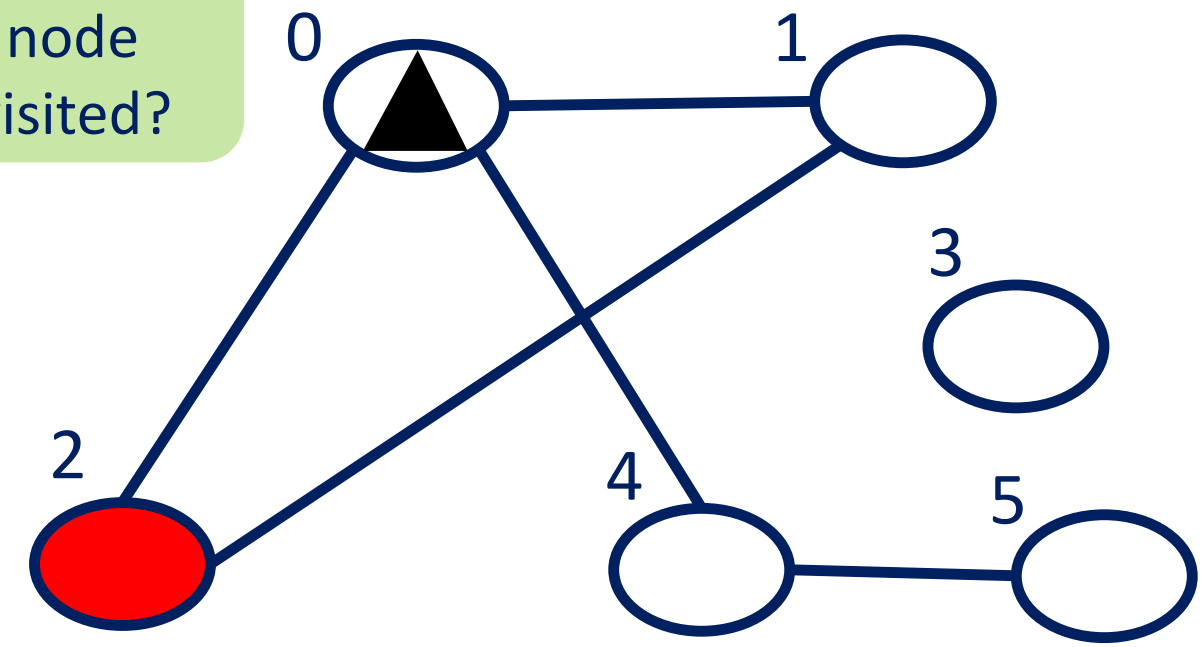
Depth first search: updated version



Traversal: 2

Depth first search: updated version

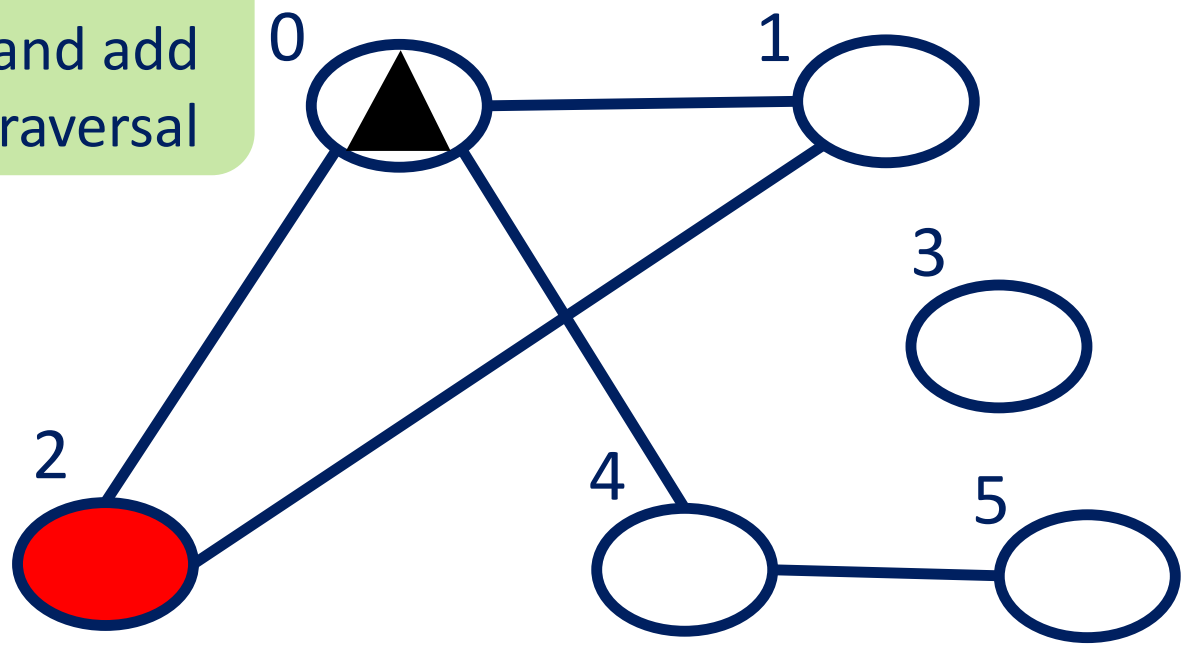
Is the node visited?



Traversal: 2

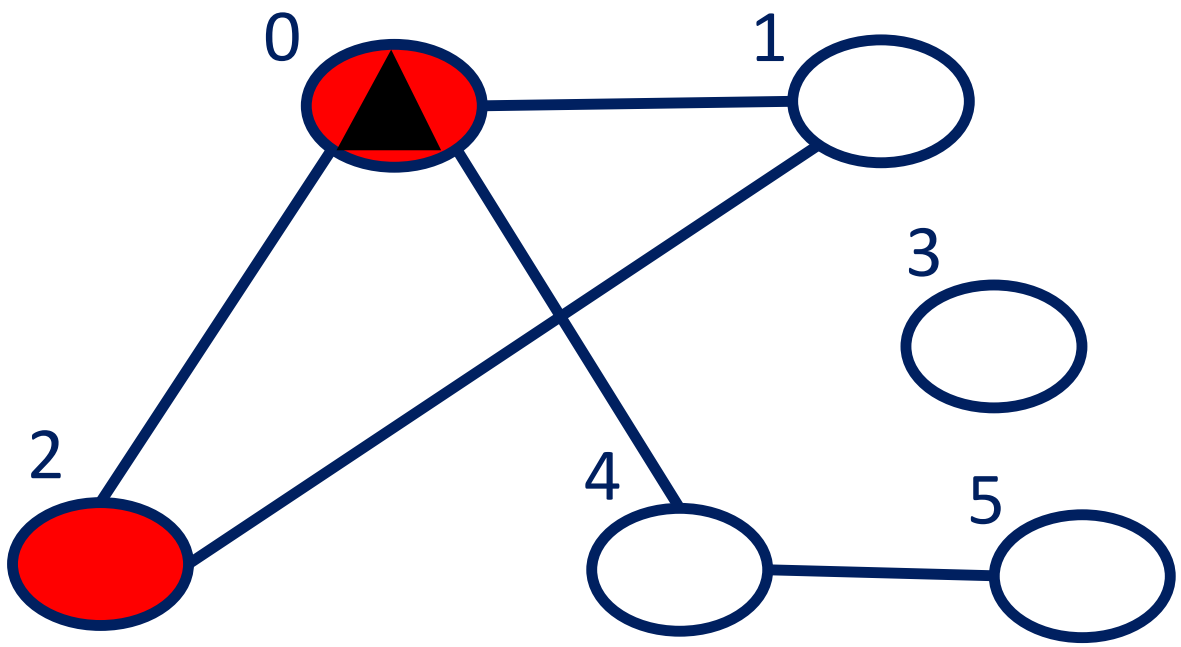
Depth first search: updated version

No! => Make it visited and add to the traversal



Traversal: 2

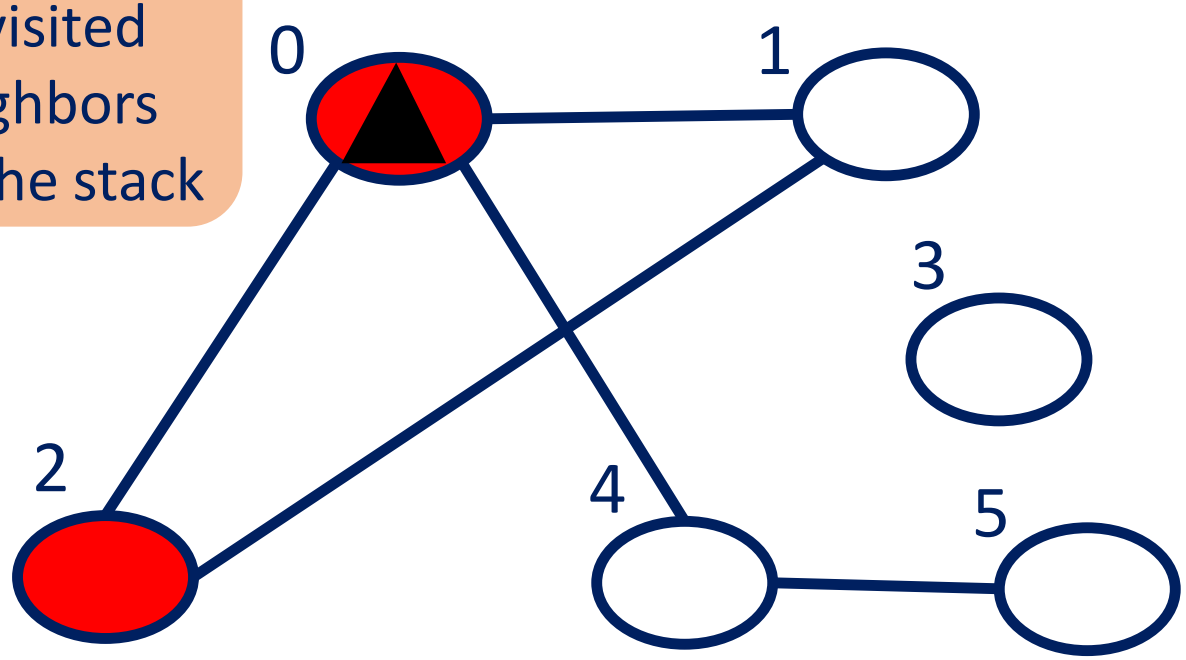
Depth first search: updated version



Traversal: 2,0

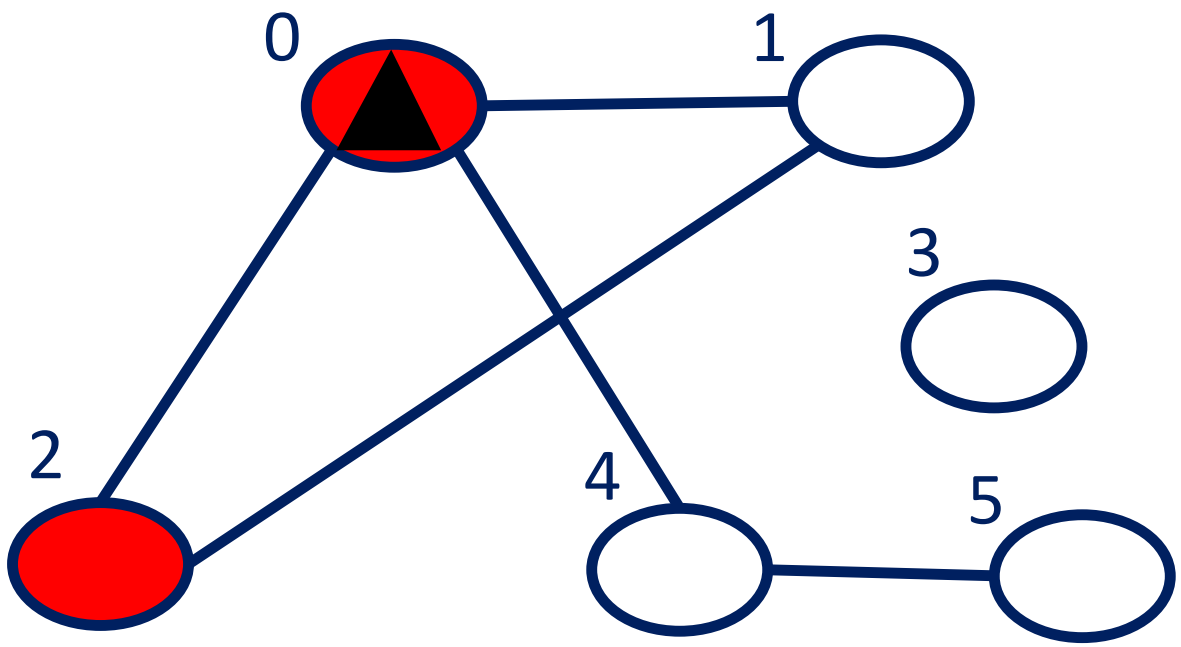
Depth first search: updated version

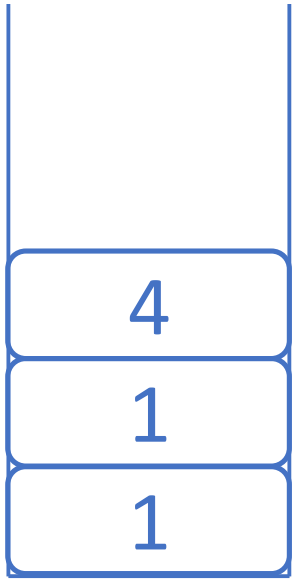
Push all unvisited neighbors into the stack



Traversal: 2,0

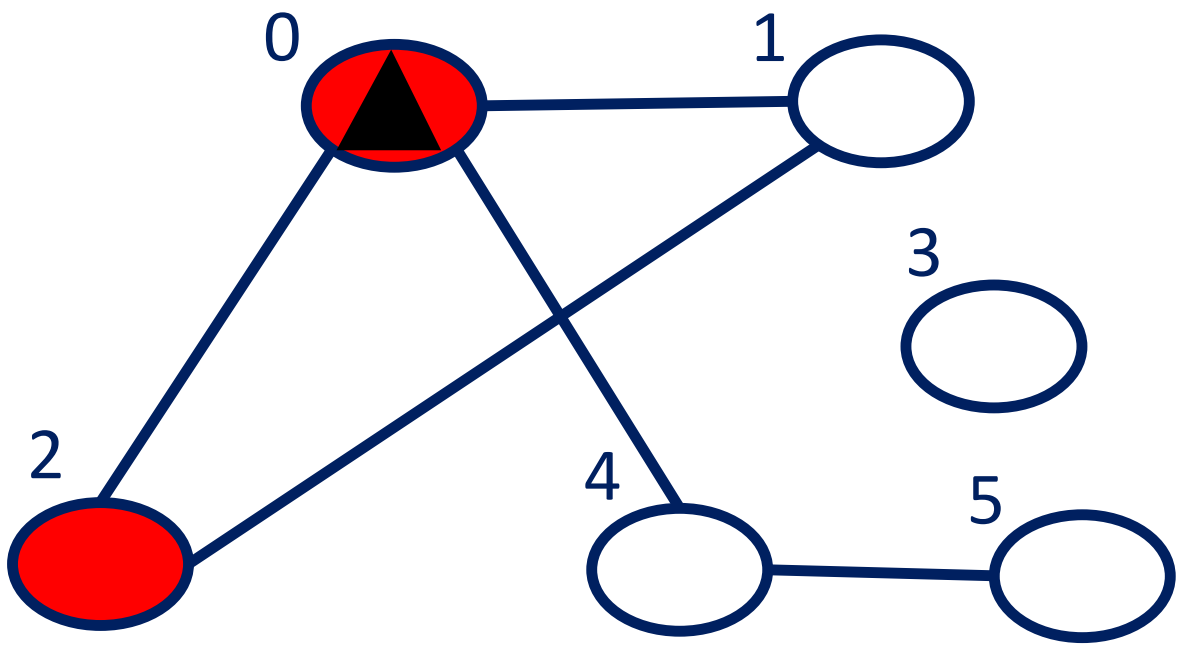
Depth first search: updated version



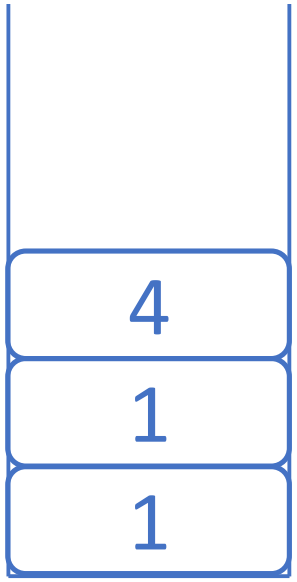


Traversal: 2,0

Depth first search: updated version



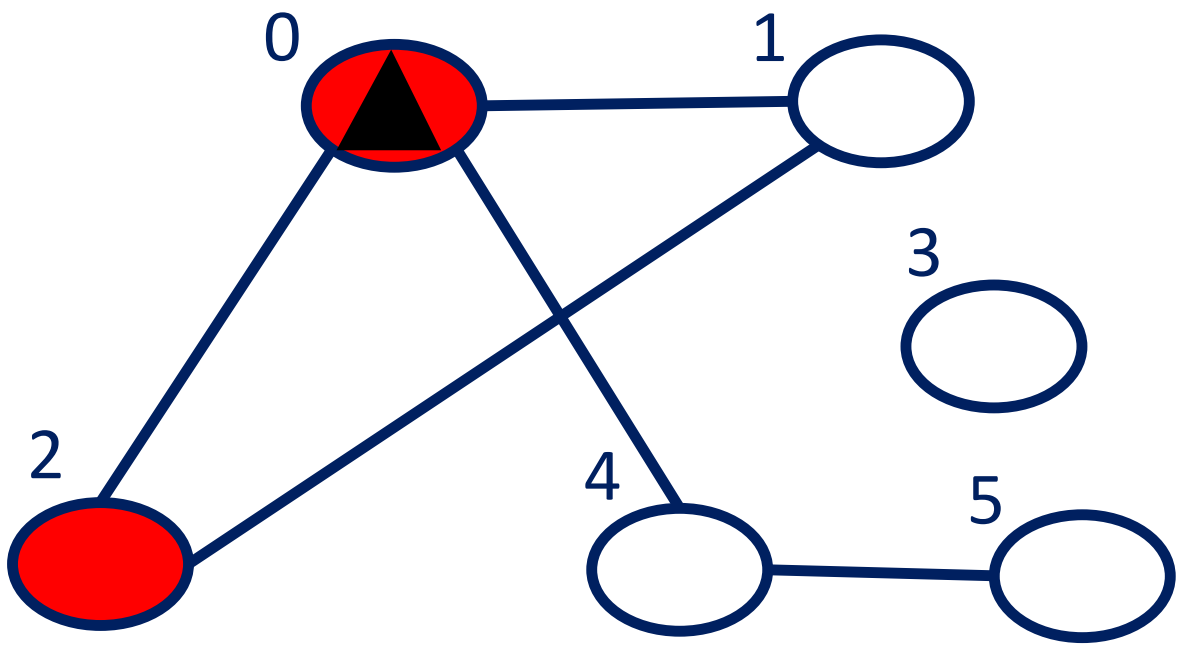
Is the stack empty?

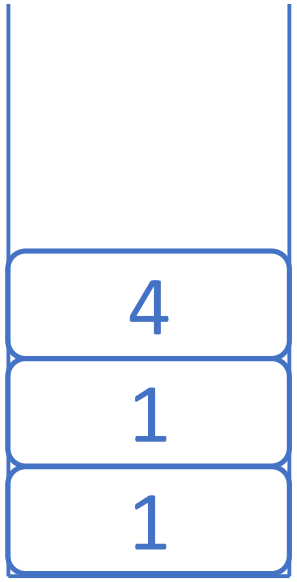


Traversal: 2,0

Depth first search: updated version

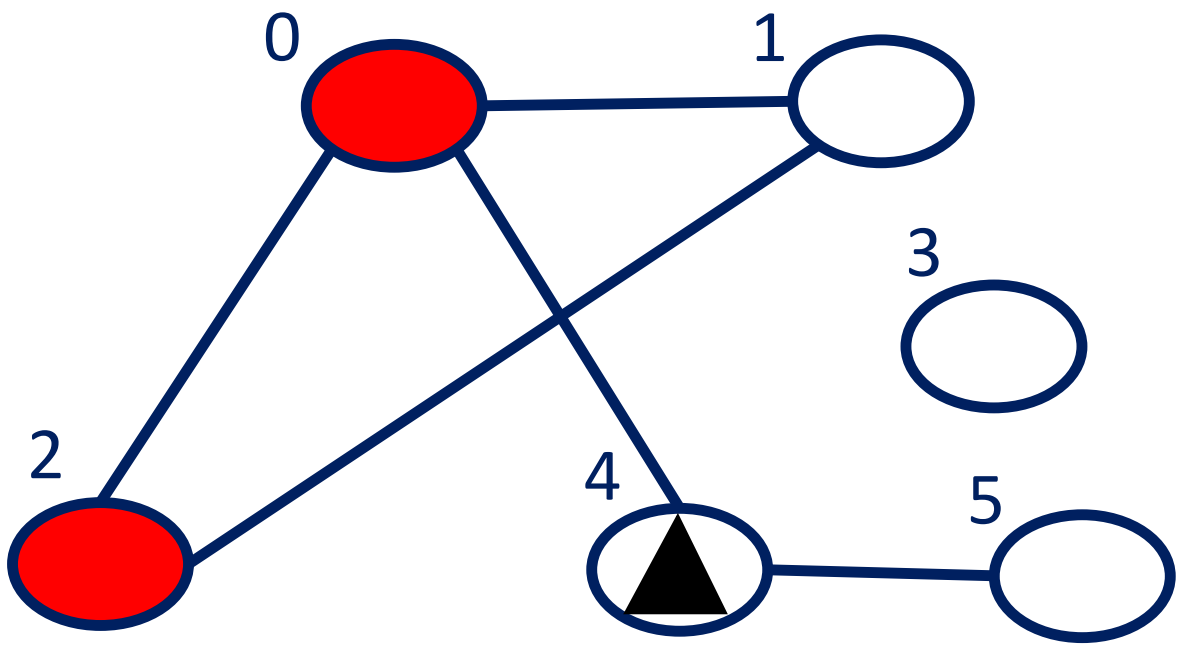
No! => Pop the node and make it current

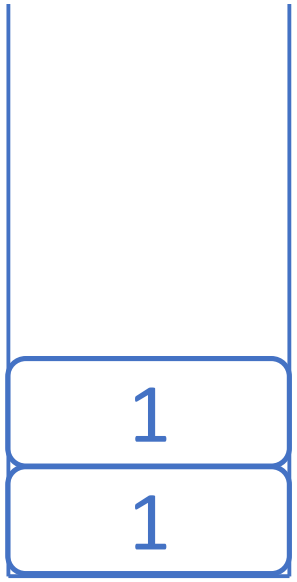




Traversal: 2,0

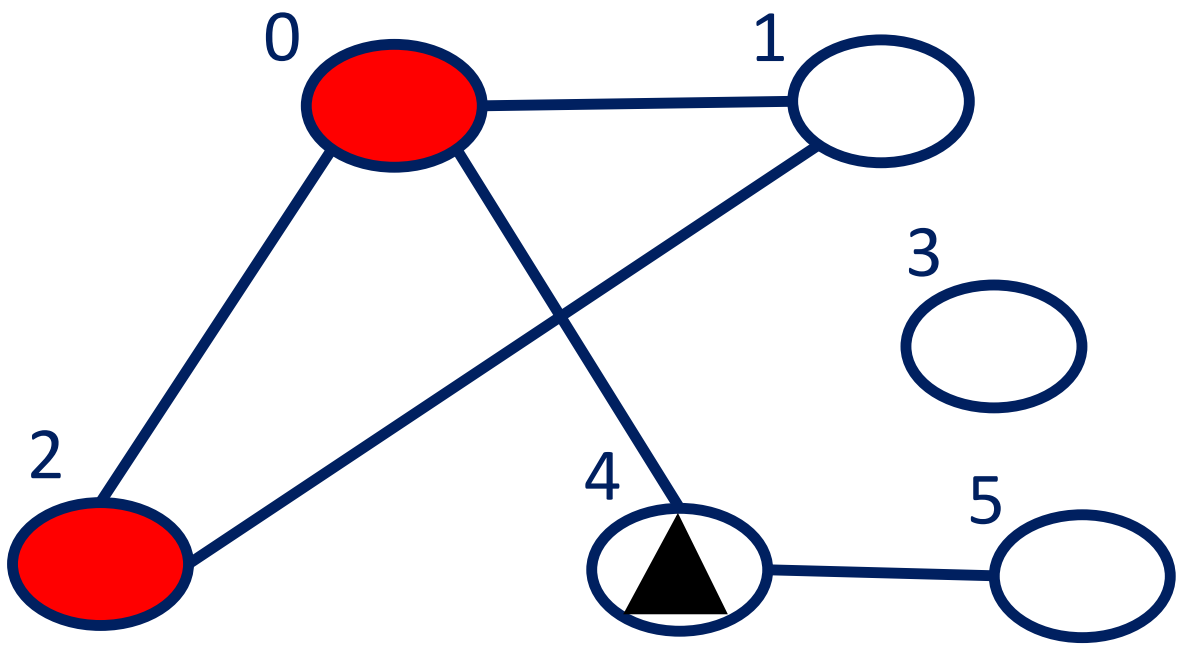
Depth first search: updated version

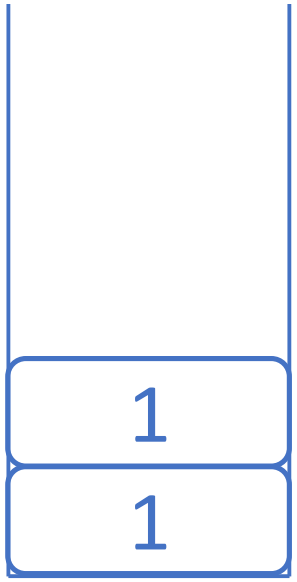




Traversal: 2,0

Depth first search: updated version

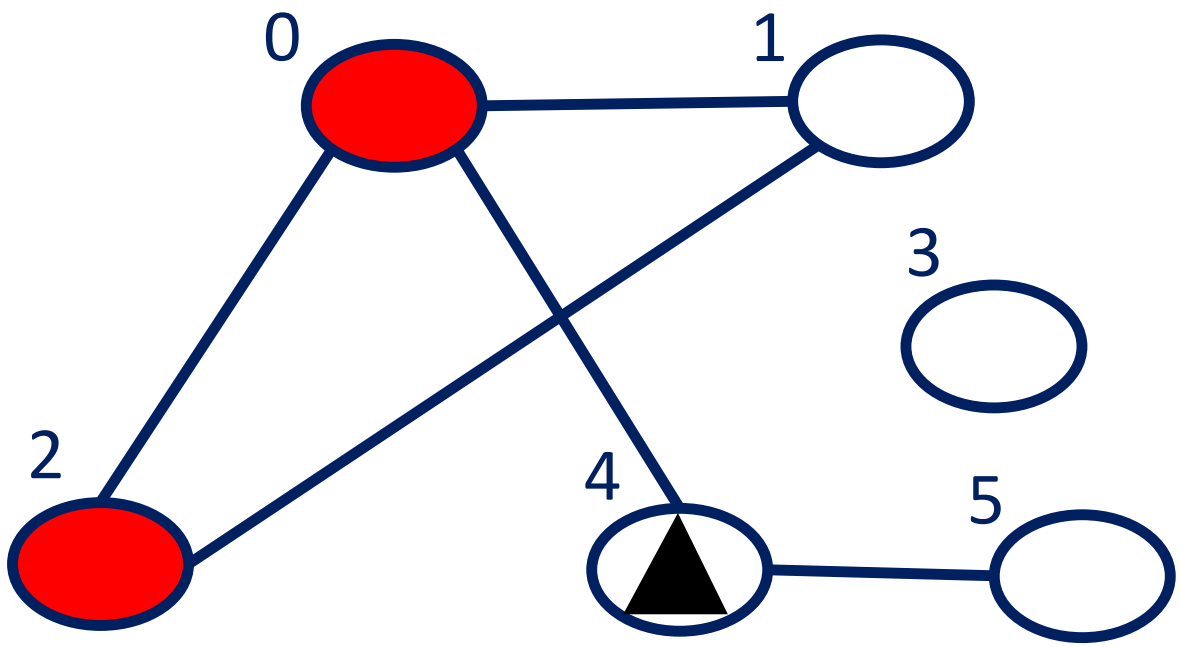


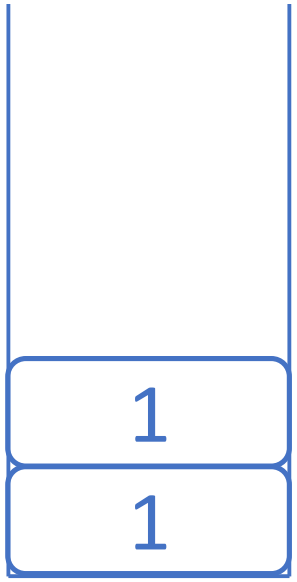


Is the node visited?

Traversal: 2,0

Depth first search: updated version

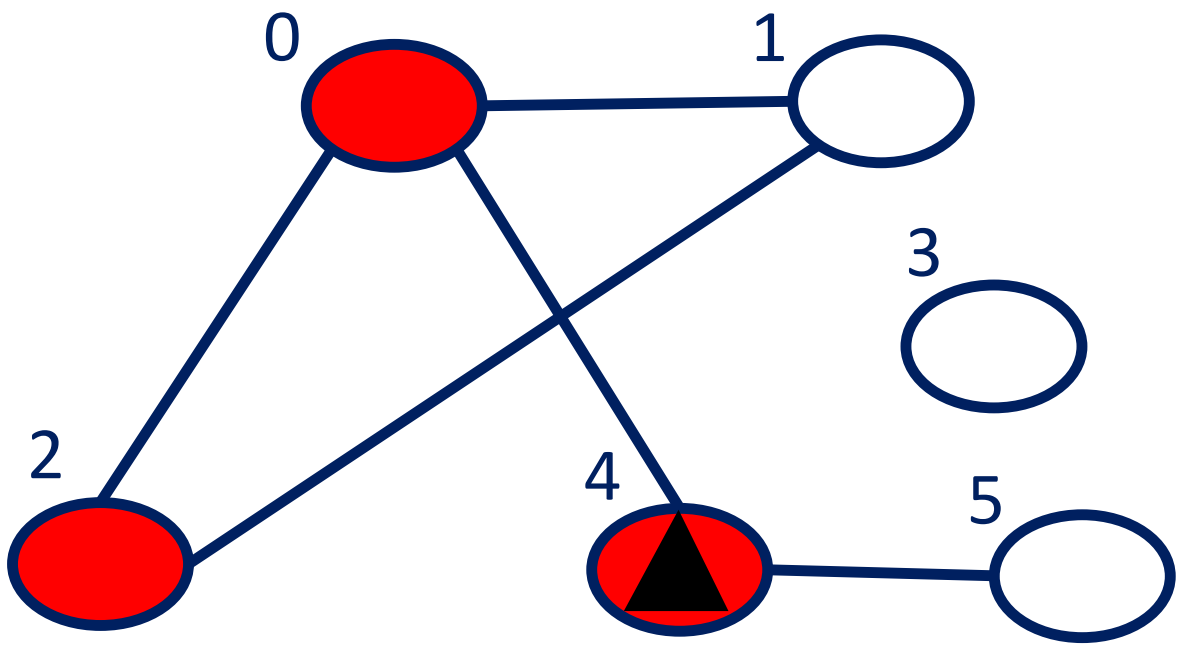


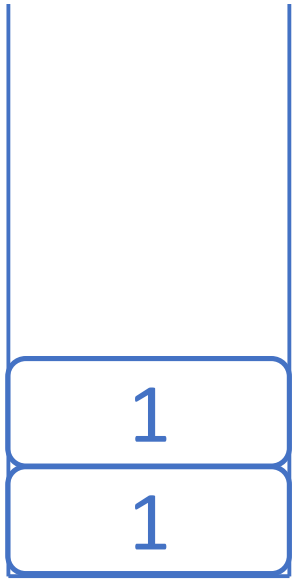


No! => Make it visited and add to the traversal

Traversal: 2,0

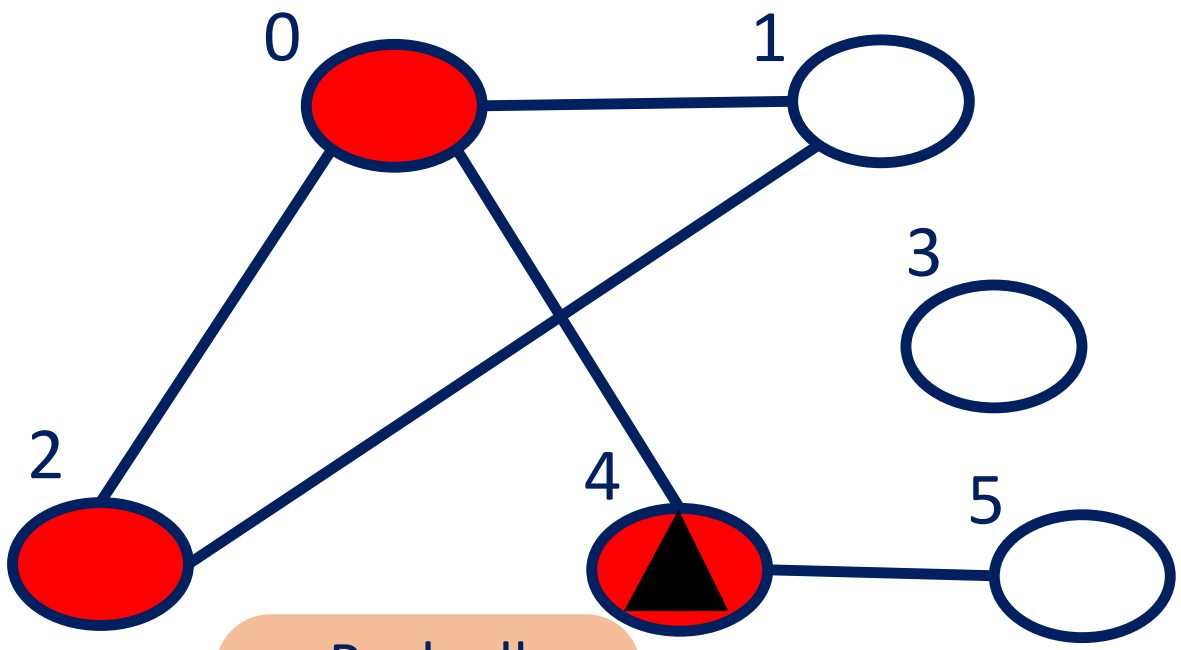
Depth first search: updated version



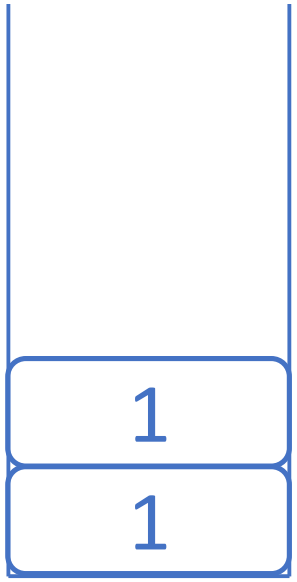


Traversal: 2,0,4

Depth first search: updated version

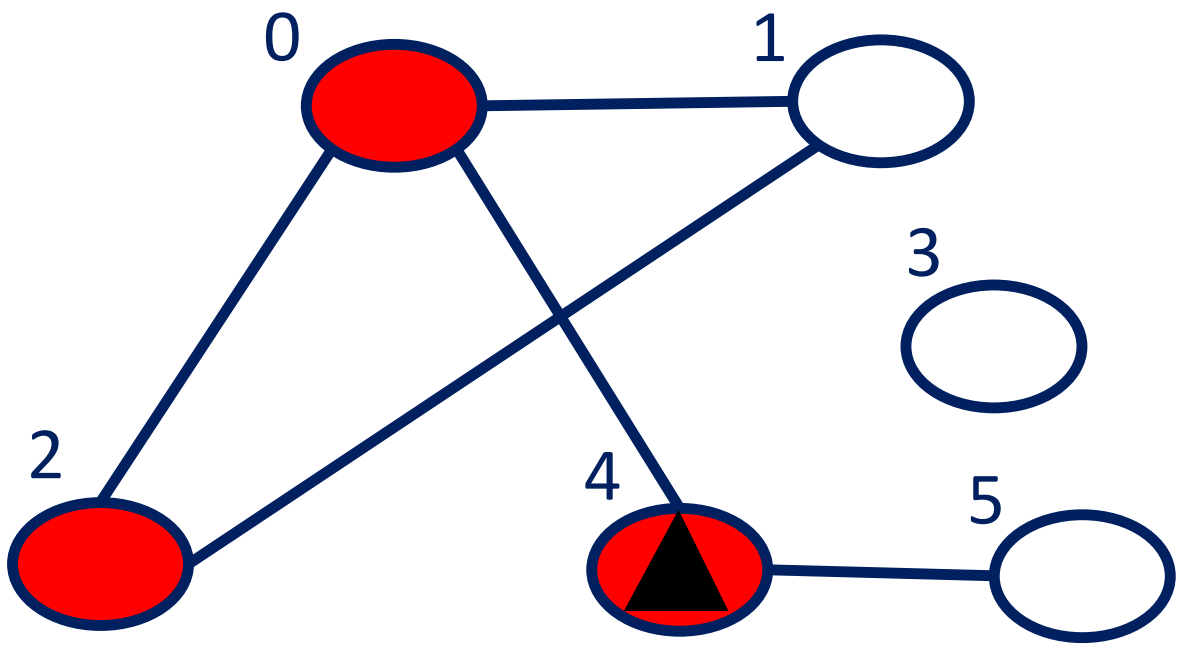


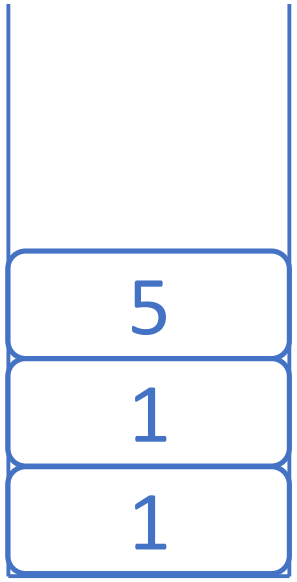
Push all unvisited neighbors into the stack



Traversal: 2,0,4

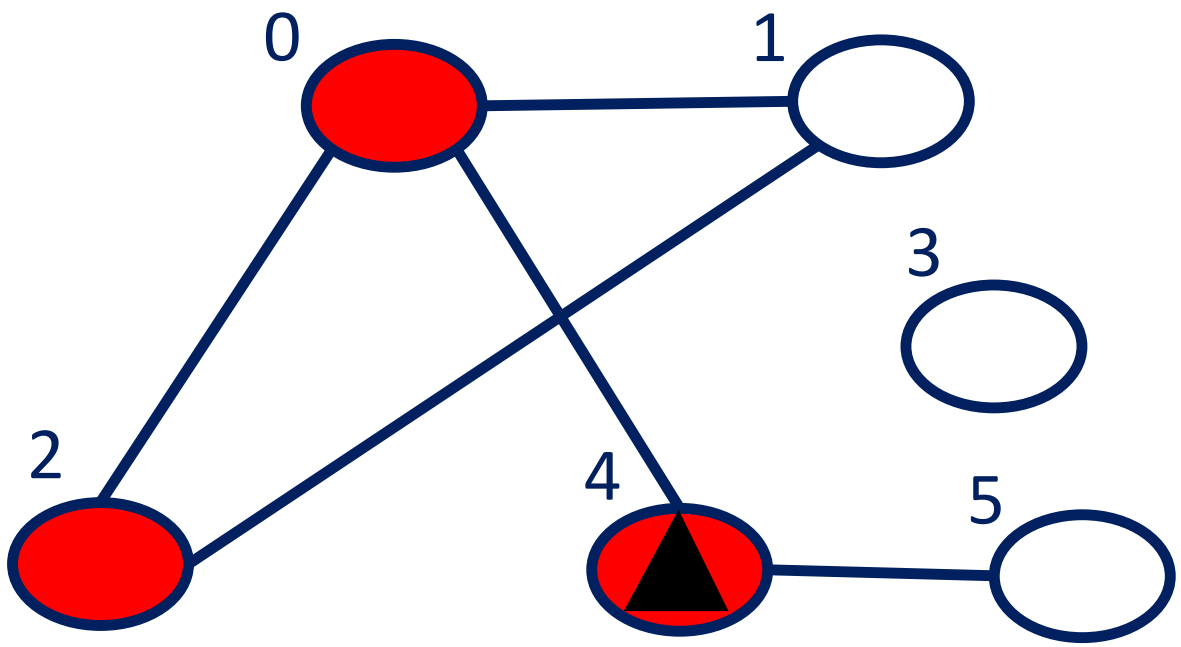
Depth first search: updated version



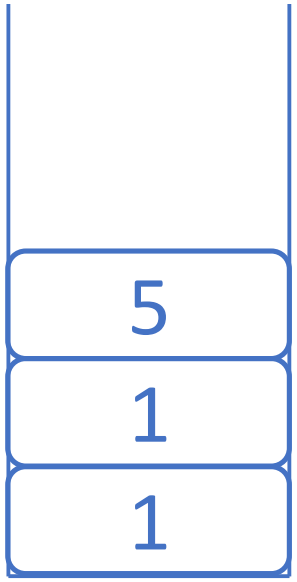


Traversal: 2,0,4

Depth first search: updated version



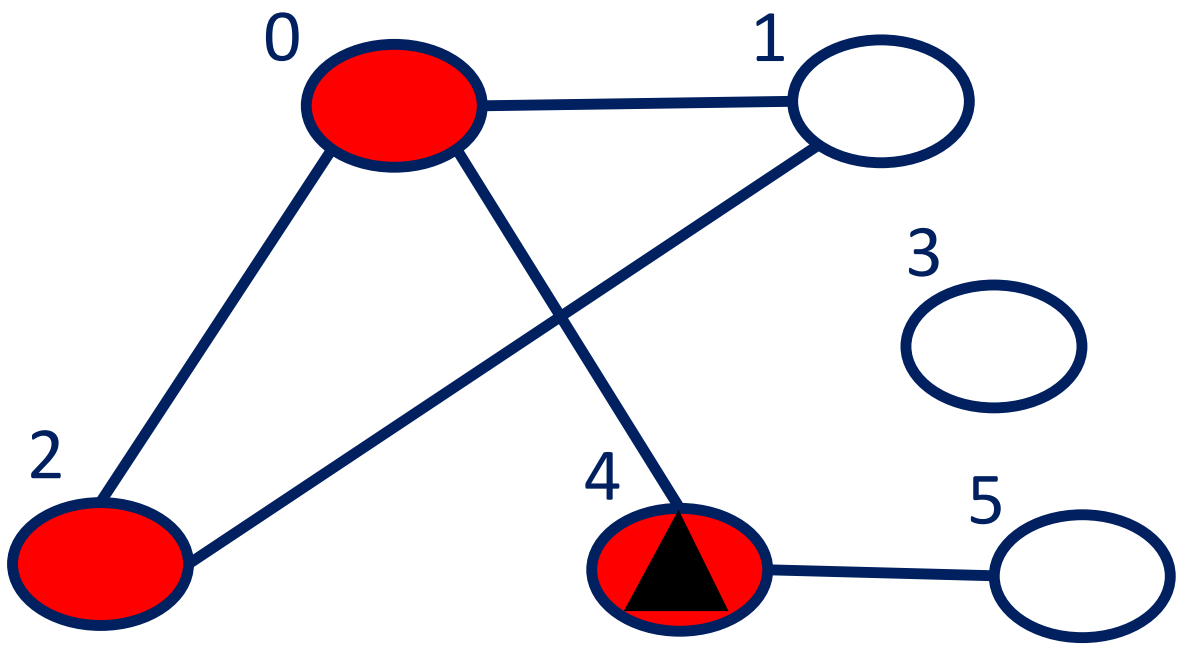
Is the stack empty?

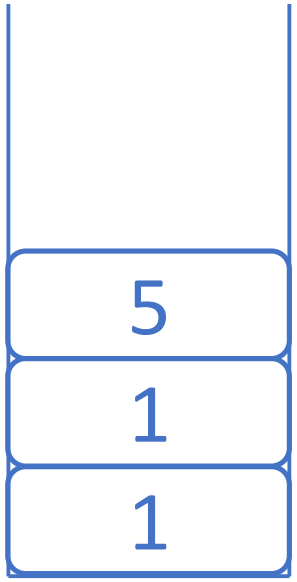


Traversal: 2,0,4

Depth first search: updated version

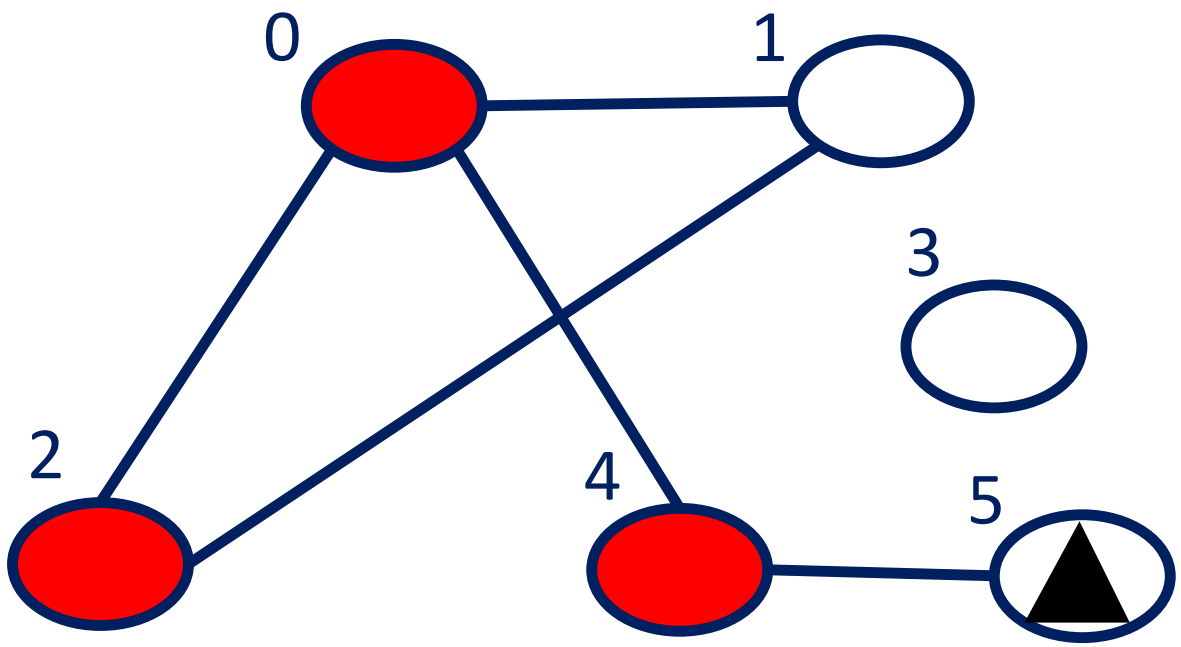
No! => Pop the node and make it current

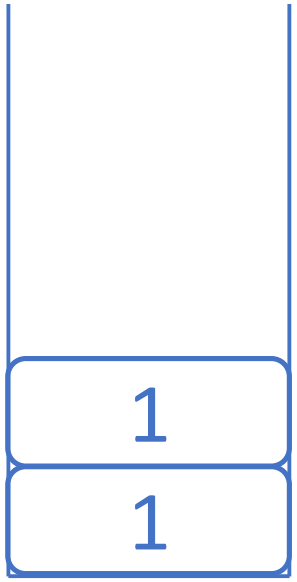




Traversal: 2,0,4

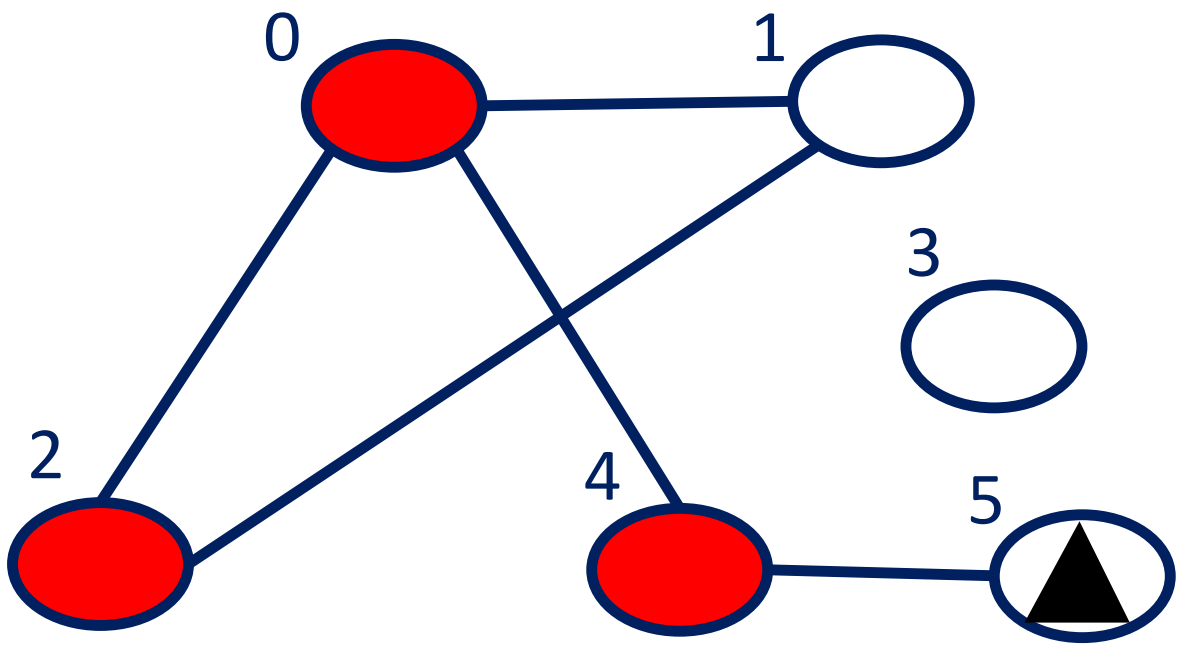
Depth first search: updated version

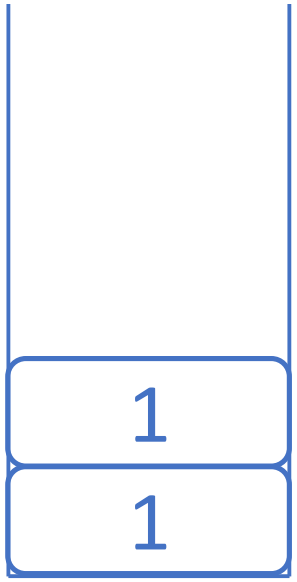




Traversal: 2,0,4

Depth first search: updated version

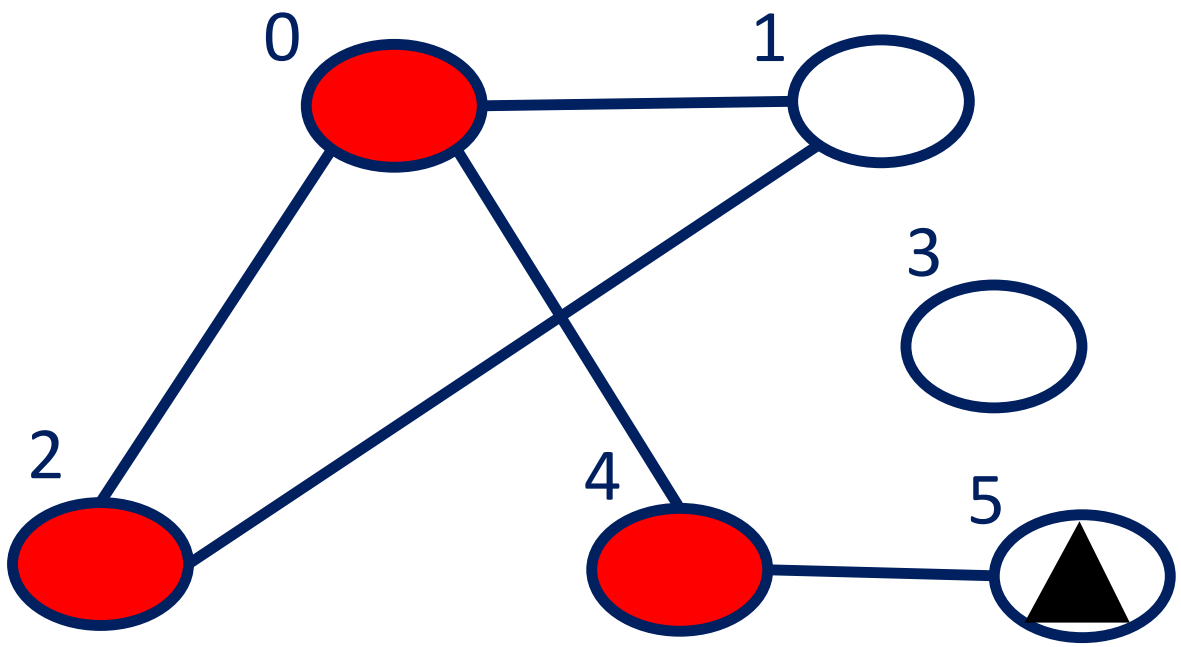


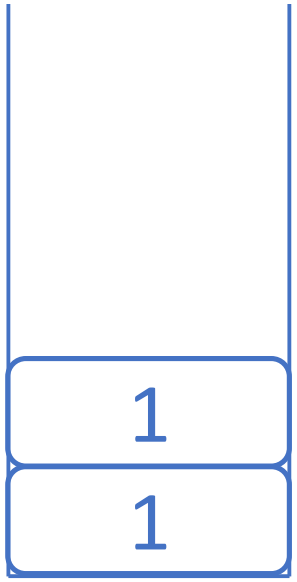


Is the node visited?

Traversal: 2,0,4

Depth first search: updated version

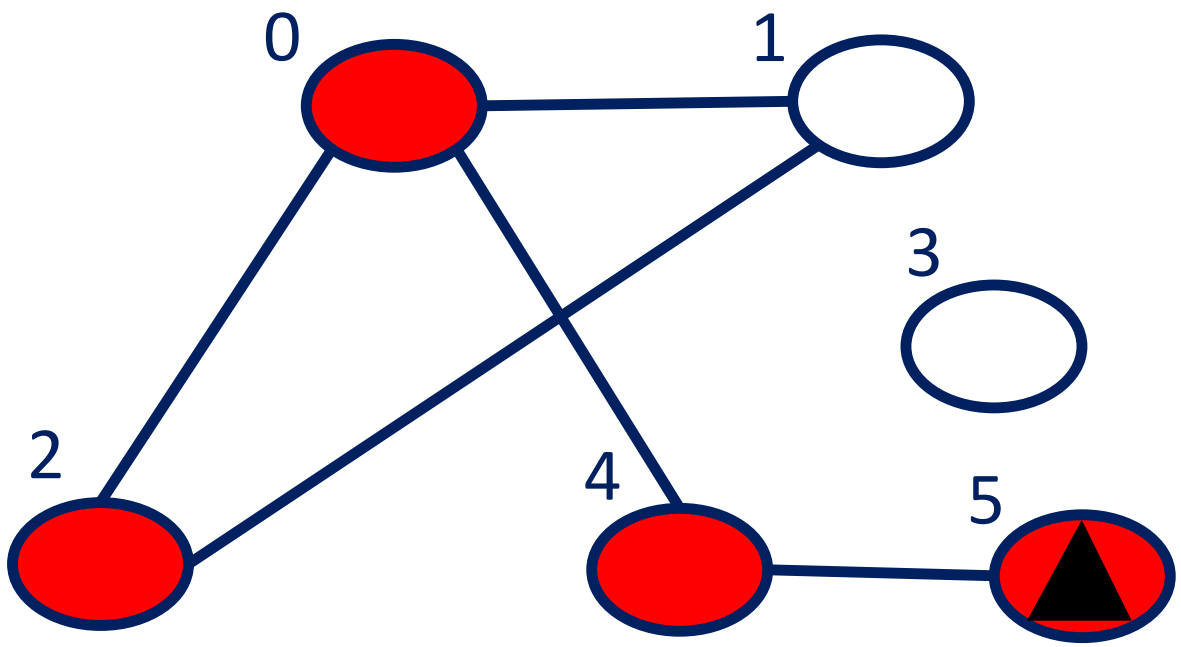


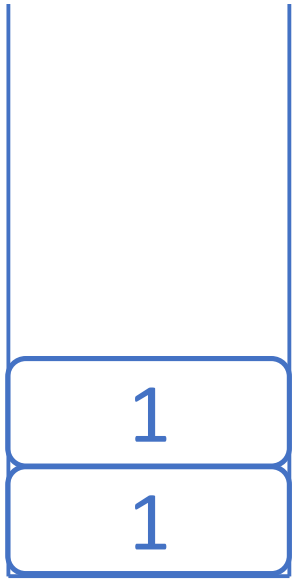


No! => Make it visited and add to the traversal

Traversal: 2,0,4

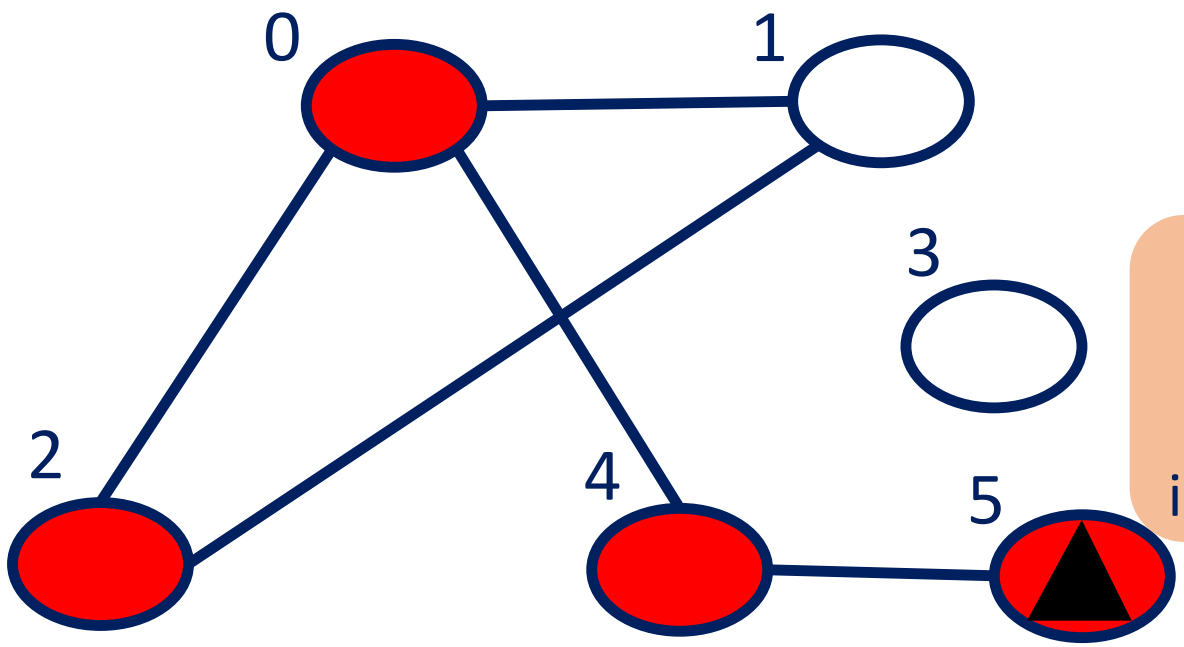
Depth first search: updated version



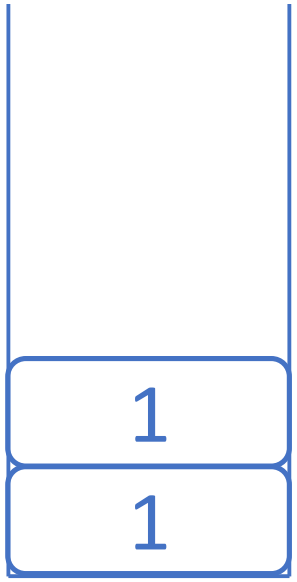


Traversal: 2,0,4,5

Depth first search: updated version

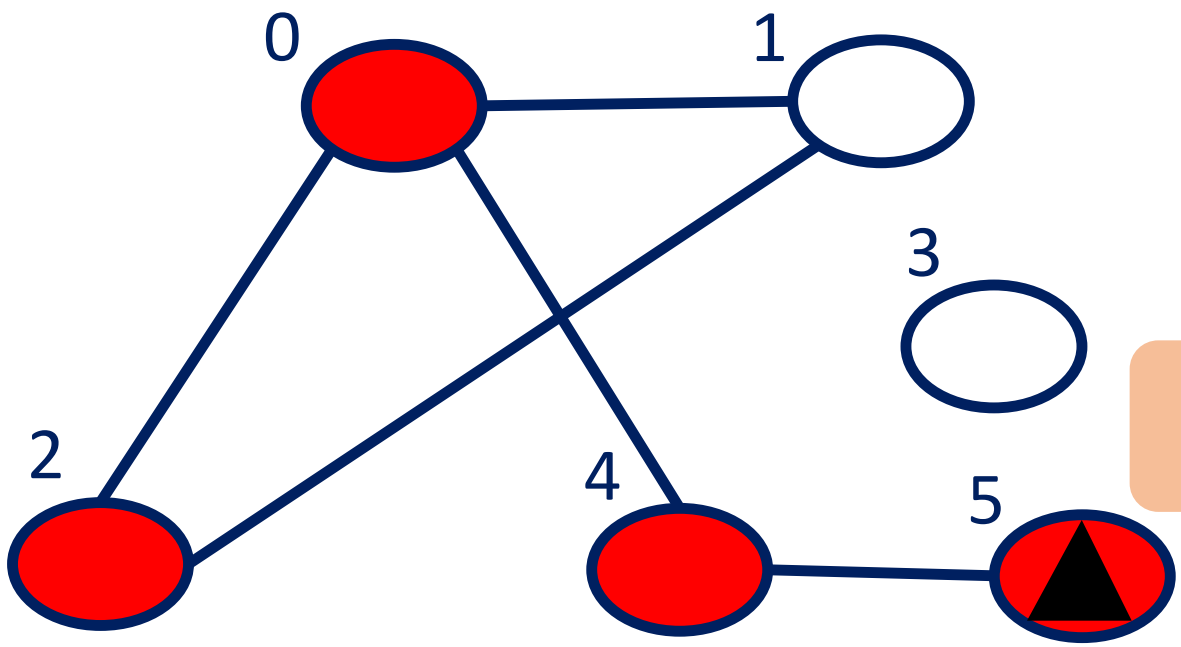


Push all unvisited neighbors into the stack

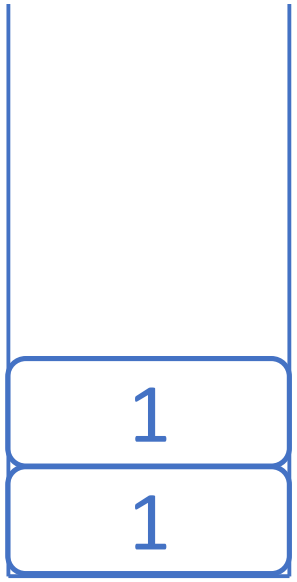


Traversal: 2,0,4,5

Depth first search: updated version

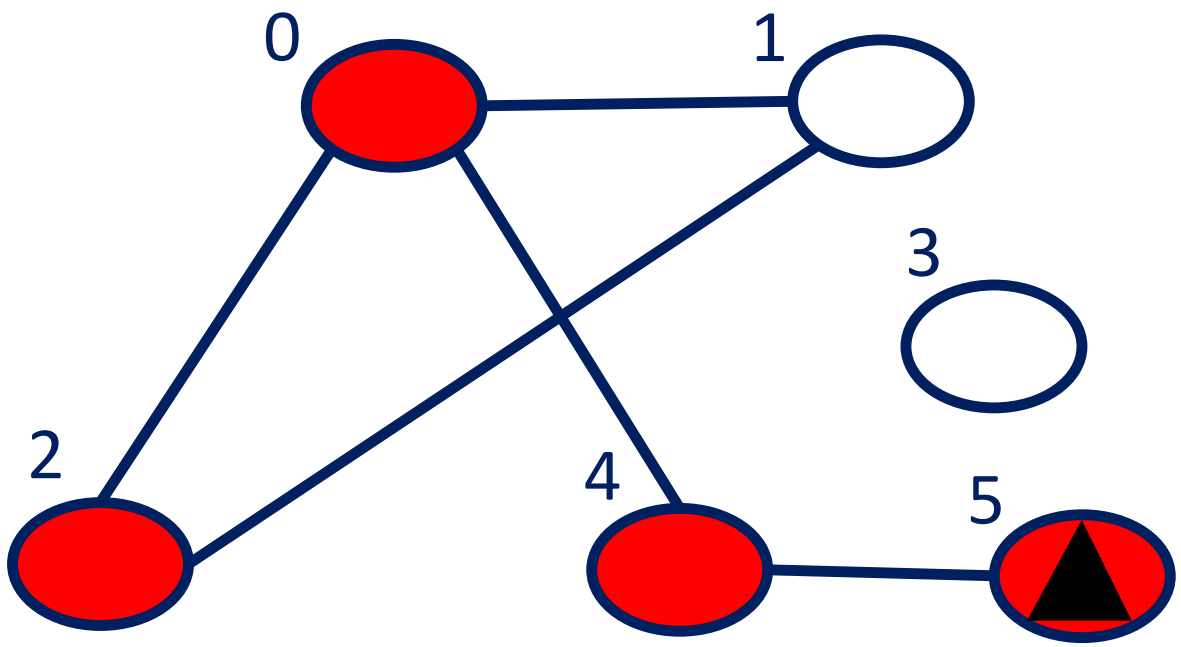


None exist!

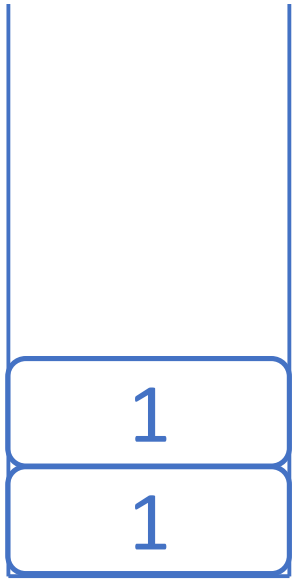


Traversal: 2,0,4,5

Depth first search: updated version



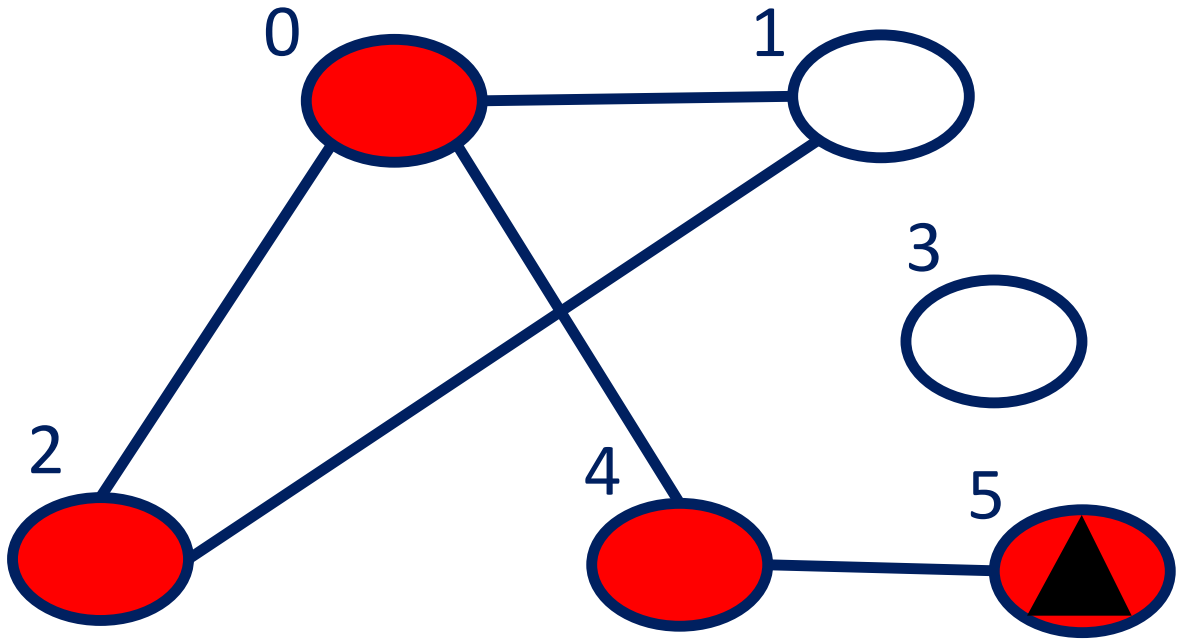
Is the stack empty?

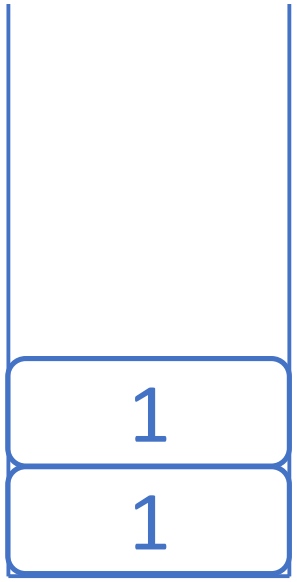


Traversal: 2,0,4,5

Depth first search: updated version

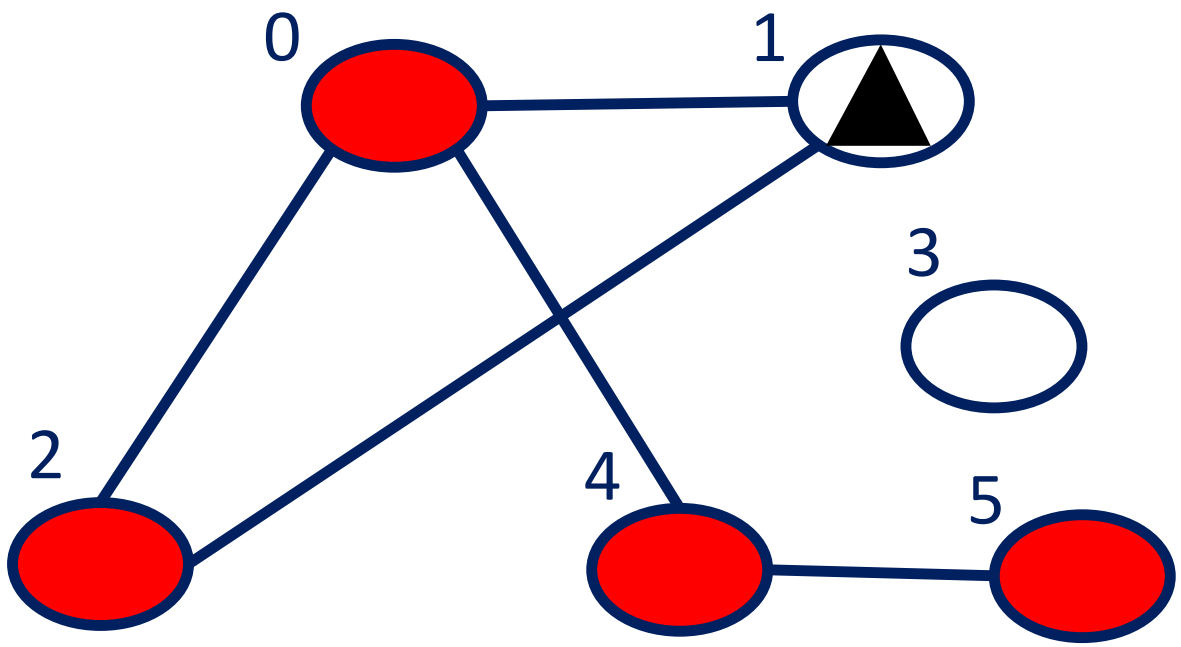
No! => Pop the node and make it current





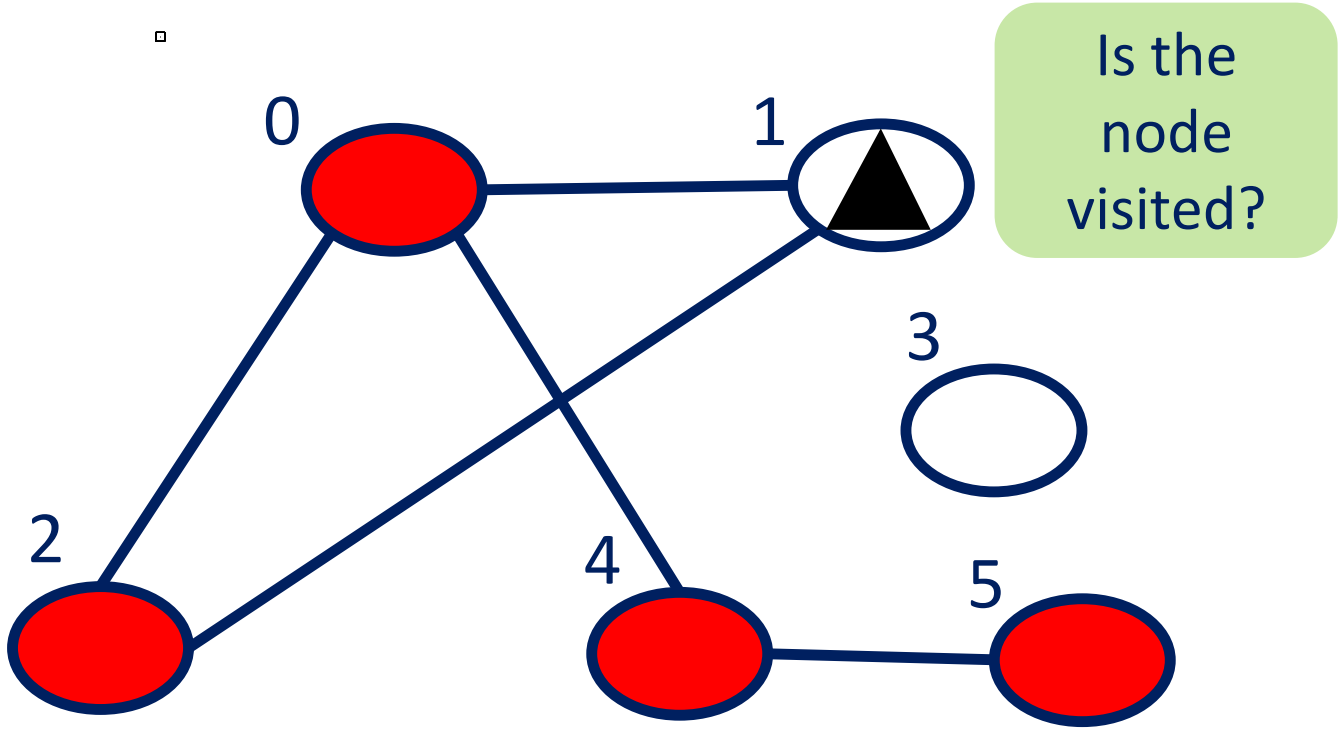
Traversal: 2,0,4,5

Depth first search: updated version



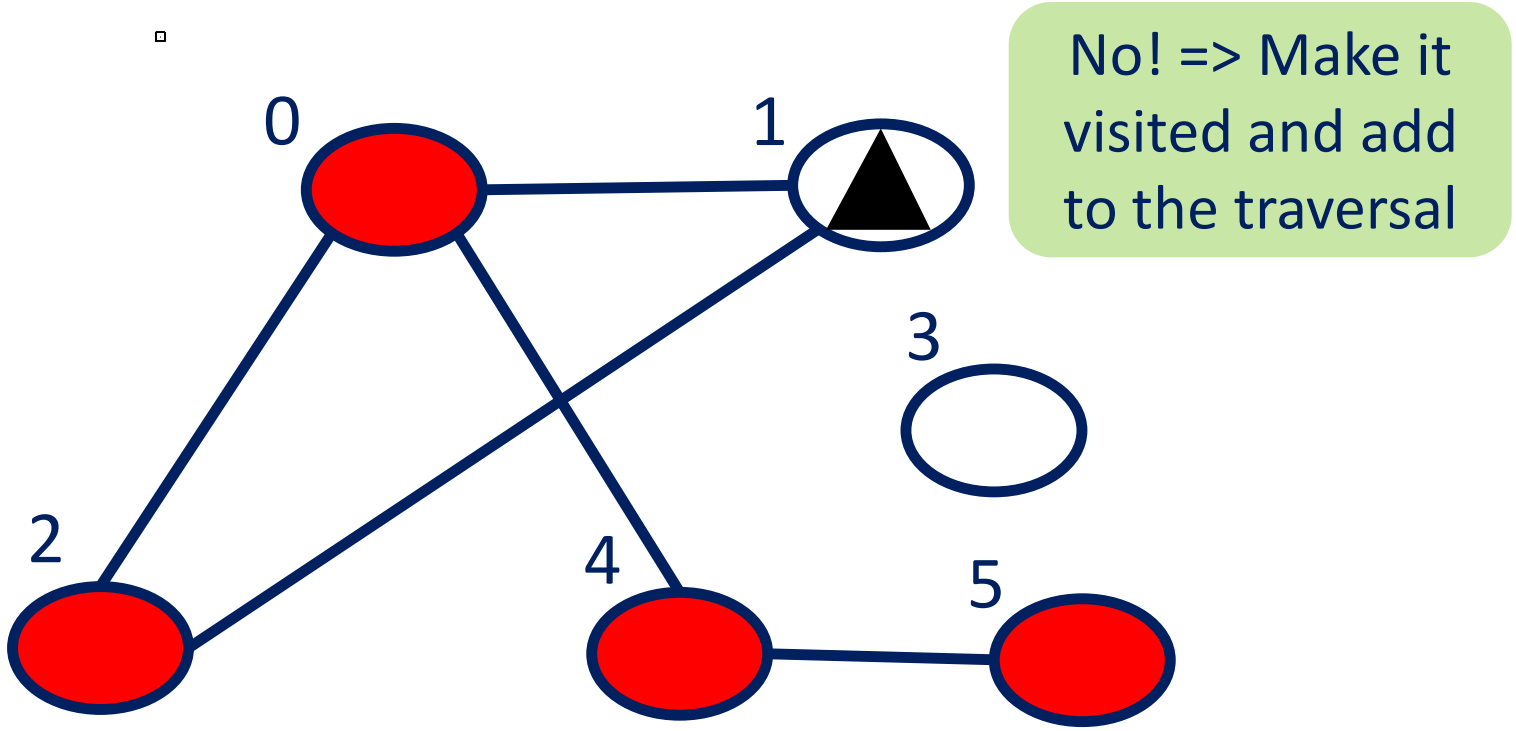
Traversal: 2,0,4,5

Depth first search: updated version



Traversal: 2,0,4,5

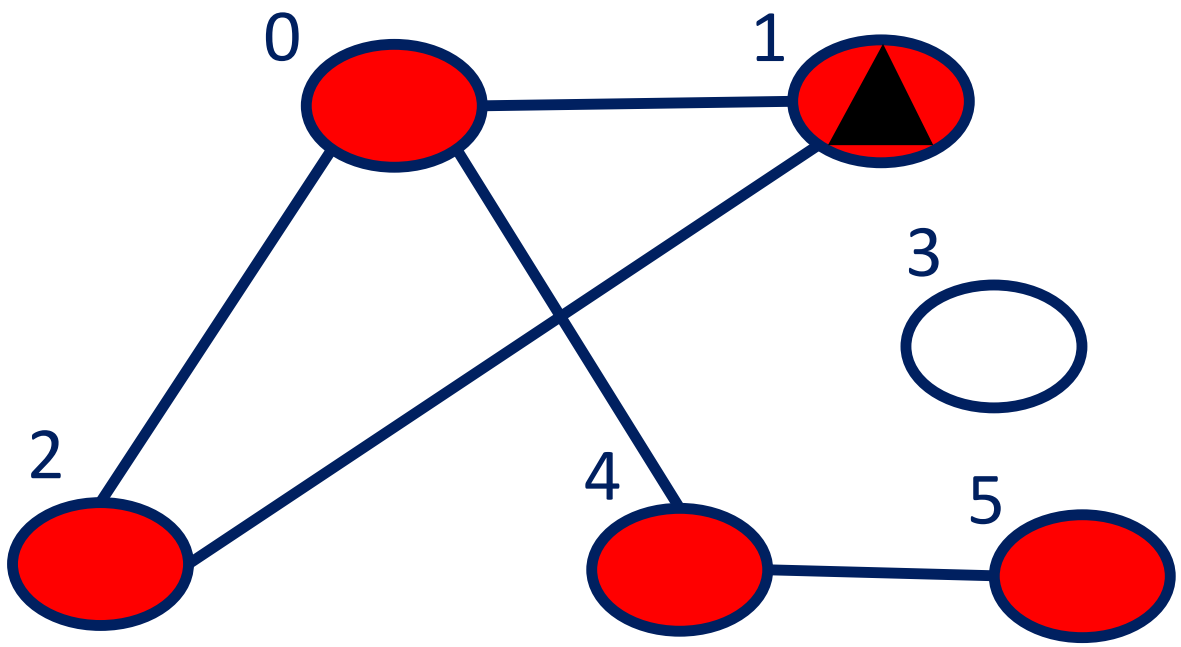
Depth first search: updated version



No! => Make it visited and add to the traversal

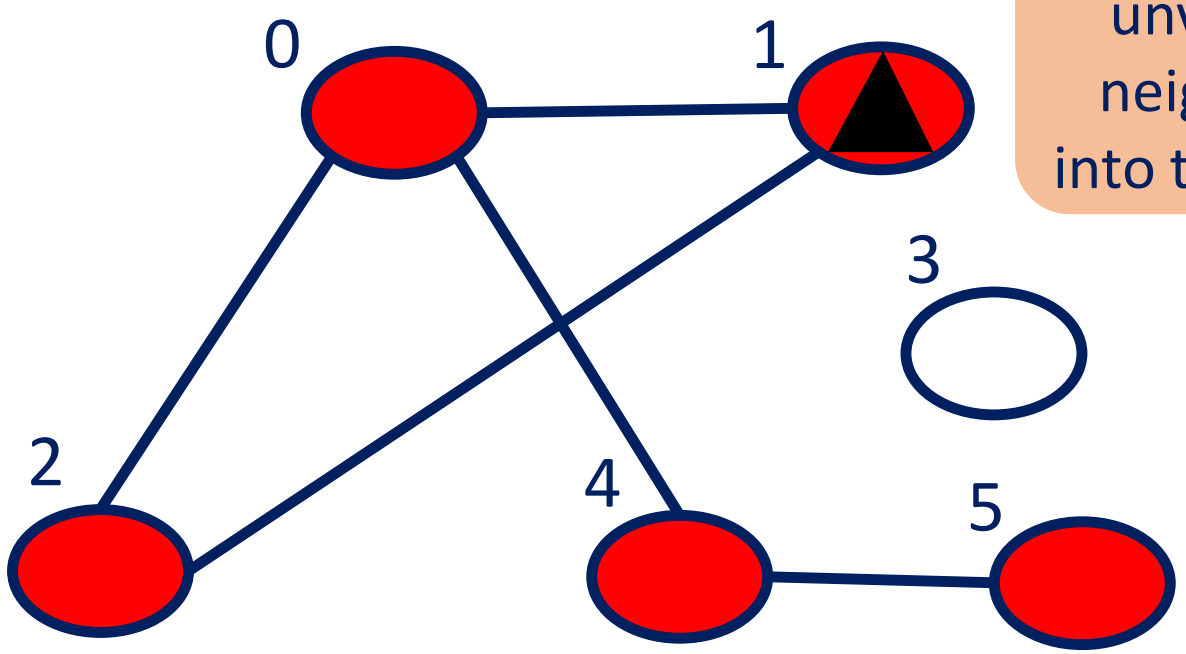
Traversal: 2,0,4,5

Depth first search: updated version



Traversal: 2,0,4,5,1

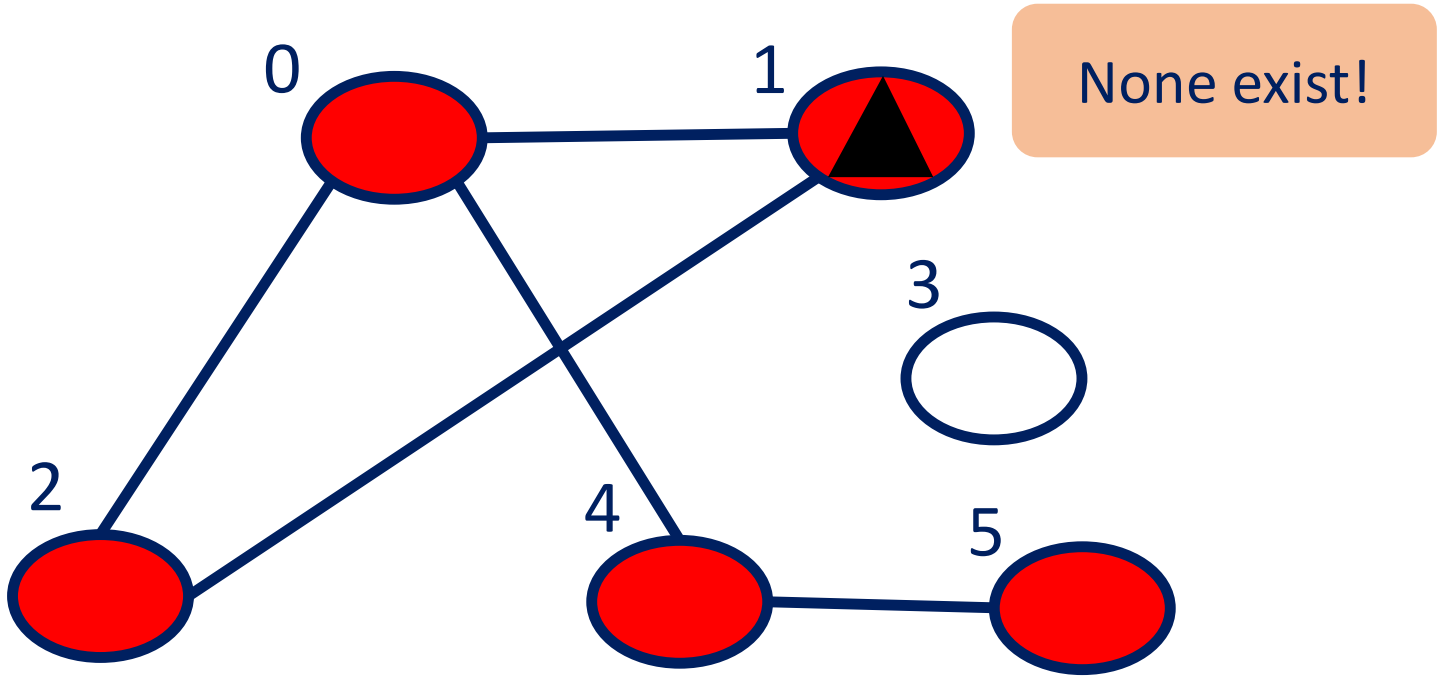
Depth first search: updated version



Push all unvisited neighbors into the stack

Traversal: 2,0,4,5,1

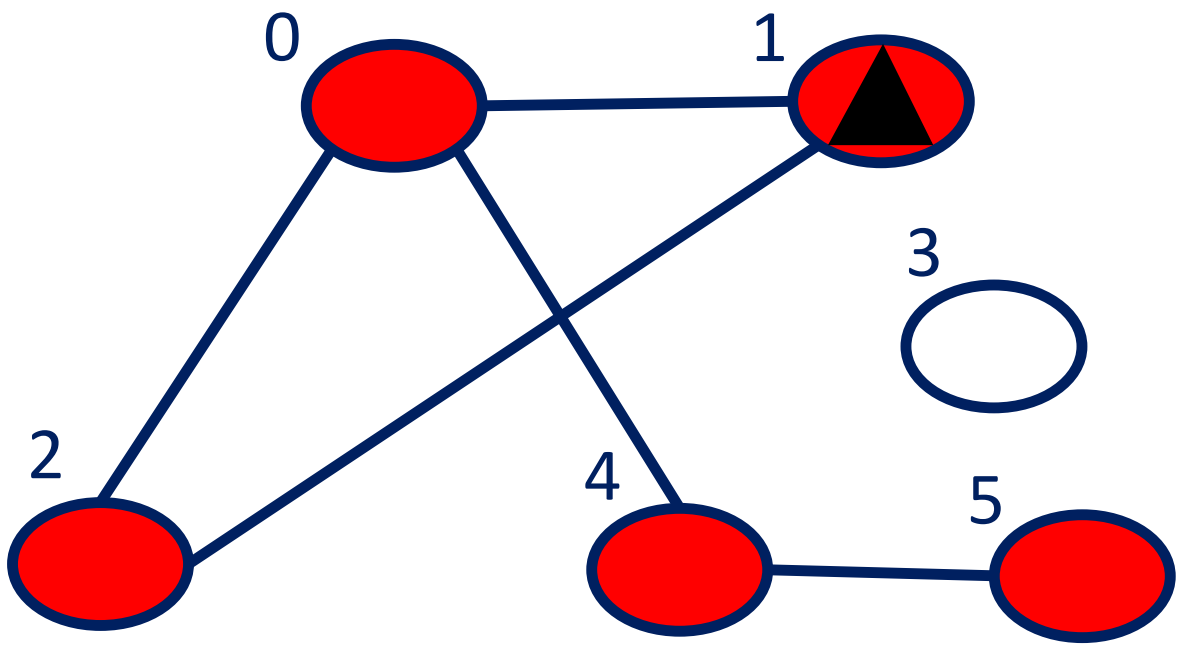
Depth first search: updated version



None exist!

Traversal: 2,0,4,5,1

Depth first search: updated version

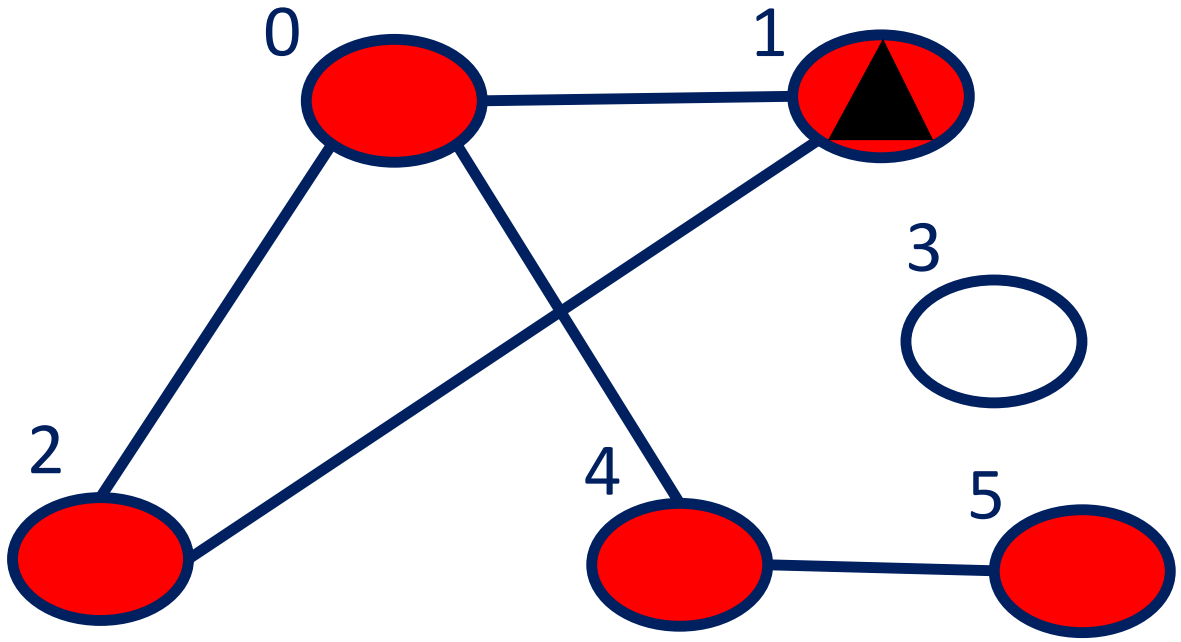


Is the stack empty?

Traversal: 2,0,4,5,1

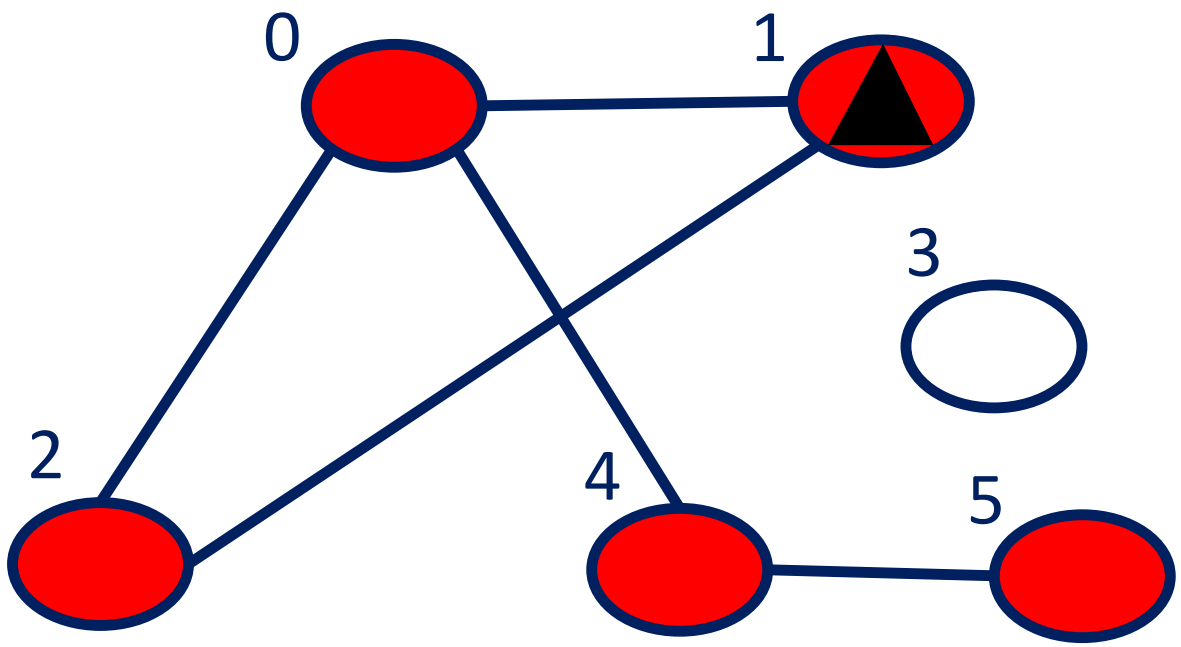
Depth first search: updated version

No! => Pop the node and make it current



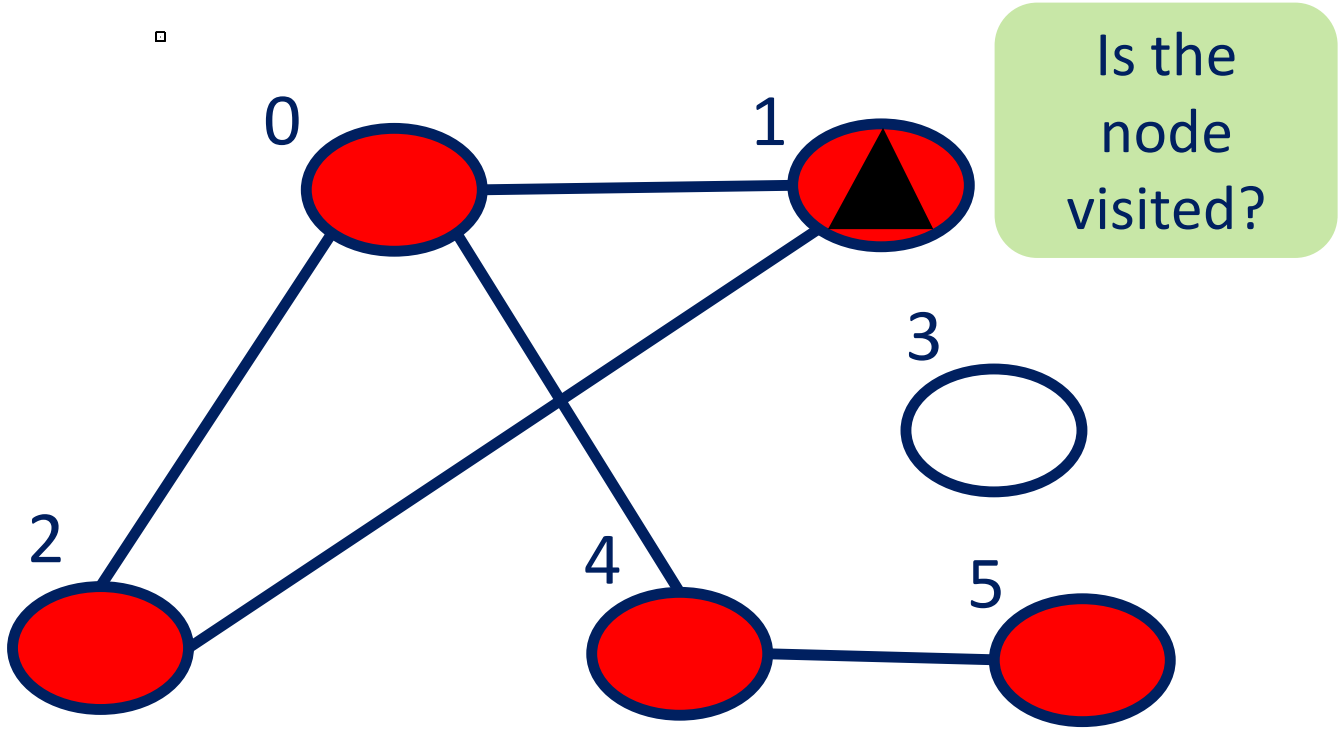
Traversal: 2,0,4,5,1

Depth first search: updated version



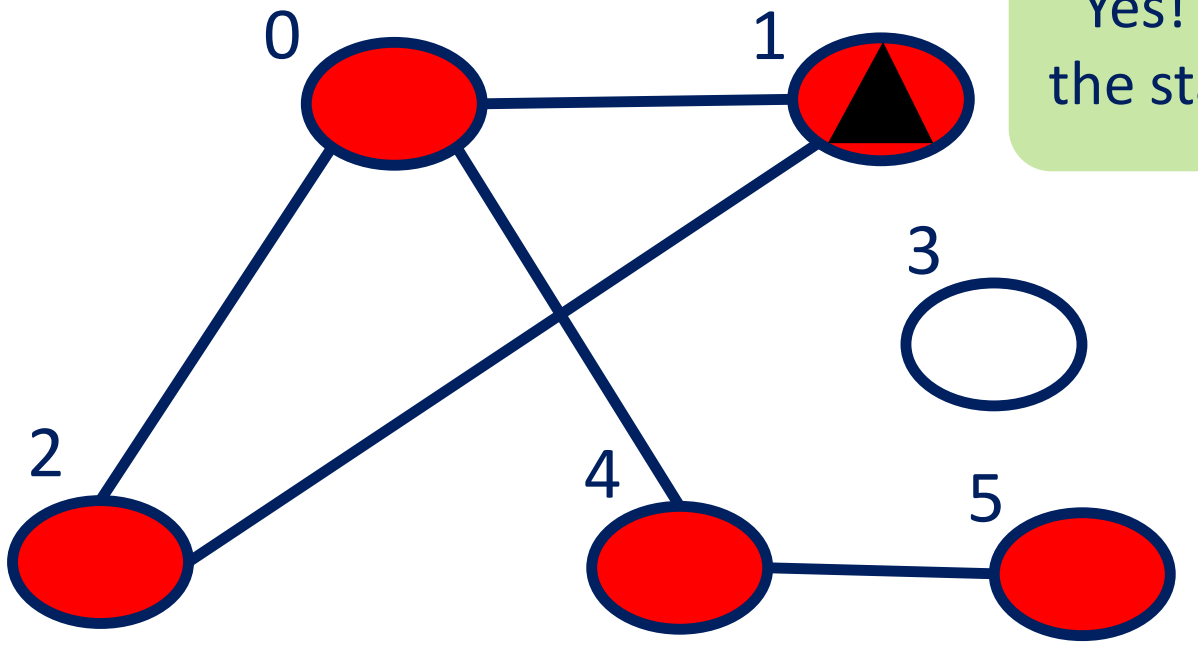
Traversal: 2,0,4,5,1

Depth first search: updated version



Traversal: 2,0,4,5,1

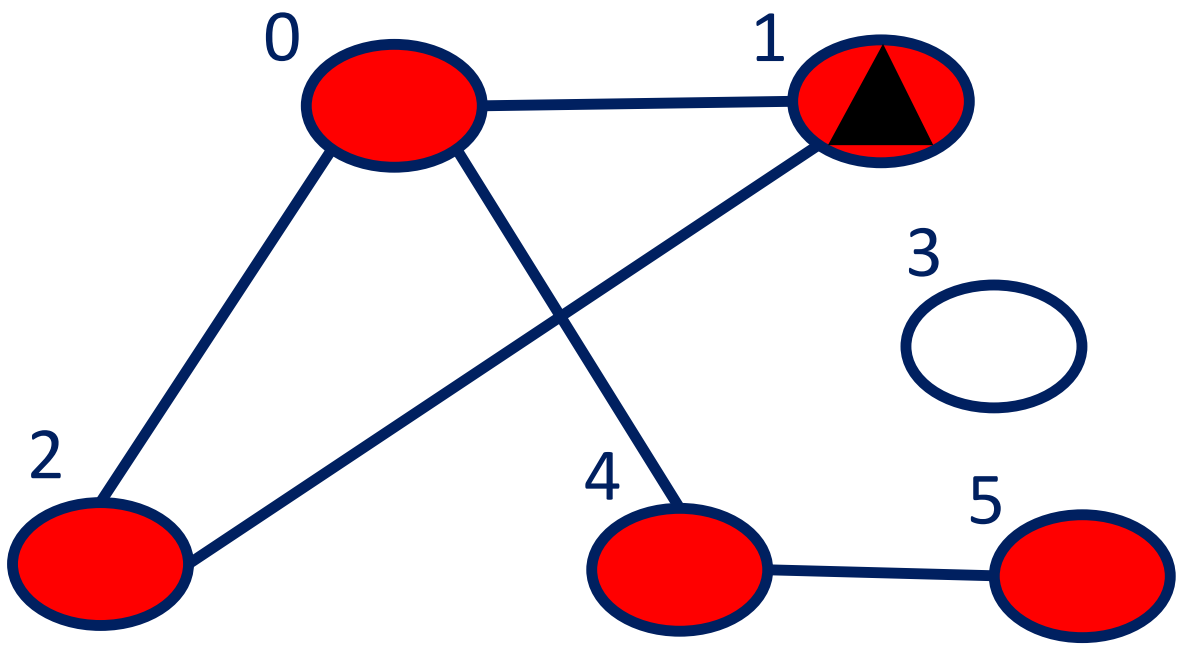
Depth first search: updated version



Yes! => Check if the stack is empty

Traversal: 2,0,4,5,1

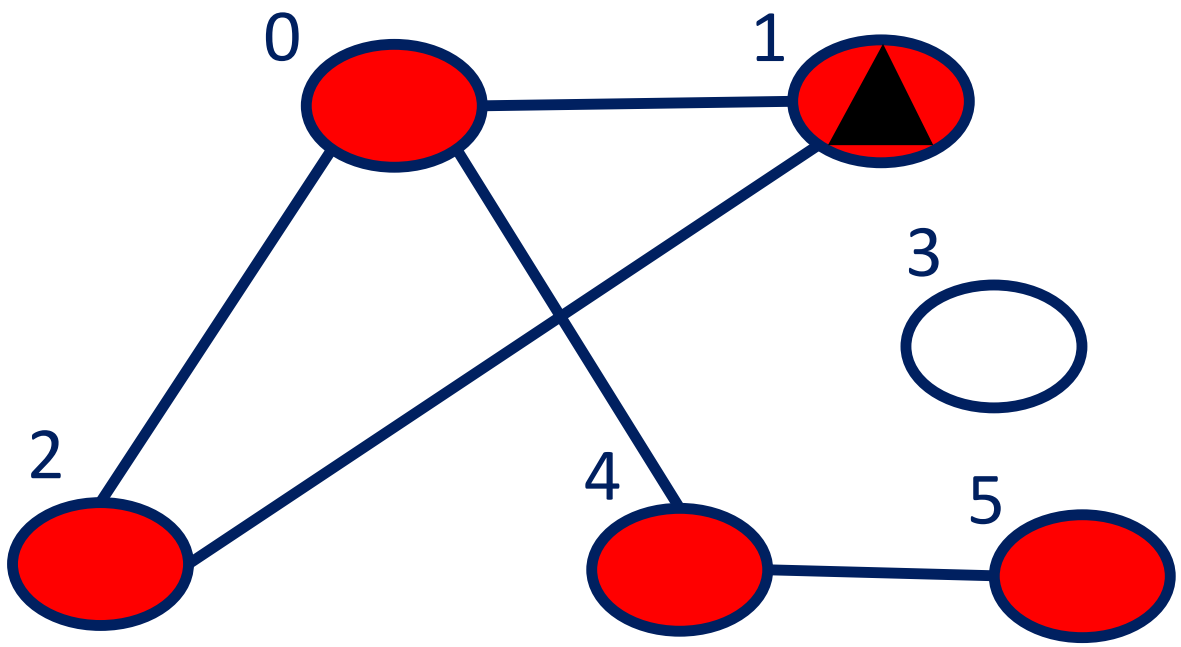
Depth first search: updated version



Is the stack empty?

Traversal: 2,0,4,5,1

Depth first search: updated version

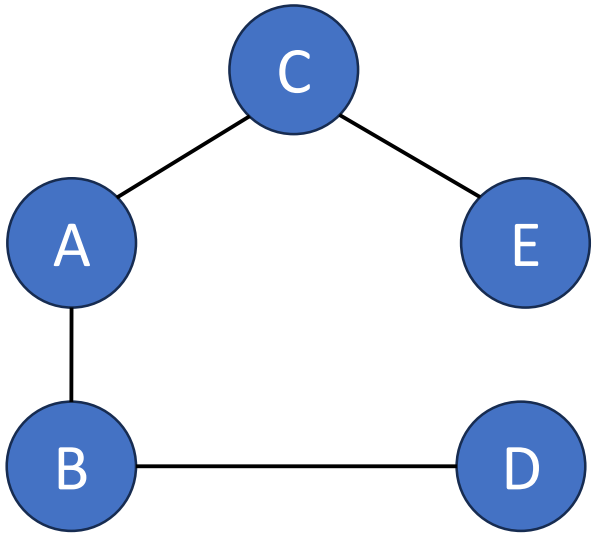


Yes! => STOP

Traversal: 2,0,4,5,1

```python
from collections import deque

def dfs_iterative(self, start):
    if start not in self.adj:
        return

    res = []
    visited = set()
    stack = deque([start])  # using deque as a stack

    while stack:
        current = stack.pop()
        if current not in visited:
            visited.add(current)
            res.append(current)
            for neighbor in self.adj[current]:
                stack.append(neighbor)
    print(res)
```

```
# Driver code
graph = Graph()
for label in ['A','B','C','D','E']:
    graph.add_node(label)

graph.add_edge('A','B')
graph.add_edge('A','C')
graph.add_edge('B','D')
graph.add_edge('C','E')
print('Iterative DFS traversal result (start=B):')
graph.dfs_iterative('B')
```



Attendance